

使用手册

1. 关于本手册

本手册面向于NJET应用引擎的非管理员用户，即属于普通角色的用户，主要包括各个功能模块的操作指南。

2. NJET应用引擎介绍

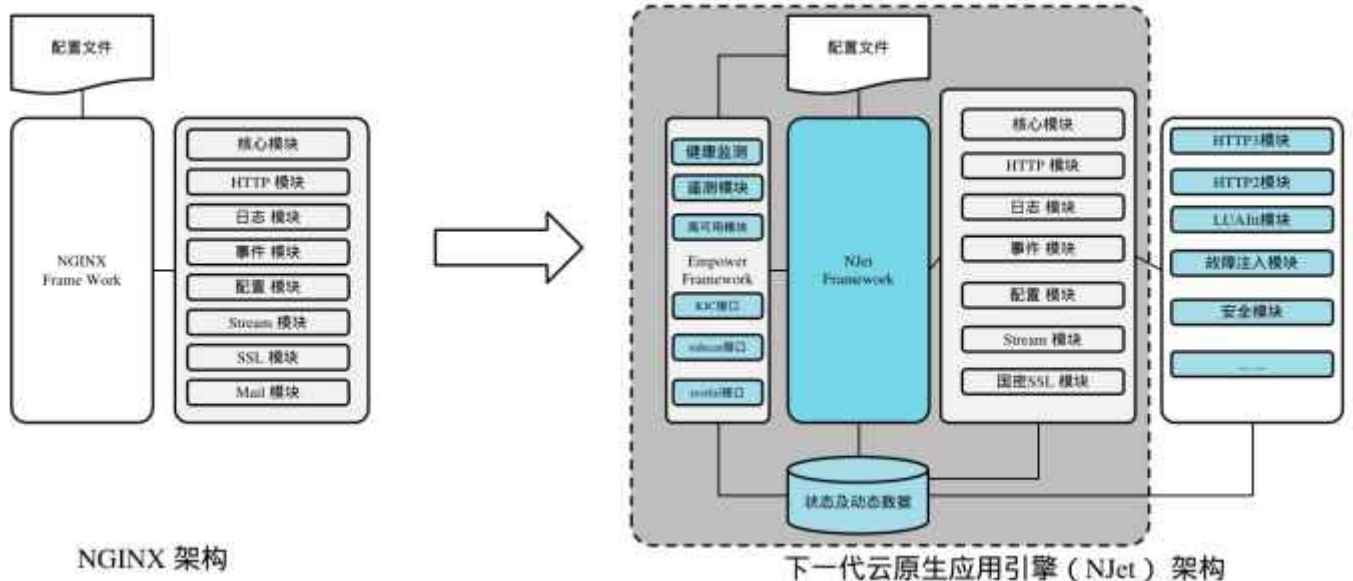
2.1 应用引擎技术介绍

应用引擎是面向互联网和云原生应用提供的运行时组态服务程序。具备环境感知、安全控制、加速优化等能力，一般呈现为Web服务、流媒体服务、代理(Proxy)、应用中间件、API网关、消息队列等产品形态。

应用引擎在云原生架构中，除了提供南北向通信网关的功能以外，因为提供了服务网格中东西向通信、透明流量劫持、熔断、遥测与故障注入等新功能特性，其地位和作用在云原生架构中变得愈发重要。

NJet最早是基于NGINX1.19基础，fork并独立演进的开源应用引擎，并随着NGINX版本迭代，吸收上游NGINX的更新，已经同步更新到NGINX1.23.1版本。NJet的目标在于适应国内特定的技术规范及标准，如国密算法套件支持，并构建安全可控的云原生数据面，支撑我国云原生产业生态。作为底层引擎，NJet利用动态加载机制可以实现不同的产品形态，如API网关、消息代理、出入向代理，负载均衡，WAF等等

2.2 系统架构介绍



相比市面其他类型的API网关，高性能是NGINX主要的优点，但动态配置能力的缺乏一直受到业界的诟病。NJet在NGINX的架构上进行了扩充，对其框架进行了改写，增加了copilots framework及可持久化的动态存储能力，解决了指令配置变更动态生效的关键问题，扩展了NJet的应用场景。此外，业界对应用引擎可观测性的需求，需要应用引擎持续不断的采集性能指标、日志数据以及注入跟踪信息，但这对应用引擎的性能造成了不可忽视的影响，NJet利用copilots framework隔离了业务处理及配置变更和指标采集，避免了遥测对性能的影响。作为云原生的应用引擎，NJet需要支持业界流行的Ingress及sidecar的api规范，基于动态配置+ copilots framework架构，NJet可以通过不断更新独立的相关copilots module，实现对响应标准规范的及时支持

2.3 njet编译与安装

2.3.1 编译环境：

- i. 准备一台能够访问外网的
- ii. CentOS Linux release 7.9.2009 (Core)

2.3.2 编译环境配置

2.3.2.1 配置yum源：

1. 执行以下指令：sudo yum --enablerepo=extras install -q -y epel-release centos-release-scl-rh <https://repo.ius.io/ius-release-el7.rpm>
2. sudo curl -o /etc/yum.repos.d/mercurial.repo <https://www.mercurial-scm.org/release/centos7/mercurial.repo>
3. 上面步骤完成后，文件系统的目录/etc/yum.repos.d 将生成对应的repo文件。
4. [root@CDN102 home]# ls -al /etc/yum.repos.d/mercurial.repo

5. -rw-r--r--. 1 root root 267 3月 7 11:26 /etc/yum.repos.d/mercurial.repo

2.3.2.2 yum 安装软件包：

1. `sudo yum install -y devtoolset-8-make devtoolset-8-toolchain ca-certificates mercurial zlib-devel cmake3 ninja-build libunwind-devel pcre-devel openssl-devel libtool libtool-ltdl`

2.3.2.3 创建符号连接：

1. `sudo ln -s /opt/rh/devtoolset-8/root/usr/bin/gcc /usr/local/bin/gcc`
2. `sudo ln -s /opt/rh/devtoolset-8/root/usr/bin/c++ /usr/local/bin/c++`
3. `sudo ln -s /opt/rh/devtoolset-8/root/usr/bin/cc /usr/local/bin/cc`
4. `sudo ln -s /opt/rh/devtoolset-8/root/usr/bin/make /usr/local/bin/make`

2.3.2.4 修改 ld.so.conf 配置

执行以下命令：

```
sudo bash -c 'echo "/usr/local/lib" >> /etc/ld.so.conf'
```

```
sudo ldconfig
```

2.3.3 编译代码

把 njet1.0.zip 包上传到 /home 目录下。并解压 `unzip njet1.0.zip`，如图：

```
[root@CDN102 home]# ls
njet1.0  njet1.0.zip
[root@CDN102 home]# cd njet1.0/
[root@CDN102 njet1.0]# ls
3rd_lib  build      build_k8s.sh  CHANGES.ru  configure  html      luajit  man      mosquito  objs      README      src
auto     build_cc.sh  CHANGES     conf          contrib    LICENSE   luaLib  modules  njet_sidecar.conf  openapi  README.md  telemetry
```

2.3.3.1 编译：njet

2.3.3.1.1 执行： `sh build_cc.sh conf`

```

+ njet_http_kv_module was configured
adding module in ./modules/njet-http-sendmsg-module
+ njet_http_sendmsg_module was configured
adding module in ./modules/njet-http-split-clients-2-module
+ njet_http_split_clients_2_module was configured
adding module in ./modules/njet-health-check-helper
checking for njet_health_check_helper ... found
+ njet_health_check_helper was configured
checking for zlib library ... found
creating objs/Makefile

Configuration summary
+ using PCRE library: auto/lib/pcre-8.45
+ using OpenSSL library: auto/lib/tassl
+ using system zlib library

njet path prefix: "/etc/njet"
njet binary file: "/usr/sbin/njet"
njet modules path: "/usr/lib/njet/modules"
njet configuration prefix: "/etc/njet"
njet configuration file: "/etc/njet/njet.conf"
njet pid file: "/etc/njet/logs/njet.pid"
njet error log file: "/etc/njet/logs/error.log"
njet http access log file: "/etc/njet/logs/access.log"
njet http client request body temporary files: "client_body_temp"
njet http proxy temporary files: "proxy_temp"
njet http fastcgi temporary files: "fastcgi_temp"
njet http uwsgi temporary files: "uwsgi_temp"
njet http cgi temporary files: "cgi_temp"

[root@CDN102 njet1.0]#

```

2.3.3.1.2 执行：make

如果make后，有如下提示：则继续执行 make 指令。

```

Using OS-specific seed configuration
Creating configdata.pm
Creating Makefile

*****
***
***      OpenSSL has been successfully configured      ***
***
***      If you encounter a problem while building, please open an ***
***      issue on GitHub <https://github.com/openssl/openssl/issues> ***
***      and include the output from the following command: ***
***
***      perl configdata.pm --dump ***
***
***      (If you are new to OpenSSL, you might want to consult the ***
***      'Troubleshooting' section in the INSTALL file first) ***
***
*****
*****
***
***      Please run the same make command again ***
***
*****
make[2]: *** [Makefile:690: configdata.pm] 错误 1
make[2]: 离开目录"/home/njet1.0/auto/lib/tassl"

```

2.3.3.1.3 正确编译完成：如图：

```

/usr/bin/cc -c -fPIC -I/etc/njet/luajit/include/luajit-2.1 -I/etc/njet/luajit/include/luajit-2.1 -pipe -O -w -Wall -Wpointer-arith -Wno-unu-
ter -Werror -g -O0 -ggdb -DNDK_SET_VAR -I src/core -I src/event -I src/event/modules -I src/os/unix -I ./modules/njet-stream-proto-module -I ./
a/kit/objs -I objs/addon/ndk -I src/ext/luah/http/src/api -I src/ext/luah/stream/src/api -I ./modules/njet-stream-upstream-dynamic-servers-modu-
les/njet-http-upstream-dynamic-servers-module -I ./modules/njet-http-match-module -I ./modules/njet-http-health-check-module -I ./modules/nj-
tick-module -I ./modules/njet-mqconf-module -I ./modules/njet-location-module/src -I ./modules/njet-http-kv-module/src -I ./modules/nj-
ndmsg-module/src -I auto/lib/pcre-8.45 -I auto/lib/tassl/include -I objs -I src/http -I src/http/modules -I src/http/v2 -I src/ext/li-
-I src/ext/luah/kit/src -I src/ext/luah/kit/objs -I objs/addon/ndk -I /etc/njet/luajit/include/luajit-2.1 -I src/mail -I src/stream -I /etc/nj-
include/luajit-2.1 -I ./modules/njet-stream-health-check-module/src \
-o objs/njt_helper_health_check_agent_modules.o \
objs/njt_helper_health_check_agent_modules.c
/usr/bin/cc -o objs/njt_helper_health_check_agent.so \
objs/addon/src/njt_helper_health_check_agent.o \
objs/addon/src/njt_common_health_check.o \
objs/njt_helper_health_check_agent_modules.o \
-L/etc/njet/luajit/lib -L/etc/njet/luajit/lib -L./3rd_lib -ldl \
-shared
sed -e "s|%%PREFIX%%|/etc/njet|" \
-e "s|%%PID_PATH%%|/etc/njet/logs/njet.pid|" \
-e "s|%%CONF_PATH%%|/etc/njet/njet.conf|" \
-e "s|%%ERROR_LOG_PATH%%|/etc/njet/logs/error.log|" \
< man/njet.8 > objs/njet.8
make[1]: 离开目录"/home/njet1.0"
[root@CDN102 njet1.0]#

```

2.3.3.1.4 最后执行：make install

3. 系统功能

1.0目标是实现基本动态框架，为动态配置打下基础，全面改造SSL支持，从而支持国密，DTLS.初步实现sidecar的xDS接口，可作为边车应用在服务网格中。共涉及引擎内核，边车、出入口网关、

web引擎、应用代理、安全、监控、可编程控制、api网关等4大类38项功能。在下述功能清单中，着重描述扩展、定制的功能，原生NGINX的能力可以参考 <https://nginx.org/en/docs>

3.1 基本使用

3.1.1 目录结构及功能说明

1	— 3rd_lib	依赖的第三方动态库，比如libmosquitto_emb.so
2	— build/rpm	rpm包编译脚本
3	— auto	自动检测系统环境以及编译相关的脚本
4	— cc	关于编译器相关的编译选项的检测脚本
5	— lib	njet编译所需要的一些库的检测脚本
6	— os	与平台相关的一些系统参数与系统调用相关的检测
7	— types	与数据类型相关的一些辅助脚本
8	— conf	存放默认配置文件，在make install后，会拷贝到安装目录中去
9	— contrib	存放一些实用工具，如geo配置生成工具（geo2njet.pl）
10	— html	存放默认的网页文件，在make install后，会拷贝到安装目录中去
11	— repos	存放yum数据源
12	— doc	njet的api文档
13	— swagger	openapi 接口网页文档
14	— gui	前端展示页面文档
15	— manual	njet文档手册
16	— luajit	luajit
17	— lualib	lualib
18	— modules	njet动态模块以及util模块
19	— openapi	openapi 定义文件
20	— src	存放njet的源代码
21	— core	njet的核心源代码，包括常用数据结构的定义，以及njet初始化运行
22	— event	对系统事件处理机制的封装，以及定时器的实现相关代码
23	— modules	不同事件处理方式的模块化，如select、poll、epoll、kqueue等
24	— http	njet作为http服务器相关的代码
25	— modules	包含http的各种功能模块
26	— ext/lua	lua模块
27	— mail	njet作为邮件代理服务器相关的代码
28	— stream	tcp/ udp 四层网络代理服务器相关的代码
29	— misc	一些辅助代码，测试 c++ 头的兼容性，以及对google_perftools
30	— os	主要是对各种不同体系系统结构所提供的系统函数的封装，对外提供统一

3.1.2 使用命令

3.1.2.1 显示帮助信息

njet -h

3.1.2.2 启动

njet -p /tmpr/njet/ -c conf/njet.conf

常见启动参数：

-p 指定prefix配置文件路径，不指定，默认/etc/njet

-c 指定配置文件，不指定，默认njet.conf

-e 指定error 日志文件

3.1.2.3 测试配置信息是否有错误

njet -t

3.1.2.4 显示版本

njet -v

3.1.2.5 显示编译阶段的参数

njet -V

3.1.2.6 快速停止

njet -s stop 或者 kill -TERM {进程id}

3.1.2.7 优雅停止服务

njet -s quit 或者 kill -QUIT {进程id}

3.1.2.8 重新加载配置

njet -s reload 或者 kill -HUP {进程id}

3.2 CoPilot

copilot类似于NGINX中的helper，但通过配置helper指令，实现装载不同的helper实现。从而实现不同的功能。目前的NJet的事件总线、控制平面集成接口都是通过其设置

copilot框架依赖mqconf提供的helper指令：

指令说明：

Syntax	helper tag so_file conf_file;

Default	---
Context	main

参数说明：

参数名称	是否必须	参数说明
tag	是	指定一个名字标签
so_file	是	so文件
conf_file	是	配置文件

3.2.1 copilot:ctrl

该模块是一个基础模块，给 http-sendmsg 和 http_health_check 等模块提供运行环境。

为了运行该模块，需要在主配置文件中配置：

下面为示例配置：

```
helper ctrl objs/njt_helper_ctrl_module.so njt_ctrl.conf;
```

该模块自身的配置，即上面主配置文件中helper指令中指定的配置文件的内容。

配置的指令分为两类：

1) Njet中标准的指令；

1. **需要注意：**用 `error_log` 和 `access_log` 指令指定的log文件要不同于主配置文件中的log文件。

2) 通过 `load_module` 指令加载的动态模块中扩展出来的指令

配置示例如下：

```
1 load_module /home/njet/modules/njt_http_sendmsg_module.so;
2 load_module /home/njet/modules/njt_http_health_check_helper.so;
3 user njet njet;
4
5 error_log logs/error_ctrl.log info;
6
7 events {
8     worker_connections 1024;
```

```

9  }
10
11 http {
12     dyn_sendmsg_conf conf/iot_msg.conf;
13     access_log logs/access_ctrl.log combined;
14
15     server {
16         listen      8443;
17
18         location / {
19             return 200 "NJet control_plane stub\n";
20         }
21         location /hc {
22             health_check_api;
23         }
24     }
25 }

```

3.2.2 copilot:broker

该模块提供消息服务端功能，要开启该功能在njet.conf 的 main block 中添加如下指令：

helper broker /home/njet/modules/njt_helper_broker_module.so conf/mqtt.conf;

mqtt.conf Message Broker 配置文件

配置项	必须修改	配置说明
log_dest	否	日志输出方法，一般配置为文件输出，配置值为file 文件 默认： \$PREFIX/logs/mosquitto.log
log_type	否	日志级别： debug, error, warning, notice, information java 中的不一样，配置了某个日志级别，只有该日志级别 需要为需要的日志级别单独配置一行。默认： error
listener	否	监听端口及地址，默认： 0 \$PREFIX/data/mosquitto.sc
allow_anonymous	否	是否允许匿名连接，目前配置为true
persistence	否	是否开启消息持久化（true, false), 默认： true
autosave_on_changes	否	是否有消息时进行自动保存 默认： true
autosave_interval	否	当自动保存开启时，当新消息个数大于这个配置值时，

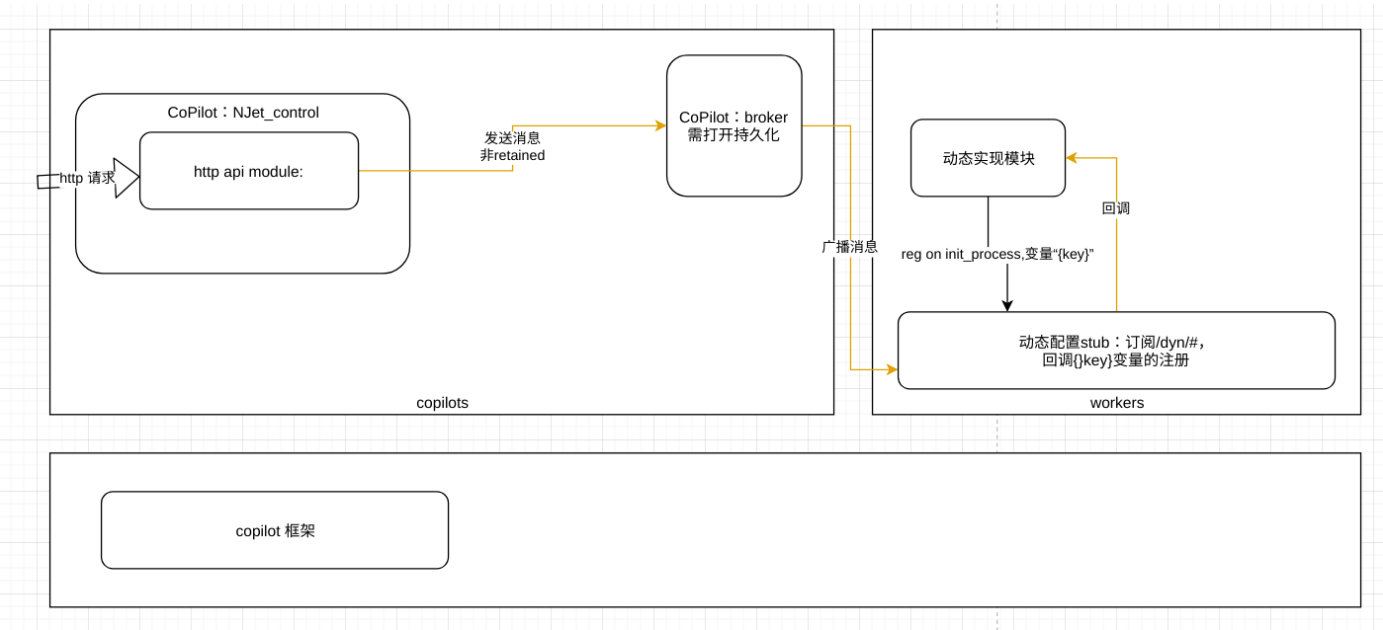
persistence_location	否	持久化文件保存路径，将在该路径下创建 mosquitto.db
----------------------	---	---------------------------------

配置都会有默认值，建议只配置日志级别。

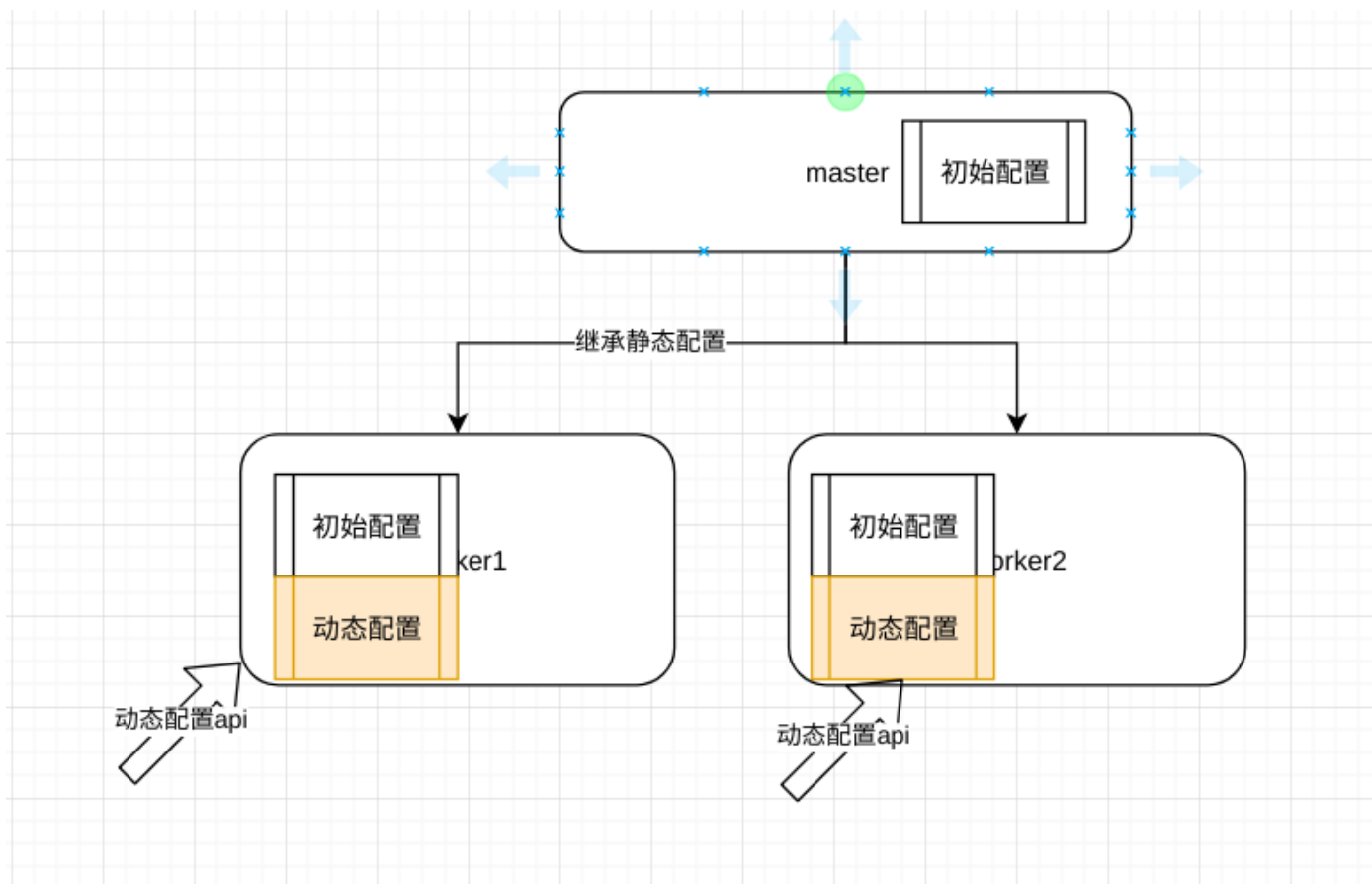
```
1 log_type error
```

3.3 动态配置框架

njet提供基于eventbus构建的消息总线，其包括copilot: broker，动态配置stub模块 njet_http_kv,sendmsg模块 njet_http_sendmsg模块。



如上图所示，NJet的动态配置一般由控制面发送配置消息，不同的worker通过加载动态配置stub，监听不同的消息，并回调不同的处理函数。



3.3.1 kv模块配置

在 http block 下，指定该模块的配置文件：

```

1  http {
2      dyn_kv_conf conf/iot.conf;
3  }

```

iot.conf MQTT 客户端配置

配置项	必须修改	配置说明
broker_addr	否	MQ broker 地址。默认： unix:\$PREFIX/data/mo
log_type	否	日志级别： debug, error, warning, notice, infor 一行。默认： error
topic	否	默认订阅的主题 /cluster+/kv_set/# 用于kvstore 值的设置 /dyn/#

		用于动态 location 更新 /rpc/# 用于发送rpc消息
keepalive	否	给服务端发送PING命令的间隔时间。默认：30
kv_store_dir	否	Key value store 持久化文件路径。默认：\$PREF
protocol_version	否	使用的协议版本，要支持 request response消息，必须填 5。默认：5

配置都会有默认值，建议只配置日志级别

```
1 log_type error
```

3.3.2 Sendmsg

要开启该功能, 需在njet_ctrl.conf 的 main block 中加载该模块：

```
load_module /home/njet/modules/njt_http_sendmsg_module.so;
```

并在 http block 下，指定该模块的配置文件：

```
1 http {
2     dyn_sendmsg_conf conf/iot_ctrl.conf;
3     server {
4         ...
5         location /kv {
6             dyn_sendmsg_kv;
7         }
8         ...
9     }
10 }
```

dyn_sendmsg_kv 配置不是必须的， 这个是对外提供了 kv 值设置及查询的 http API 接口。

iot_ctrl.conf MQTT 客户端配置

配置项	必须修改	配置说明

broker_addr	否	MQ broker 地址. 默认: unix:\$PREFIX/data/mossc
log_type	否	日志级别: debug, error, warning, notice, inform 一行。默认: error
keepalive	否	给服务端发送PING命令的间隔时间. 默认: 30
kv_store_dir	否	Key value store 持久化文件路径. 默认: \$PREFIX,
protocol_version	否	使用的协议版本, 要支持 request response消息, 必须填 5. 默认: 5

配置都会有默认值, 建议只配置日志级别

```
1 log_type error
```

3.4 国密支持

3.4.1 功能说明

使用国密版OpenSSL库来使用SM2、SM3、SM4算法（以下简称为SM234算法）。
SM234算法使用双证书, 包括签名证书和加密证书, 这是国密支持在证书上的一个特点。
同时, 还要求使用的国密版OpenSSL兼容标准OpenSSL。
根据NJet的使用场景, 国密支持的场景也细分为Server和反向代理两种场景。

3.4.2 配置说明

NJet作为Server时国密支持需要增加双证书的配置:

```
1      server {
2          listen      443 ssl;
3          server_name localhost;
4
5          # 原RSA证书
6          ssl_certificate      /home/certs/cert/sslserver.crt;
7          ssl_certificate_key  /home/certs/cert/sslserver.key;
8
9          # 国密支持添加的证书
10         ssl_certificate      /home/certs/certgm/serversign.crt;
11         ssl_certificate_key  /home/certs/certgm/serversign.key;
```

```

12         ssl_certificate      /home/certs/certgm/serverenc.crt;
13         ssl_certificate_key  /home/certs/certgm/serverenc.key;
14
15         location / {
16             root    html;
17             index   index.html index.htm;
18         }
19     }

```

NJet作为反向代理时，增加以下三个指令：

`proxy_ssl_gm`指令，用于NJet连接后端服务器时，指定是否使用国密。

```

1 Syntax:      proxy_ssl_gm on | off;
2 Default:    proxy_ssl_gm off
3 Context:    http, server, location

```

`proxy_ssl_enc_certificate`指令，用于NJet连接后端服务器时，指定加密证书文件。

```

1 Syntax:      proxy_ssl_enc_certificate file;
2 Default:    -
3 Context:    http, server, location

```

`proxy_ssl_enc_certificate_key`指令，用于NJet连接后端服务器时，指定加密密钥文件。

```

1 Syntax:      proxy_ssl_enc_certificate_key file;
2 Default:    -
3 Context:    http, server, location

```

配置示例：

```

1     server {
2         listen      8000;
3         server_name localhost;
4
5         proxy_ssl on;
6         proxy_ssl_gm on;
7         proxy_ssl_ciphers ECDHE-SM4-SM3; #也可以指定ECC-SM4-SM3国密套件

```

```
8
9     proxy_ssl_certificate      /home/certs/CS.crt;
10    proxy_ssl_certificate_key  /home/certs/CS.key;
11    proxy_ssl_enc_certificate  /home/certs/CE.crt;
12    proxy_ssl_enc_certificate_key /home/certs/CE.key;
13
14    # 其他配置，例如：
15    location / {
16        proxy_pass https://192.168.183.111:443;
17    }
18 }
```

3.4.3 功能验证

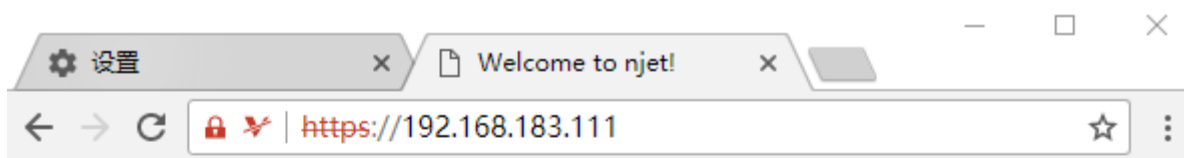
3.4.3.1 验证服务器对国密的支持

验证工具为分别为密信浏览器和gmcurl。

密信浏览器：



使用密信浏览器访问server：



Welcome to njet!

If you see this page, the njet web server is successfully installed and working. Further configuration is required.

Thank you for using njet.

使用gmcurl访问server:

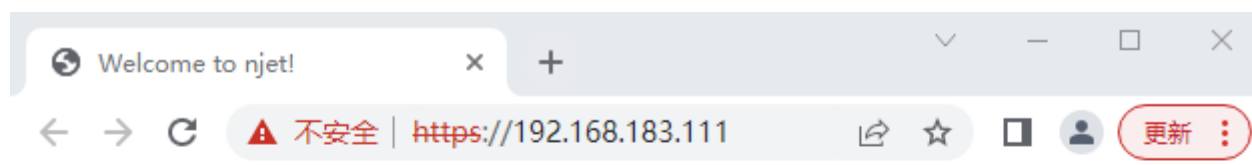
```
[root@112 ~]# ./gmcurl --gmssl -k https://192.168.183.111:443
GM Version: 1.0.0 Ported by www.gmssl.cn
GM options:
--gmssl, use TLCP protocol
--cert,  use sm2 sig pem cert
--key,   use sm2 sig pem key
--cert2, use sm2 enc pem cert
--key2,  use sm2 enc pem key
<!DOCTYPE html>
<html>
<head>
<title>Welcome to njet!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to njet!</h1>
<p>If you see this page, the njet web server is successfully installed and
working. Further configuration is required.</p>

<p><em>Thank you for using njet.</em></p>
</body>
</html>
```

3.4.3.2 验证服务器对RSA证书的支持

上节的server的配置中分别指定了RSA证书和国密证书，再验证一下服务器对RSA证书的支持。验证工具分别为Chrome浏览器和curl。

使用Chrome浏览器访问server:



Welcome to njet!

If you see this page, the njet web server is successfully installed and working. Further configuration is required.

Thank you for using njet.

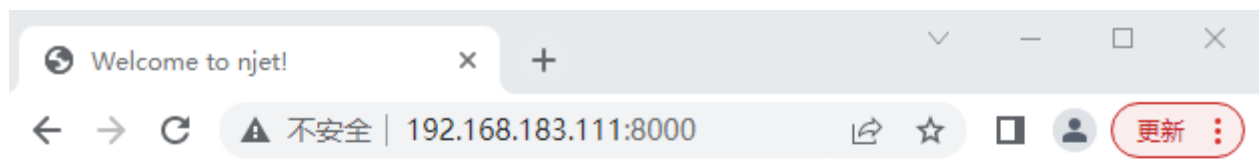
使用curl访问server:

```
[root@112 ~]# curl -k https://192.168.183.111:443
<!DOCTYPE html>
<html>
<head>
<title>Welcome to njet!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to njet!</h1>
<p>If you see this page, the njet web server is successfully installed and
working. Further configuration is required.</p>

<p><em>Thank you for using njet.</em></p>
</body>
</html>
```

3.4.3.3 后端upstream使用国密验证

NJet作反向代理访问后端国密upstream服务器。验证工具为分别为Chrome浏览器和curl。



Welcome to njet!

If you see this page, the njet web server is successfully installed and working. Further configuration is required.

Thank you for using njet.

```
[root@112 ~]# curl -k 192.168.183.111:8000
<!DOCTYPE html>
<html>
<head>
<title>Welcome to njet!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to njet!</h1>
<p>If you see this page, the njet web server is successfully installed and
working. Further configuration is required.</p>

<p><em>Thank you for using njet.</em></p>
</body>
</html>
```

3.5 动态Location

3.5.1 功能说明

3.5.2 配置说明

配置样例

conf/njet.conf

```
user root root;
```

```
worker_processes 3;
```

```
load_module /home/njet/modules/njt_http_location_module.so;
```

```
error_log /home/njet/logs/error.log debug;
```

```

events {
    worker_connections 1024;
}

http {
    dyn_kv_conf conf/iot.conf;
    #dyn_sendmsg_conf conf/iot.conf;
    error_log /home/njet/logs/error.log debug;
    access_log /home/njet/logs/access.log combined;
    upstream http1{
        zone upstream_http 64k;
        server 192.168.40.139:9002 route=a;
        server 192.168.40.139:9003 route=b;
    }
    server {
        listen 8000;
        location =/ {
            return 200 "welcome to main http server\n";
        }
        #return 200 "default";
    }
}

helper ctrl /home/njet/modules/njt_helper_ctrl_module.so
/home/njet/conf/njet_ctrl.conf;

helper broker /home/njet/modules/njt_helper_broker_module.so
conf/mqtt.conf;

cluster_name helper;

node_name node1;

```

conf/njet_ctrl.conf

```

load_module /home/njet/modules/njt_http_sendmsg_module.so;
load_module /home/njet/modules/njt_http_location_api_module.so;

```

```

user njet njet;

events {

    worker_connections 1024;

}

http {

    dyn_sendmsg_conf conf/iot_ctrl.conf;

    error_log /home/njet/logs/error_ctrl.log debug;

    access_log /home/njet/logs/access_ctrl.log combined;

    upstream http1{

        zone upstream_http 64k;

        server 192.168.40.139:9002 route=a;

        server 192.168.40.139:9003 route=b;

    }

    server {

        listen 8443;

        location /add_location {

            dyn_location_api;

        }

    }

}

cluster_name helper;

node_name node1;

```

conf/mqtt.conf

```
log_type error
```

conf/iot.conf

```
log_type error
```

conf/iot-ctrl.conf

```
log_type error
```

3.5.3 API 说明:

配置项	必填	配置说明
type	是	“add” 添加location, “del” 删除location
addr_port	是	添加的主机的, port 端口。 例如: "192.168.40.203: 8000"
server_name	否	主机的server_name, 例如: " cluster.tmlake.com "
location_rule	否	Location 的表达式, 例如: “=” 或 其他的正则式
location_name	是	location的名字, 也就是表达式后面的。例如: “/” 或 ‘
location_body	否	Location {} 中的内容。例如 “return 200 123 “, 证书location_body , proxy_pass 必须有一个。
proxy_pass	否	proxy_pass 信息, 如果后面是 upstream 名字, 则需要该u 先定义 location_body , proxy_pass 必须有一个。

3.5.4 调用样例

3.5.4.1 post 添加:

```
curl -v -s -X POST http://127.0.0.1:8443/dyn\_location -d '\
{"type":"add","addr_port":"8000","location_rule":"=","location_name":"/clb","location_bo
dy":"set $ret1 1;return 200 $ret1;","proxy_pass":"http://http1"} \
'
```

3.5.4.2 查询:

```
curl -v -s http://127.0.0.1:8000/clb
```

3.5.4.3 put 禁用删除

```
curl -v -s -X PUT http://127.0.0.1:8443/dyn\_location -d
'{"type":"del","addr_port":"8000","location_rule":"=","location_name":"/clb"}'
```

```
[root@cdn108 home]#
[root@cdn108 home]# curl -v -s -X POST http://127.0.0.1:8443/dyn_location -d '{"type":"add","addr_port":"8000","server_name":"cluster1","location_rule":"=", "location_name":"/c1b1","location_body":{"set $ret1;return 200 "new location\n";,"proxy_pass":"http://http1"}}'
* About to connect() to 127.0.0.1 port 8443 (#0)
* Trying 127.0.0.1...
* Connected to 127.0.0.1 (127.0.0.1) port 8443 (#0)
> POST /dyn_location HTTP/1.1
> User-Agent: curl/7.29.0
> Host: 127.0.0.1:8443
> Accept: */*
> Content-Length: 191
> Content-Type: application/x-www-form-urlencoded
>
upload completely sent off: 191 out of 191 bytes
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Tue, 28 Feb 2023 02:16:11 GMT
< Content-Type: text/plain
< Content-Length: 7
< Connection: keep-alive
<
* Connection #0 to host 127.0.0.1 left intact
success[root@cdn108 home]#
```

添加clb1

返回new location

```
[root@cdn108 home]#
[root@cdn108 home]#
[root@cdn108 home]#
[root@cdn108 home]# curl http://127.0.0.1:8000/c1b1
new location
[root@cdn108 home]#
```

访问 clb1

```
[root@cdn108 home]#
[root@cdn108 home]# curl -v -s -X PUT http://127.0.0.1:8443/dyn_location -d '{"type":"del","addr_port":"8000","server_name":"cluster1","location_rule":"=", "location_name":"/c1b1","location_body":{"set $ret1;return 200 $ret1;,"proxy_pass":"http://http1"}}'
* About to connect() to 127.0.0.1 port 8443 (#0)
* Trying 127.0.0.1...
* Connected to 127.0.0.1 (127.0.0.1) port 8443 (#0)
> PUT /dyn_location HTTP/1.1
> User-Agent: curl/7.29.0
> Host: 127.0.0.1:8443
> Accept: */*
> Content-Length: 178
> Content-Type: application/x-www-form-urlencoded
>
upload completely sent off: 178 out of 178 bytes
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Tue, 28 Feb 2023 02:19:14 GMT
< Content-Type: text/plain
< Content-Length: 7
< Connection: keep-alive
<
* Connection #0 to host 127.0.0.1 left intact
Success[root@cdn108 home]#
```

删除clb1

```
[root@cdn108 home]#
[root@cdn108 home]#
[root@cdn108 home]#
[root@cdn108 home]# curl http://127.0.0.1:8000/c1b1
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>njet/1.23.1</center>
</body>
</html>
[root@cdn108 home]#
```

再次访问clb1

3.6 主动健康检查

3.6.1 简介

主动健康检查与被动健康检查：主动健康检查由Njet主动发起请求测试上游服务是否正常，如不正常则及时剔除对应服务；被动健康检查由Njet检测上游服务的异常返回发现服务异常，不能主动感知上游服务状态。

helper进程主动健康检查：可以实现健康检查由独立进程完成，对work进程无影响，完成后修改共享内存完成upstream状态回写。

3.6.2 指令说明

health_check_api



语法： health_check_api;

默认值： —

允许配置位置： location

在当前location开启健康检查API。

API说明

参见下文openapi 链接

<http://192.168.40.141:9000/#0>

部分api字段补充

健康检查 HTTP > status 配置方式

	A	B
1	配置内容	判定为健康的描述
2	200	http返回状态码为200
3	! 500	http返回状态码不是500
4	200 204	http返回状态码是200 或者 204
5	! 301 302	http返回状态码不是301且不是302
6	200-399	http返回状态码是200到399的，包含200和399
7	! 400-599	http返回状态码不是400到599的，包含400和599
8	301-303 307	http返回状态码是301到303的，包含301和303，或者307

- 1 配置内容判定为健康的描述
- 2 200http返回状态码为200
- 3 ! 500http返回状态码不是500
- 4 200 204http返回状态码是200 或者 204
- 5 ! 301 302http返回状态码不是301且不是302
- 6 200-399http返回状态码是200到399的，包含200和399
- 7 ! 400-599http返回状态码不是400到599的，包含400和599
- 8 301-303 307http返回状态码是301到303的，包含301和303，或者307

健康检查 http > header 配置方式

	A	B
1	配置内容	判定为健康的描述
2	Content-Type = text/html	返回header中存在"Content-Type" header，且值为text/html
3	Content-Type != text/html	返回header中存在"Content-Type" header，且值不为text/html
4	Connection ~ close	返回header中存在 Connection，且值为close
5	Connection !~ close	返回 header中存在 Connection，且值不为close
6	Host	返回 header中存在 Host
7	!X-Accel-Redirect	返回 header中不存在 X-Accel-Redirect

健康检查 http > body配置方式

	A	B
1	配置内容	判定为健康的描
2	~ Welcome	返回body内容匹配表达式”
3	!~ Welcome	返回body内容不匹配表达式”

3.6.3 请求示例

! 以下请求请注意替换ip、端口、uri

增加新的健康检查

请求

```
1 POST http://127.0.0.1:8443/hc/1/hc/http/demo
2 Content-Type: application/json
3
4 {
5   "interval": "3s",
6   "jitter": "1s",
7   "timeout": "10s",
8   "passes": 2,
9   "fails": 1,
10  "http": {
11    "header": [
12      "Host"
13    ],
14    "uri": "/test",
15    "body": "~ ok",
16    "status": "200 204"
17  },
18  "ssl": {
19    "enable": false
20  }
21 }
```

返回值

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Fri, 10 Feb 2023 13:06:24 GMT
```

```
4 Content-Type: application/json
5 Content-Length: 37
6 Connection: keep-alive
7
8 {
9   "code": 0,
10  "msg": "success"
11 }
```

删除健康检查

请求

```
1 DELETE http://127.0.0.1:8443/hc/1/hc/http/demo
```

返回值

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Fri, 10 Feb 2023 13:06:21 GMT
4 Content-Type: application/json
5 Content-Length: 37
6 Connection: keep-alive
7
8 {
9   "code": 0,
10  "msg": "success"
11 }
```

查看健康检查列表

请求

```
1 GET http://127.0.0.1:8443/hc/1/hc
```

返回值

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
```



```
3 Date: Fri, 10 Feb 2023 13:07:12 GMT
4 Content-Type: application/json
5 Content-Length: 44
6 Connection: keep-alive
7
8 [
9   {
10     "upstream": "demo",
11     "type": "http"
12   }
13 ]
```

查看健康检查配置详情

请求

```
1 GET http://127.0.0.1:8443/hc/1/hc/http/demo
```

返回值

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Fri, 10 Feb 2023 13:07:14 GMT
4 Content-Type: application/json
5 Content-Length: 157
6 Connection: keep-alive
7
8 {
9   "interval": "3s",
10  "jitter": "1s",
11  "timeout": "10s",
12  "passes": 2,
13  "fails": 1,
14  "http": {
15    "uri": "/test",
16    "status": "200",
17    "body": "~ ok",
18    "header": [
19      "Host"
20    ]
21  }
22 }
```

健康检查证书配置

请求

```
1 POST http://127.0.0.1:8443/hc/1/hc/http/demos
2 Content-Type: application/json
3
4 {
5   "interval": "3s",
6   "jitter": "1s",
7   "timeout": "10s",
8   "passes": 2,
9   "fails": 1,
10  "http": {
11    "uri": "/test",
12    "status": "200"
13  },
14  "ssl": {
15    "enable": true,
16    "certificate": "data:-----BEGIN CERTIFICATE-----\nMIIEITCCAwmgAwIBAgIUk6l
17    "certificateKey": "data:-----BEGIN RSA PRIVATE KEY-----\nMIIEogIBAAKCAQEA
18  }
19 }
```

返回

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Mon, 13 Feb 2023 07:35:27 GMT
4 Content-Type: application/json
5 Content-Length: 37
6 Connection: keep-alive
7
8 {
9   "code": 0,
10  "msg": "success"
11 }
```

3.7 metrics (zxx)

3.7.1 功能介绍

提供对虚拟主机状态信息的访问。包含当前状态、服务器、上游、缓存等信息。

首先，指令 `vhost_traffic_status_zone` 是必需的，然后如果指令 `vhost_traffic_status_display` 设置后，可按如下方式访问：

`/status/format/json`

- 如果您要求 `/status/format/json`，将使用包含当前活动数据的 JSON 文档进行响应，以便在实时仪表板和第三方监视工具中使用。
- `/status/format/html`
 - 如果您要求 `/status/format/html`，将使用 HTML 中的内置实时仪表板进行响应，该仪表板在内部请求 `/status/format/json`。
- `/status/format/jsonp`
 - 如果您要求 `/status/format/jsonp`，将使用包含当前活动数据的 JSONP 回调函数进行响应，以便在实时仪表板和第三方监视工具中使用。
- `/status/format/prometheus`
 - 如果您要求 `/status/format/prometheus`，将响应 prometheus 格式，包含当前活动数据的文档。

3.7.1.1 JSON 示例

```
{
  "hostName": ...,
  "moduleVersion": ...,
  "nginxVersion": ...,
  "loadMsec": ...,
  "nowMsec": ...,
  "connections": {
    "active":...,
    "reading":...,
    "writing":...,
    "waiting":...,
    "accepted":...,
    "handled":...,
    "requests":...
  },
  "sharedZones": {
    "name":...,
```

```
"maxSize":...,
"usedSize":...,
"usedNode":...
},
"serverZones":{
  "...":{
    "requestCounter":...,
    "inBytes":...,
    "outBytes":...,
    "responses":{
      "1xx":...,
      "2xx":...,
      "3xx":...,
      "4xx":...,
      "5xx":...,
      "miss":...,
      "bypass":...,
      "expired":...,
      "stale":...,
      "updating":...,
      "revalidated":...,
      "hit":...,
      "scarce":...
    },
    "requestMsecCounter":...,
    "requestMsec":...,
    "requestMsecs":{
      "times":[...],
      "msecs":[...]
    },
    "requestBuckets":{
      "msecs":[...],
      "counters":[...]
```

```
    },
  }
  ...
},
"filterZones": {
  "...": {
    "...": {
      "requestCounter": ...,
      "inBytes": ...,
      "outBytes": ...,
      "responses": {
        "1xx": ...,
        "2xx": ...,
        "3xx": ...,
        "4xx": ...,
        "5xx": ...,
        "miss": ...,
        "bypass": ...,
        "expired": ...,
        "stale": ...,
        "updating": ...,
        "revalidated": ...,
        "hit": ...,
        "scarce": ...
      },
      "requestMsecCounter": ...,
      "requestMsec": ...,
      "requestMsecs": {
        "times": [...],
        "msecs": [...]
      },
      "requestBuckets": {
        "msecs": [...],
```

```
    "counters":[...]
  },
},
...
},
...
},
"upstreamZones": {
  "...": [
    {
      "server":...,
      "requestCounter":...,
      "inBytes":...,
      "outBytes":...,
      "responses":{
        "1xx":...,
        "2xx":...,
        "3xx":...,
        "4xx":...,
        "5xx":...
      },
      "requestMsecCounter":...,
      "requestMsec":...,
      "requestMsecs":{
        "times":[...],
        "msecs":[...]
      },
      "requestBuckets":{
        "msecs":[...],
        "counters":[...]
      },
      "responseMsecCounter":...,
      "responseMsec":...,
```

```
"responseMsecs":{
  "times":[...],
  "msecs":[...]
},
"responseBuckets":{
  "msecs":[...],
  "counters":[...]
},
"weight":...,
"maxFails":...,
"failTimeout":...,
"backup":...,
"down":...
}
...
],
...
}
"cacheZones": {
  "...":{
    "maxSize":...,
    "usedSize":...,
    "inBytes":...,
    "outBytes":...,
    "responses":{
      "miss":...,
      "bypass":...,
      "expired":...,
      "stale":...,
      "updating":...,
      "revalidated":...,
      "hit":...,
      "scarce":...
```

```
    }  
  },  
  ...  
}  
}
```

3.7.1.2 JSON 字段含义

以下含义以 JSON 示例顺序提供：

hostName

主机名

moduleVersion

{Version}{|.dev.{commit}}格式的模块版本。

njetVersion

njet版本。

loadMsec

加载过程时间（以毫秒为单位）。

nowMsec

当前时间（毫秒）

connections

active

当前活动客户端连接数。

reading

读取客户端连接的总数。

writing

写入客户端连接的总数。

waiting

等待客户端连接的总数。

accepted

接受的客户端连接总数。

handled

已处理的客户端连接总数。

requests

请求的客户端连接总数。

sharedZones

name

在配置中指定的共享内存的名称。（默认值： vhost_traffic_status）

maxSize

在配置中指定的共享内存的最大大小的限制。

usedSize

共享内存的当前大小。

usedNode

共享内存中使用的当前节点数。它可以通过以下公式获得一个节点的大致大小: $(usedSize / usedNode)$

serverZones

requestCounter

从客户端收到的客户端请求总数。

inBytes

从客户端接收的总字节数。

outBytes

发送到客户端的总字节数。

responses

1xx, 2xx, 3xx, 4xx, 5xx

状态代码为 1xx、2xx、3xx、4xx 和 5xx 的响应数。

miss

cache miss的个数。

bypass

缓存绕过的数量。

expired

缓存过期的数量。

stale

缓存过时的数量。

updating

缓存更新的次数。

revalidated

重新验证的缓存数。

hit

缓存命中数。

scarce

缓存不足的数量。

requestMsecCounter

累积请求处理时间的次数（以毫秒为单位）。

requestMsec

请求处理时间的平均时间（以毫秒为单位）。

requestMsecs

times

请求处理时间以毫秒为单位。

msecs

请求处理时间（包括上游），以毫秒为单位。

requestBuckets

msecs

由vhost_traffic_status_histogram_buckets指令设置的柱状图的桶值。

counters

每个存储桶值大于或等于请求处理时间的累积值。

filterZones

它提供了与serverzone相同的字段，只是包含了组名。

upstreamZones

server

服务器的地址。

requestCounter

转发到此服务器的客户端连接总数。

inBytes

从此服务器接收的总字节数。

outBytes

发送到此服务器的总字节数。

responses

1xx, 2xx, 3xx, 4xx, 5xx

状态代码为 1xx、2xx、3xx、4xx 和 5xx 的响应数。

requestMsecCounter

包括上游在内的累计请求处理时间数（以毫秒为单位）。

requestMsec

请求处理时间（包括上游）的平均时间（以毫秒为单位）。

requestMsecs

times

请求处理时间以毫秒为单位。

msecs

请求处理时间（包括上游），以毫秒为单位。

requestBuckets

msecs

由vhost_traffic_status_histogram_buckets指令设置的柱状图的桶值。

counters

每个存储桶值大于或等于请求处理时间（包括上游）的累积值。

responseMsecCounter

累积的仅上游响应处理时间的数量，以毫秒为单位。

responseMsec

仅上游响应处理时间的平均值（以毫秒为单位）。

responseMsecs

times

请求处理时间以毫秒为单位。

msecs

唯一的上游响应处理时间（以毫秒为单位）。

responseBuckets

msecs

由vhost_traffic_status_histogram_buckets指令设置的柱状图的桶值。

counters

每个存储桶值大于或等于唯一上游响应处理时间的累积值。

weight

当前服务器设置的权重

maxFails

服务器当前max_fails设置。

failTimeout

服务器当前fail_timeout设置。

backup

服务器当前备份设置。

down

服务器当前down设置。基本上，这只是一个标记ngx_http_upstream_module的服务器关闭(例如。服务器 [backend3.example.com](#) 关闭)，而不是实际的上游服务器状态。如果你启用了upstream zone指令，它将改变为

cacheZones

maxSize

在配置中指定的缓存的最大大小的限制。如果proxy_cache_path指令中的max_size未指定，则默认分配系统依NGX_MAX_OFF_T_VALUE

usedSize

缓存的当前大小。这个值取自njet，就像上面的maxSize值一样。

inBytes

从缓存接收的总字节数。

outBytes

从缓存发送的总字节数。

responses

miss

缓存未命中数。

bypass

缓存绕过的数量。

expired

缓存过期的数量。

stale

缓存过时的数量。

updating

缓存更新的次数。

revalidated

重新验证的缓存数。

hit

缓存命中数。

scarce

缓存不足的数量。

3.7.2 功能展示

3.7.3 配置说明

3.7.3.1 vhost_traffic_status

	A	B
1	-	-
2	语法	vhost_traffic_status <on off>
3	默认	关闭
4	上下文	http、server、location
5	配置	数据面

描述: 启用或禁用数据面统计指定上下文内性能指标数据。如果设置 了vhost_traffic_status_zone 指令，将自动启用。

3.7.3.2 vhost_traffic_status_zone

	A	B
1	-	-
2	语法	vhost_traffic_status_zone [shared:name:size]
3	默认	shared:vhost_traffic_status:1m
4	上下文	http
5	配置	数据面

描述: 为数据面共享内存区域设置参数，该区域将保留各种键的状态。缓存在所有工作进程之间共享。

在大多数情况下，njet-module-vts使用的共享内存大小不会增加太多。当使用vhost_traffic_status_filter_by_set_key指令时，共享内存大小会增加很多，但如果过滤器的键是固定的(例如。国家代码的总数约为240)，则不会持续增加。

如果使用vhost_traffic_status_filter_by_set_key指令，设置如下：

- 默认设置为上限32M的共享内存大小。(vhost_traffic_status_zone shared:vhost_traffic_status:32 m)
- 如果在error_log中打印出：(“ngx_slab_alloc() failed: no memory in vhost_traffic_status_zone”)，则增加到大于(usedSize * 2)。

3.7.3.3 vhost_traffic_status_filter_by_set_key

	A	B
1	-	-
2	语法	vhost_traffic_status_filter_by_set_key key [name]
3	默认	-
4	上下文	http
5	配置	数据面

描述: 通过用户定义的变量作为展示的维度。key和name可以使用变量，如\$host、\$server_name等

示例如下：

```
http{  
    vhost_traffic_status_filter_by_set_key $request_uri "$realip_remote_addr to  
$server_name";  
  
    ...  
}
```

3.7.3.4 vhost_traffic_status_display

-	-
语法	vhost_traffic_status_display
默认	-
上下文	location
配置	控制面

描述: 启用或禁用控制面显示处理程序。

3.7.3.5 vhost_traffic_status_display_format

-	-
语法	vhost_traffic_status_display_format <json html jsonp prometheus>
默认	json
上下文	location
配置	控制面

描述: 设置控制面数据输出的格式。

- 如果设置 json，将使用 JSON 格式进行响应；
- 如果设置 html，将使用 HTML 中的内置实时仪表板进行响应；

- 如果设置 jsonp，将使用 JSONP 回调函数（默认：`ngx_http_vhost_traffic_status_jsonp_callback`）；
- 如果设置 prometheus，将响应 prometheus 格式的数据。

3.7.3.6 配置示例

3.7.3.6.1 控制面

```
load_module objs/njt_http_vtsd_module.so;
error_log logs/error-ctrl.log info;
events {
....
}
http {
    access_log logs/access-ctrl.log combined;
    server {
        listen    8089;
        location / {
            return 200 "welcome to sub http server\n";
        }
        location /status {
            vhost_traffic_status_display;
            vhost_traffic_status_display_format html;
        }
    }
}
```

3.7.3.6.2 数据面

```
load_module objs/njt_http_vtsc_module.so;
helper ctrl  objs/njt_helper_ctrl_module.so conf/njet_ctrl.conf;
error_log logs/error.log info;
events {
```



```
....  
}  
http{  
    vhost_traffic_status_zone;  
    vhost_traffic_status_filter_by_set_key $request_uri "$realip_remote_addr to $server_name";  
....  
....  
}
```

3.7.4 动态开关

vts模块可以通过动态配置接口实现对是否统计及统计指标的动态配置，详见：[动态开关功能](#)

3.8 telemetry功能

3.8.1 介绍

3.8.1.1 什么是OpenTelemetry?

OpenTelemetry合并了OpenTracing和OpenCensus项目，提供了一组API和库来标准化遥测数据的采集和传输。OpenTelemetry提供了一个安全，厂商中立的工具，这样就可以按照需要将数据发往不同的后端。

OpenTelemetry项目由如下组件构成：

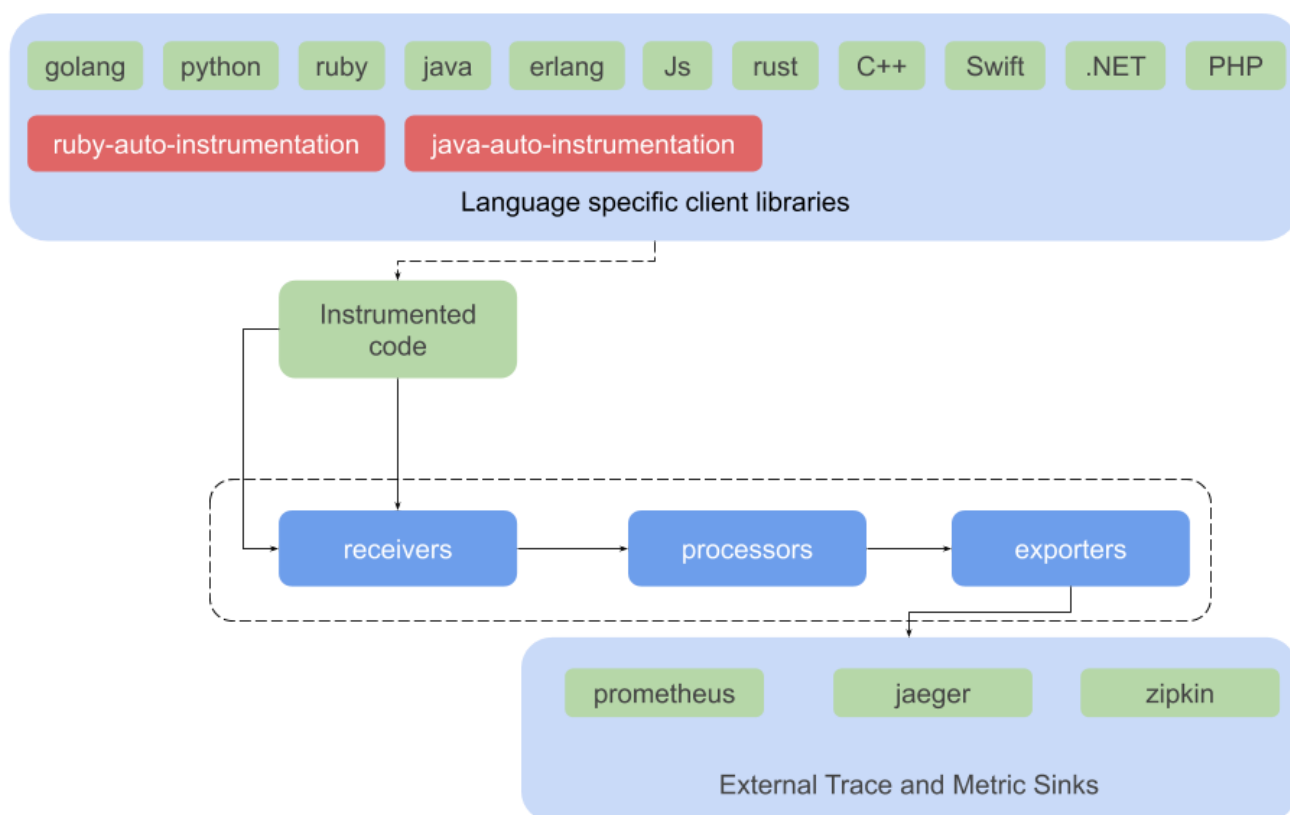
- 推动在所有项目中使用一致的规范
- 基于规范的，包含接口和实现的APIs
- 不同语言的SDK(APIs的实现)，如 Java, Python, Go, Erlang等
- Exporters：可以将数据发往一个选择的后端
- Collectors：厂商中立的实现，用于处理和导出遥测数据

3.8.1.2 术语

- Traces：记录经过分布式系统的请求活动，一个trace是spans的有向无环图
- Spans：一个trace中表示一个命名的，基于时间的操作。Spans嵌套形成trace树。每个trace包含一个根span，描述了端到端的延迟，其子操作也可能拥有一个或多个子spans。

- Metrics：在运行时捕获的关于服务的原始度量数据。OpenTelemetry定义的metric instruments(指标工具)如下。Observer支持通过异步API来采集数据，每个采集间隔采集一个数据。
- Context：一个span包含一个span context，它是一个全局唯一的标识，表示每个span所属的唯一的请求，以及跨服务边界转移trace信息所需的数据。OpenTelemetry 也支持correlation context，它可以包含用户定义的属性。correlation context不是必要的，组件可以选择不携带和存储该信息。
- Context propagation：表示在不同的服务之间传递上下文信息，通常通过HTTP首部。Context propagation 是 OpenTelemetry 系统的关键功能之一。除了tracing之外，还有一些有趣的用法，如，执行A/B测试。OpenTelemetry支持通过多个协议的Context propagation来避免可能发生的问题，但需要注意的是，在自己的应用中最好使用单一的方法。

3.8.1.3 OpenTelemetry架构



opentelemetry也是个插件式的架构，针对不同的开发语言会有相应的Client组件，叫 **Instrumentation**，也就是在代码中埋点调用的api/sdk采集telemetry数据

NJet主要集成了分布式服务 telemetry 模块：

分布式服务追踪模块: njt_otel_module.so

3.8.2 分布式服务追踪模块: njt_otel_module.so

3.8.2.1 指令介绍

opentelemetry

Enable or disable OpenTelemetry (default: enabled).

- **required:** false
- **syntax:** opentelemetry on|off
- **block:** http, server, location

opentelemetry_trust_incoming_spans

Enables or disables using spans from incoming requests as parent for created ones. (default: enabled).

- **required:** false
- **syntax:** opentelemetry_trust_incoming_spans on|off
- **block:** http, server, location

opentelemetry_attribute

Adds a custom attribute to the span. It is possible to access nginx variables, e.g.

```
opentelemetry_attribute "my.user.agent" "$http_user_agent".
```

- **required:** false
- **syntax:** opentelemetry_attribute <key> <value>
- **block:** http, server, location

opentelemetry_config

Exporters, processors

- **required:** true
- **syntax:** opentelemetry_config /path/to/config.toml
- **block:** http

opentelemetry_operation_name

Set the operation name when starting a new span.

- **required:** `false`
- **syntax:** `opentelemetry_operation_name <name>`
- **block:** `http`, `server`, `location`

`opentelemetry_propagate`

Enable propagation of distributed tracing headers, e.g. `traceparent`. When no parent trace is given, a new trace will be started. The default propagator is W3C.

The same inheritance rules as `proxy_set_header` apply, which means this directive is applied at the current configuration level if and only if there are no `proxy_set_header` directives defined on a lower level.

- **required:** `false`
- **syntax:** `opentelemetry_propagate` or `opentelemetry_propagate b3`
- **block:** `http`, `server`, `location`

`opentelemetry_capture_headers`

Enables the capturing of request and response headers. (default: disabled).

- **required:** `false`
- **syntax:** `opentelemetry_capture_headers on|off`
- **block:** `http`, `server`, `location`

`opentelemetry_sensitive_header_names`

Sets the captured header value to `[REDACTED]` for all headers where the name matches the given regex (case insensitive).

- **required:** `false`
- **syntax:** `opentelemetry_sensitive_header_names <regex>`
- **block:** `http`, `server`, `location`

`opentelemetry_sensitive_header_values`

Sets the captured header value to [REDACTED] for all headers where the value matches the given regex (case insensitive).

- **required:** false
- **syntax:** opentelemetry_sensitive_header_values <regex>
- **block:** http, server, location

opentelemetry_ignore_paths

No span will be created for URIs matching the given regex (case insensitive).

- **required:** false
- **syntax:** opentelemetry_ignore_paths <regex>
- **block:** http, server, location

3.8.2.2 Njet.conf 配置

```
1 #user root;
2 worker_processes 1;
3 daemon off;
4 #master_process on;
5 error_log /home/njet/clbtest/logs/error.log debug;
6 pid /home/njet/clbtest/njet.pid;
7
8 #load_module /root/project/telemetry/opentelemetry-cpp-contrib-main/instrumer
9 load_module /home/njet/modules/njt_otel_module.so;
10
11 events {
12     worker_connections 1024;
13 }
14
15
16 http {
17     opentelemetry_config /home/njet/clbtest/conf/otel-njet.toml;
18     opentelemetry off;
19
20     upstream http1{
21         server 192.168.40.139:9002;
22         #server 192.168.40.136:8082;
23     }
24
25     server {
```

```

26     listen 8081;
27     server_name otel_example;
28
29     location = / {
30         opentelemetry on;
31         opentelemetry_operation_name my_example_backend;
32         opentelemetry_propagate;
33         proxy_pass http://http1;
34     }
35
36
37     location = /b3 {
38         opentelemetry on;
39         opentelemetry_operation_name my_other_backend;
40         opentelemetry_propagate b3;
41         # Adds a custom attribute to the span
42         opentelemetry_attribute "req.time" "$msec";
43         proxy_pass http://http1;
44     }
45 }
46 }

```

3.8.2.3 Otel module 配置

```

1 exporter = "otlp"
2 processor = "batch"
3
4 [exporters.otlp]
5 #collector server address
6 # Alternatively the OTEL_EXPORTER_OTLP_ENDPOINT environment variable can also
7 host = "192.168.40.136"
8 port = 4317
9 # Optional: enable SSL, for endpoints that support it
10 # use_ssl = true
11 # Optional: set a filesystem path to a pem file to be used for SSL encryption
12 # (when use_ssl = true)
13 # ssl_cert_path = "/path/to/cert.pem"
14
15 [processors.batch]
16 max_queue_size = 2048
17 schedule_delay_millis = 5000
18 max_export_batch_size = 512
19

```

```
20 [service]
21 # Can also be set by the OTEL_SERVICE_NAME environment variable.
22 name = "njet-proxy" # Opentelemetry resource name
23
24 [sampler]
25 name = "AlwaysOn" # Also: AlwaysOff, TraceIdRatioBased
26 ratio = 0.1
27 parent_based = false
28
```

3.8.3 启动collector以及jaeger服务

通过docker-compose 启动collector 以及jaeger服务

docker-compose up -f docker-compose.yaml -d

docker-compose.yaml

```
1 version: "2"
2 services:
3
4   # Jaeger
5   jaeger-all-in-one:
6     image: jaegertracing/all-in-one:latest
7     restart: always
8     ports:
9       - "16686:16686"
10      - "14268"
11      - "14250"
12
13   # Zipkin
14   zipkin-all-in-one:
15     image: openzipkin/zipkin:latest
16     restart: always
17     ports:
18       - "9411:9411"
19
20   # Collector
21   otel-collector:
22     image: ${OTELCOL_IMG}
23     restart: always
24     command: ["--config=/etc/otel-collector-config.yaml", "${OTELCOL_ARGS}"]
25     volumes:
```

```
26     - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
27   ports:
28     - "1888:1888"    # pprof extension
29     - "8888:8888"    # Prometheus metrics exposed by the collector
30     - "8889:8889"    # Prometheus exporter metrics
31     - "13133:13133"  # health_check extension
32     - "4317:4317"    # OTLP gRPC receiver
33     - "4318:4318"    # OTLP http receiver
34     - "55679:55679"  # zpages extension
35   depends_on:
36     - jaeger-all-in-one
37     - zipkin-all-in-one
38
39   #demo-client:
40   #   build:
41   #     dockerfile: Dockerfile
42   #     context: ./client
43   #   restart: always
44   #   environment:
45   #     - OTEL_EXPORTER_OTLP_ENDPOINT=otel-collector:4317
46   #     - DEMO_SERVER_ENDPOINT=http://demo-server:7080/hello
47   #   depends_on:
48   #     - demo-server
49
50   #demo-server:
51   #   build:
52   #     dockerfile: Dockerfile
53   #     context: ./server
54   #   restart: always
55   #   environment:
56   #     - OTEL_EXPORTER_OTLP_ENDPOINT=otel-collector:4317
57   #   ports:
58   #     - "7080"
59   #   depends_on:
60   #     - otel-collector
61
62   prometheus:
63     container_name: prometheus
64     image: prom/prometheus:latest
65     restart: always
66     volumes:
67       - ./prometheus.yaml:/etc/prometheus/prometheus.yml
68     ports:
69       - "9090:9090"
```

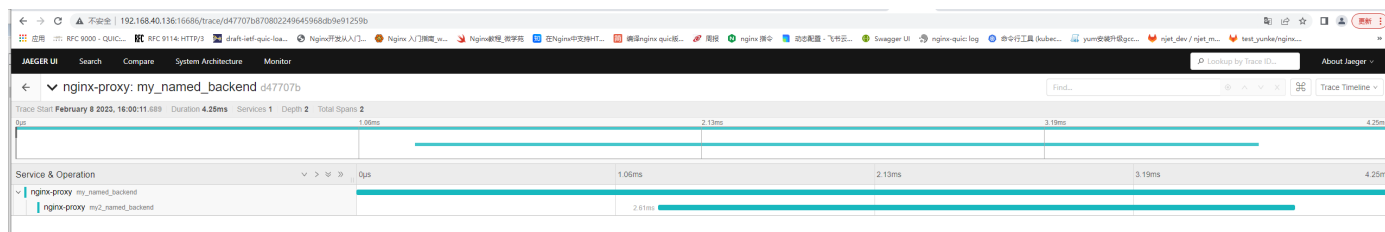

3.8.4 效果

3.8.4.1 分布式服务追踪模块: njet_otel_module.so

Jaeger ui: <http://192.168.40.136:16686>

通过jaeger ui 查看trace

<http://192.168.40.136:16686>



3.9 Upstream member 维护

3.9.1 简介

该模块使用restful API, 实现动态添加, 删除, 修改http,stream 的 upstream server 的功能。

3.9.2 配置说明

要开启该功能, 需在njet_ctrl.conf 的 main block 中加载该模块:

```
load_module /home/njet/modules/njet_http_upstream_api_module.so;
```

并在 http block 下, 指定该模块的配置文件:

```
http {  
    access_log /home/njet/logs/access_ctrl.log combined;  
    server {  
        listen 8443;  
        location /api {  
            api write=on;  
        }  
    }  
}
```

配置项	必须修改	配置说明
api	是	在该location 下，接收upstream api 请求
write	否	off 只支持查询get 方法（只能查询）。 on 支持 g 修改）。

HTTP Upstreams			^
GET	/http/upstreams/	Return status of all HTTP upstream server groups	▼
GET	/http/upstreams/{httpUpstreamName}/	Return status of an HTTP upstream server group	▼
DELETE	/http/upstreams/{httpUpstreamName}/	Reset statistics of an HTTP upstream server group	▼
GET	/http/upstreams/{httpUpstreamName}/servers/	Return configuration of all servers in an HTTP upstream server group	▼
POST	/http/upstreams/{httpUpstreamName}/servers/	Add a server to an HTTP upstream server group	▼
GET	/http/upstreams/{httpUpstreamName}/servers/{httpUpstreamServerId}	Return configuration of a server in an HTTP upstream server group	▼
PATCH	/http/upstreams/{httpUpstreamName}/servers/{httpUpstreamServerId}	Modify a server in an HTTP upstream server group	▼
DELETE	/http/upstreams/{httpUpstreamName}/servers/{httpUpstreamServerId}	Remove a server from an HTTP upstream server group	▼

Stream Upstreams			^
GET	/stream/upstreams/	Return status of all stream upstream server groups	▼
GET	/stream/upstreams/{streamUpstreamName}/	Return status of a stream upstream server group	▼
DELETE	/stream/upstreams/{streamUpstreamName}/	Reset statistics of a stream upstream server group	▼
GET	/stream/upstreams/{streamUpstreamName}/servers/	Return configuration of all servers in a stream upstream server group	▼
POST	/stream/upstreams/{streamUpstreamName}/servers/	Add a server to a stream upstream server group	▼
GET	/stream/upstreams/{streamUpstreamName}/servers/{streamUpstreamServerId}	Return configuration of a server in a stream upstream server group	▼
PATCH	/stream/upstreams/{streamUpstreamName}/servers/{streamUpstreamServerId}	Modify a server in a stream upstream server group	▼
DELETE	/stream/upstreams/{streamUpstreamName}/servers/{streamUpstreamServerId}	Remove a server from a stream upstream server group	▼

3.9.3 参考说明：

参考swagger-ui：

server:8081/doc/swagger/

3.9.4 Api 实例：

3.9.4.1 说明：

查询http 的，和stream 的格式类似。指定http 或 stream。upstream 名为http 或stream 的upstream 根据实际名字查询。

<http://127.0.0.1:8443/api/7/> {http 或 stream} /upstreams/ {http 或stream 的upstream 名称} /servers

3.9.4.2 添加:

```
curl -v -X POST -s http://127.0.0.1:8443/api/7/http/upstreams/http1/servers -d '{"server":"192.168.40.1:9003"}'|jq .
```

3.9.4.3 查询:

```
curl -v -s http://127.0.0.1:8443/api/7/http/upstreams/ |jq .
```

3.9.4.4 修改:

```
curl -v -X PATCH -s http://127.0.0.1:8443/api/7/http/upstreams/http1/servers/0 -d '{"server":"192.168.40.2:9003"}'|jq .
```

3.9.4.5 删除:

```
url -X DELETE http://127.0.0.1:8443/api/7/http/upstreams/http1/servers/0 |jq .
```

3.9.4.6 重置:

针对upstream，清空健康检查等数据:

```
curl -v -X DELETE http://127.0.0.1:8443/api/7/http/upstreams/http1/ |jq .
```

3.10 Split Clients_2

3.10.1 功能说明

开源nginx提供了一个流量配比的模块[split_clients](#)，但是该模块是根据指定的一个变量值（如客户端ip, url 中的参数等），然后计算Hash 值, 然后根据计算的值在 $0 - 2^{32}-1$ 的位置来进行流量配比，无法做到根据请求的数量做概率分布。

Split Clients 2 模块提供流量配比，是根据概率进行分布，当访问样本足够大的情况下，与设置的比例基本达到一致, 通过该模块的动态配置，可以实现蓝绿发布（配置比例为 0%， 及 100%）

3.10.2 流量配置实现方式

Split Clients 2 模块提供了两种方式进行流量配比：

- 静态配置方式：通过静态配置文件实现不同流量配比，分配的百分比，已经配置好，可以在配置文件中进行修改，但需要重新reload才能生效
- 动态配置方式：通过声明式API实现动态修改流量配比

静态配置

配置指令：

 语法： **split_clients_2** `$variable` { ... };

默认值：

允许配置位置： http

配置样例：

```
1 split_clients_2 $backend {
2     10%          backend1;
3     *           backend2;
4 }
```

配置补充说明：

- 静态配置百分比修改后，需要reload才能生效。
- 百分比一行结束后，如果缺少分号，会导致下一行配比不生效。注意不能缺少分号。
- 配比分流upstream的数量，限制只能配置两个。

动态配置

参考声明式api配置splitclient章节

验证分流是否生效

可以通过如下脚本实现模拟大样本的流量配比验证。如配比为50%， 请求100次，会在40多到50多区间进行配比，如下。

```
[root@master110 conf]# for in in `seq 1 100`; do curl localhost:80; sleep 1s; done
welcome to 8081 http server
welcome to 8080 http server
welcome to 8080 http server
welcome to 8080 http server
welcome to 8080 http server
welcome to 8080 http server
^C
[root@master110 conf]# rm -rf static.txt;for in in `seq 1 100`; do curl localhost:80 >> static.txt 2> /dev/null; done;grep -o "8081" static.txt|wc -l
57
[root@master110 conf]# ^C
[root@master110 conf]#
```

3.11 动态Access Log

3.11.1 功能说明

通过动态access_log，无需对配置文件进行编辑及重载，就可以实现对access log开关状态的动态控制功能，现提供了动态查询接口和动态修改接口。

动态access_log开关accessLogOn缺省值为true。该动态开关配置在http，server，location的上下文配置中修改没有继承关系。

静态access log开关指令为access_log off，再把动态access log开关同时配置情况下，以动态access log开关配置为优先。

3.11.2 动态查询接口

可以查看HTTP下的所有server及location，以及各个location块内access log当前的开关状态；发起请求：

```
1 GET http://127.0.0.1:8443/api/1/config/http_log
```

结果：

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Fri, 24 Feb 2023 06:52:19 GMT
4 Content-Type: application/json
5 Content-Length: 480
6 Connection: keep-alive
7
8 {
9   "servers": [
10     {
11       "listens": [
12         "0.0.0.0:1081"
13       ],
14       "serverNames": [
15         ""
16       ],
17       "locations": [
18         {
19           "location": "/",
20           "accessLogOn": true,
21           "locations": [
22             {
23               "location": "/demo",
24               "accessLogOn": false
```

```
25         }
26     ]
27 },
28 {
29     "location": "= /test/demo",
30     "accessLogOn": true
31 }
32 ]
33 },
34 {
35     "listens": [
36         "0.0.0.0:1080"
37     ],
38     "serverNames": [
39         ""
40     ],
41     "locations": [
42         {
43             "location": "/",
44             "accessLogOn": true
45         },
46         {
47             "location": "= /test/demo",
48             "accessLogOn": true
49         },
50         {
51             "location": "/demo",
52             "accessLogOn": true
53         },
54         {
55             "location": "/demos",
56             "accessLogOn": true
57         },
58         {
59             "location": "/named",
60             "accessLogOn": true
61         }
62     ]
63 }
64 ]
65 }
```

3.11.2.1 验证方式

3.11.2.1.1 配置

```

load_module /home/dyn_accesslog/modules/nginx_http_kv_module.so;
load_module /home/dyn_accesslog/modules/nginx_agent_dynlog_module.so;
helper broker /home/dyn_accesslog/modules/nginx_helper_broker_module.so conf/eqtt.conf;
helper ctrl /home/dyn_accesslog/modules/nginx_helper_ctrl_module.so conf/ctrl.conf;

cluster_name helper;
node_name node1;

worker_processes 1;
user root;
pid run/nginx.pid;
error_log logs/error.log info;
events {
    worker_connections 1024;
}

http {
    dyn_kv conf conf/tot-work.conf;
    log_format json_combined escape=json '{"@timestamp":"$time_iso8601",'
        "host":"$http_host",'
        "uri":"$uri",'
        "status":"$status",'
        "server":"$server_name",'
        "port":"$server_port",'
        "protocol":"$server_protocol",'
        "request_uri":"$request_uri",'
        "@source":"$server_addr",'
        "engine_fields":{"
        "remote_addr":"$remote_addr",'
        "remote_user":"$remote_user",'
        "body_bytes_sent":"$body_bytes_sent",'
        "request_time":"$request_time",'
        "request_body":"$request_body",'
        "request_method":"$request_method",'
        "http_referer":"$http_referer",'
        "http_x_forwarded_for":"$http_x_forwarded_for",'
        "http_user_agent":"$http_user_agent",'
        "upstream_response_time":"$upstream_response_time",'
        "upstream_addr":"$upstream_addr"}}';
    access_log logs/access_http.log json_combined;
    server {
        listen unix:/var/run/nginx.sock;
        listen 2011;
        listen 193.108.40.110:2017;
        listen 2014;
        listen 2016;
        # listen [::]:2015 ipv6only=on;
        server_name serv110.com;
        access_log logs/access_server1.log json_combined;
        location /l1 {
            return 200 "l1的返回内容\n";
            location /l1/l11 {
                access_log off;
                return 200 "l11的返回内容\n";
            }
        }
        location /l2 {
            access_log logs/access_l2.log json_combined;
            return 200 "l2的返回内容\n";
        }
    }
    server {
        listen 2013;
        server_name serv110;
        access_log logs/access_server2.log json_combined;
        location /l3 {
            return 200 "l3的返回内容\n";
            location /l3/l33 {
                return 200 "l33的返回内容\n";
            }
        }
        location /l4 {
            return 200 "l4的返回内容\n";
        }
    }
}

```


如以上截图，配置2个server，每个server都是3个location。第一个server的子location日志单独设置关闭。

3.11.2.1.2 查询接口调用

```

root@buster110 conf]# curl 192.168.40.110:2012/rxiconfapi/1/config/http_log -v|jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0      0  0 0 0 0
* Connected to 192.168.40.110 (192.168.40.110) port 2012 (#0)
* GET /rxiconfapi/1/config/http_log HTTP/1.1
> Host: 192.168.40.110:2012
> User-Agent: curl/7.83.0
> Accept: */*
*
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Mon, 06 Mar 2023 02:15:32 GMT
< Content-Type: application/json
< Content-Length: 481
< Connection: keep-alive
*
[ 481 bytes data]
100 481 100 481    0     0 1845    0 --:--:-- --:--:-- --:--:-- 1858
* Connection #0 to host 192.168.40.110 left intact
{
  "servers": [
    {
      "listeners": [
        {
          "addr": "/var/run/nginx.sock",
          "0.0.0.0:2012",
          "192.168.40.110:2012",
          "0.0.0.0:2014",
          "0.0.0.0:2016"
        }
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/11",
          "accessLogOn": true,
          "locations": [
            {
              "location": "/11/111",
              "accessLogOn": false
            }
          ]
        },
        {
          "location": "/11",
          "accessLogOn": true
        }
      ]
    },
    {
      "listeners": [
        {
          "0.0.0.0:2012"
        }
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/13",
          "accessLogOn": true,
          "locations": [
            {
              "location": "/13/133",
              "accessLogOn": true
            }
          ]
        },
        {
          "location": "/14",
          "accessLogOn": true
        }
      ]
    }
  ]
}

```

调用查询接口，可见所有监听的server，及各个server下的location，并能看到上方server子location 访问日志是关闭状态。

3.11.3 动态修改接口

可以对查询结果中的access log的开关状态进行动态修改。

3.11.3.1 请求

```
1  PUT http://127.0.0.1:8443/api/1/config/http_log
2  Content-Type: application/json
3
4  {
5    "servers": [
6      {
7        "listens": [
8          "0.0.0.0:1081"
9        ],
10       "serverNames": [
11         ""
12       ],
13       "locations": [
14         {
15           "location": "/",
16           "accessLogOn": true,
17           "locations": [
18             {
19               "location": "/demo",
20               "accessLogOn": false
21             }
22           ]
23         }
24       ]
25     }
26   ]
27 }
```

3.11.3.2 结果

```
1  HTTP/1.1 204 No Content
2  Server: njet/1.23.1
3  Date: Thu, 23 Feb 2023 11:33:08 GMT
4  Connection: keep-alive
```

3.11.3.3 验证方式

3.11.3.3.1 修改前查询

```
[root@master110 conf]# curl 192.168.40.110:2012/rxlconfapi/1/config/http_log -vjq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %    0     0              0             0             0     0
0    0    0    0    0    0    0    0 --:--:-- --:--:-- --:--:--    0*   Trying 192.168.40.110:2012...
* Connected to 192.168.40.110 (192.168.40.110) port 2012 (#0)
> GET /rxlconfapi/1/config/http_log HTTP/1.1
> Host: 192.168.40.110:2012
> User-Agent: curl/7.87.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Mon, 30 Mar 2023 02:15:32 GMT
< Content-Type: application/json
< Content-Length: 481
< Connection: keep-alive
<
{ [481 bytes data]
100 481 100 481 0 0 1845 0 --:--:-- --:--:-- --:--:-- 1850
* Connection #0 to host 192.168.40.110 left intact
{
  "servers": [
    {
      "listeners": [
        {
          "unix": "/var/run/nginx.sock",
          "listen": "0.0.0.0:2011",
          "listen": "192.168.40.110:2012",
          "listen": "0.0.0.0:2014",
          "listen": "0.0.0.0:2010"
        }
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/",
          "accessLogOn": true,
          "locations": [
            {
              "location": "/1/11",
              "accessLogOn": false
            }
          ]
        },
        {
          "location": "/12",
          "accessLogOn": true
        }
      ]
    },
    {
      "listeners": [
        {
          "listen": "0.0.0.0:2013"
        }
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/",
          "accessLogOn": true,
          "locations": [
            {
              "location": "/13/133",
              "accessLogOn": true
            }
          ]
        }
      ]
    }
  ]
}
```

```

    "location": "/l4",
    "accessLogOn": true
  }
}
}
}

```

3.11.3.3.2 修改配置

对查询结果进行修改，作为要修改的内容，提供给修改接口

vi 1.json

```

{
  "servers": [
    {
      "listens": [
        "unix:/var/run/nginx.sock",
        "0.0.0.0:2011",
        "192.168.40.110:2017",
        "0.0.0.0:2014",
        "0.0.0.0:2016"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/l1",
          "accessLogOn": false,
          "locations": [
            {
              "location": "/l1/l11",
              "accessLogOn": false
            }
          ]
        },
        {
          "location": "/l2",
          "accessLogOn": false
        }
      ]
    }
  ]
}

```

```

{
  "listens": [
    "0.0.0.0:2013"
  ],
  "serverNames": [
    ""
  ],
  "locations": [
    {
      "location": "/l3",
      "accessLogOn": false,
      "locations": [
        {
          "location": "/l3/l33",
          "accessLogOn": false
        }
      ]
    },
    {
      "location": "/l4",
      "accessLogOn": false
    }
  ]
}

```

3.11.3.3.3 修改接口调用

```

[root@master110 conf]# curl 192.168.40.110:2012/zxlconfapi/1/config/http_log -v -X PUT -d @1.json
* Trying 192.168.40.110:2012...
* Connected to 192.168.40.110 (192.168.40.110) port 2012 (#0)
> PUT /zxlconfapi/1/config/http_log HTTP/1.1
> Host: 192.168.40.110:2012
> User-Agent: curl/7.87.0
> Accept: */*
> Content-Length: 951
> Content-Type: application/x-www-form-urlencoded
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 204 No Content
< Server: njet/1.23.1
< Date: Mon, 06 Mar 2023 02:28:52 GMT
< Connection: keep-alive
<
* Connection #0 to host 192.168.40.110 left intact
[root@master110 conf]#

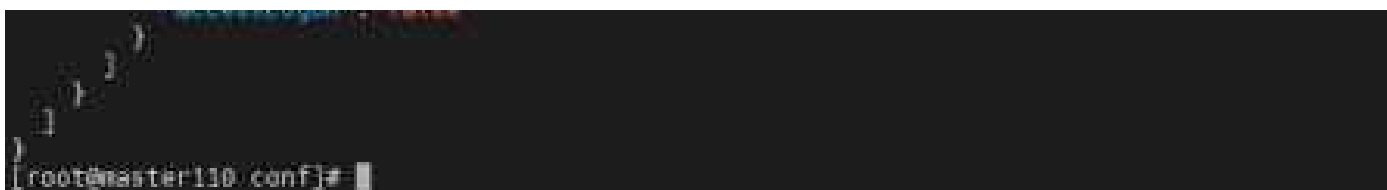
```

3.11.3.3.4 修改后再查询验证修改是否成功

```

[root@master110 conf]# curl 192.168.40.110:2012/zxlconfapi/1/config/http_log -v |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left   Speed
  0    0    0     0    0    0  --:--:-- --:--:-- --:--:--    0*   Trying 192
* Connected to 192.168.40.110 (192.168.40.110) port 2012 (#0)
> GET /zxlconfapi/1/config/http_log HTTP/1.1
> Host: 192.168.40.110:2012
> User-Agent: curl/7.87.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Mon, 06 Mar 2023 02:29:24 GMT
< Content-Type: application/json
< Content-Length: 486
< Connection: keep-alive
<
{ [486 bytes data]
100  486 100  486    0    0  558    0 --:--:-- --:--:-- --:--:--  557
* Connection #0 to host 192.168.40.110 left intact
{
  "servers": [
    {
      "listens": [
        "unix:/var/run/nginx.sock",
        "0.0.0.0:2011",
        "192.168.40.110:2017",
        "0.0.0.0:2014",
        "0.0.0.0:2016"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/",
          "accessLogOn": false,
          "locations": [
            {
              "location": "/list",
              "accessLogOn": false
            }
          ]
        },
        {
          "location": "/l2",
          "accessLogOn": false
        }
      ]
    },
    {
      "listens": [
        "0.0.0.0:2013"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/l3",
          "accessLogOn": false,
          "locations": [
            {
              "location": "/l3/list",
              "accessLogOn": false
            }
          ]
        },
        {
          "location": "/l4",
          "accessLogOn": false
        }
      ]
    }
  ]
}

```



3.12 缓存功能

3.12.1 简介

Njet会缓存来自代理 Web 或应用程序服务器的静态和动态内容，以加快向客户端的响应速度并减少上游服务器上的负载。启用缓存后，Njet将响应保存在Njet磁盘缓存中，并使用它们来响应客户端，而不必每次都对相同内容向上游服务器发起请求。

3.12.2 启用缓存

若要启用缓存，请在Http上下文中包含 `proxy_cache_path`指令。第一个参数是缓存内容的本地文件系统路径，第二个参数定义用于存储有关缓存项的元数据的共享内存区域的名称和大小。

```
1 http {  
2     # ...  
3     proxy_cache_path /data/nginx/cache keys_zone=mycache:10m;  
4 }
```

然后在要缓存server响应的上下文（http、server或location）中包含`proxy_cache`指令，并指定由上述指令参数定义的共享内存区域名称。

```
1 http {  
2     # ...  
3     proxy_cache_path /data/nginx/cache keys_zone=mycache:10m;  
4     server {  
5         proxy_cache mycache;  
6         location / {  
7             proxy_pass http://localhost:8000;  
8         }  
9     }  
10 }
```

请注意，参数定义的大小不会限制缓存响应数据的总量。缓存的响应本身与元数据的副本一起存储在文件系统上的特定文件中。要限制缓存的响应数据量，请将参数`max_size`配置在`proxy_cache_path`指令中。（但请注意，缓存的数据量可能会暂时超过此限制）。

3.12.3 指定要缓存的请求

默认情况下，NJet会在第一次从代理服务器收到此类响应时缓存对使用HTTP GET和HEAD方法发出的请求的所有响应。NJet使用请求字符串作为请求的标识符。如果请求与缓存的响应具有相同的键，NJet会将缓存的响应发送到客户端。您可以在 `http{}`、`server{}` 或 `location{}` 中包含各种指令，以控制缓存哪些响应。

要更改用于计算缓存文件标识，请使用 `proxy_cache_key` 指令。

```
1 proxy_cache_key "$host$request_uri$cookie_user";
```

要定义在缓存响应之前必须发出具有相同键的请求的最小次数，请使用 `proxy_cache_min_uses` 指令：

```
1 proxy_cache_min_uses 5;
```

要使用 GET 和 HEAD 以外的方法缓存对请求的响应，请使用 `proxy_cache_methods` 指令并配置要缓存的方法。

```
1 proxy_cache_methods GET HEAD POST;
```

3.12.4 限制或禁用缓存

默认情况下，响应无限期地保留在缓存中。仅当缓存超过最大配置大小时，才会删除它们，然后按自上次请求它们以来的时间排序最长最少使用的缓存进行删除。您可以通过在 `http{}`、`server{}` 或 `location{}` 上下文中包含指令来设置缓存响应被视为有效的时间和是否使用缓存：

要限制具有特定状态代码的缓存响应被视为有效的时间，请使用 `proxy_cache_valid` 指令：

```
1 proxy_cache_valid 200 302 10m;  
2 proxy_cache_valid 404 1m;
```

在此示例中，包含代码200或302的响应被视为有效 10 分钟，包含代码的响应的有效期为 1 分钟。要定义具有所有状态代码的响应的有效期，请指定为第一个参数为 `any`：

```
1 proxy_cache_valid any 5m;
```


要定义NJet不向客户端发送缓存响应的条件，请使用proxy_cache_bypass指令。每个参数定义一个条件，并由许多变量组成。如果至少有一个参数不为空且不等于"0"，NJet不会在缓存中查找响应，而是立即将请求转发到后端服务器。

```
1 proxy_cache_bypass $cookie_nocache $arg_nocache$arg_comment;
```

要定义NJet根本不缓存响应的条件，请使用proxy_no_cache指令，并以与proxy_cache_bypass指令相同的方式定义参数。

```
1 proxy_no_cache $http_pragma $http_authorization;
```

3.12.5 从缓存中清除内容

NJet可以从缓存中删除过时的缓存文件。这对于删除过时的缓存内容以防止同时提供旧版和新版本的网页是必需的。缓存在收到包含自定义 HTTP 标头或 HTTP 方法的特殊“PURGE”请求时被清除。

3.12.5.1 配置缓存清除功能

让我们设置一个配置来标识使用 HTTP PURGE 方法的请求并删除匹配的 URL。

a. 在上下文中，使用map创建一个新变量，例如：

```
1 http {# ...
2     map $request_method $purge_method {
3         PURGE 1;
4         default 0;
5     }
6 }
```

b. 在配置缓存的location{}块中，配置proxy_cache_purge 指令以指定缓存清除请求的条件。在我们的示例中，它是在上一步中配置的：

```
1 server {
2     listen      80;
3     server_name www.example.com;
4     location / {
5         proxy_pass https://localhost:8002;
6         proxy_cache mycache;
7         proxy_cache_purge $purge_method;
```

```
8     }  
9 }
```

3.12.5.2 发送PURGE请求

配置proxy_cache_purge指令后，需要发送特殊的缓存清除请求来清除缓存。您可以使用一系列工具发出清除请求，包括以下示例中的curl命令：

```
1 $ curl -X PURGE -D - "https://www.example.com/*"  
2 HTTP/1.1 204 No Content  
3 Server: njet/1.23.2  
4 Date: Sat, 19 May 2018 16:33:04 GMT  
5 Connection: keep-alive
```

在此示例中，将清除具有公共 URL 部分（由星号通配符指定）的资源。但是，此类缓存条目不会立即从缓存中完全删除：它们将保留在磁盘上，直到它们因不活动（由 proxy_cache_path 指令的参数确定）或缓存清除器（使用 purge 参数启用）而被删除，或者客户端尝试访问它们。

3.12.5.3 限制对清除命令的访问

我们建议您限制允许发送缓存清除请求的 IP 地址：

```
1 geo $purge_allowed {  
2     default          0; # deny from other  
3     10.0.0.1          1; # allow from 10.0.0.1 address  
4     192.168.0.0/24    1; # allow from 192.168.0.0/24  
5 }  
6 map $request_method $purge_method {  
7     PURGE             $purge_allowed;  
8     default           0;  
9 }
```

在此示例中，Njet 检查请求中是否使用了PURGE方法，如果是则分析客户端 IP 地址。如果 IP 地址已列入白名单，则 设置为： 允许清除，否则拒绝清除。

3.12.5.4 从缓存中完全删除文件

要完全删除与星号匹配的缓存文件，请激活purge功能，该功能将循环访问所有缓存条目并删除与通配符键匹配的条目。在配置中包含 purge 参数到proxy_cache_path指令：

```
1 proxy_cache_path /data/nginx/cache levels=1:2 keys_zone=mycache:10m purger=or
```

3.12.5.5 缓存清除配置示例

```
1 http {# ...
2     proxy_cache_path /data/nginx/cache levels=1:2 keys_zone=mycache:10m
   purger=on;
3     map $request_method $purge_method {
4         PURGE 1;
5         default 0;
6     }
7     server {
8         listen      80;
9         server_name www.example.com;
10        location / {
11            proxy_pass      https://localhost:8002;
12            proxy_cache      mycache;
13            proxy_cache_purge $purge_method;
14        }
15    }
16    geo $purge_allowed {
17        default          0;
18        10.0.0.1          1;
19        192.168.0.0/24    1;
20    }
21    map $request_method $purge_method {
22        PURGE      $purge_allowed;
23        default 0;
24    }
25 }
```

3.12.6 文件分片缓存

初始缓存填充操作有时需要相当长的时间，尤其是对于大文件。例如，当视频文件开始下载以满足对文件一部分的初始请求时，后续请求必须等待整个文件下载并放入缓存中。

NJet可以缓存此类分片请求，并使用缓存切片模块逐渐填充缓存，该模块将文件划分为较小的“分片”。每个Range请求都会选择覆盖所请求范围的特定切片，如果此Range仍未缓存，则将其放入缓存中。对这些切片的所有其他请求都会从缓存中获取数据。

3.12.6.1 开启分片缓存

a. 使用slice指令指定切片的大小：

```
1 location / {
2     slice 1m;
```

```
3 }
```

选择可加快切片下载速度的切片大小。如果大小太小，则内存使用量可能过多，并且在处理请求时打开了大量文件描述符，而过大可能会导致下载生成缓存缓慢。

b. 将 `$slice_range` 变量包含在缓存键中：

```
1 proxy_cache_key $uri$is_args$args$slice_range;
```

c. 状态代码206启用响应缓存：

```
1 proxy_cache_valid 200 206 1h;
```

d. 通过在标头字段中设置 `$slice_range` 变量，允许将range请求传递到代理服务器

```
1 proxy_set_header Range $slice_range;
```

全部配置如下：

```
1 location / {
2     slice                1m;
3     proxy_cache           cache;
4     proxy_cache_key       $uri$is_args$args$slice_range;
5     proxy_set_header      Range $slice_range;
6     proxy_cache_valid     200 206 1h;
7     proxy_pass             http://localhost:8000;
8 }
```

3.12.7 组合配置示例

以下示例配置结合了上述一些缓存选项。

```
1 http {
2     # ...
3     proxy_cache_path /data/nginx/cache keys_zone=mycache:10m loader_threshold=64;
4     server {
5         listen 8080;
```

```

6         proxy_cache mycache;
7     location / {
8         proxy_pass http://backend1;
9     }
10    location /some/path {
11        proxy_pass http://backend2;
12        proxy_cache_valid any 1m;
13        proxy_cache_min_uses 3;
14        proxy_cache_bypass $cookie_nocache $arg_nocache$arg_comment;
15    }
16 }
17 }
```

在此示例中，两个location使用相同的缓存，但方式不同。

由于backend1响应很少更改，因此不包括缓存控制指令。响应在首次发出请求时缓存，并无限期有效。

相比之下，对backend2请求的响应经常更改，因此它们被认为仅在 1 分钟内有效，并且在发出 3 次相同的请求之前不会缓存。此外，如果请求符合proxy_cache_bypass指令定义的条件，Njet会立即将请求传递给backend2，而无需在缓存中查找相应的响应。

3.13 key value功能

3.13.1 简介

控制面的Sendmsg模块提供了可选的配置指令dyn_sendmsg_kv，这个指令将对外提供KV 值设置及查询的 HTTP API 接口。

3.13.2 启用KV API

通过在控制面的配置文件中，添加一个location, 并在location 中使用dyn_sendmsg_kv指令开启KV HTTP API。以下的配置示例中的端口及URL 可以根据需要进行更改。

```

1     server {
2         listen 8081;
3         location /kv {
4             dyn_sendmsg_kv;
5         }
6     }
```

3.13.1 API 说明

通过配置的location 的url，使用POST方法，设置key value 值，提交的报文为JSON格式。

！ 以下请求请注意替换IP、端口、URL，与实际的配置需要一致

设置KV值

请求：

```
1 POST http://127.0.0.1:8081/kv
2 Content-Type: application/json
3
4 {
5     "key":"sc_test_key",
6     "value":"here is the test msg"
7 }
```

返回：

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Tue, 07 Mar 2023 08:10:20 GMT
4 Content-Length: 0
5 Connection: keep-alive
```

查询key value 值

请求：

```
1 GET http://127.0.0.1:8081/kv?key=sc_test_key
```

返回：

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Tue, 07 Mar 2023 08:14:38 GMT
4 Content-Type: text/plain
5 Content-Length: 20
6 Connection: keep-alive
7
```

3.14 fastcgi模块

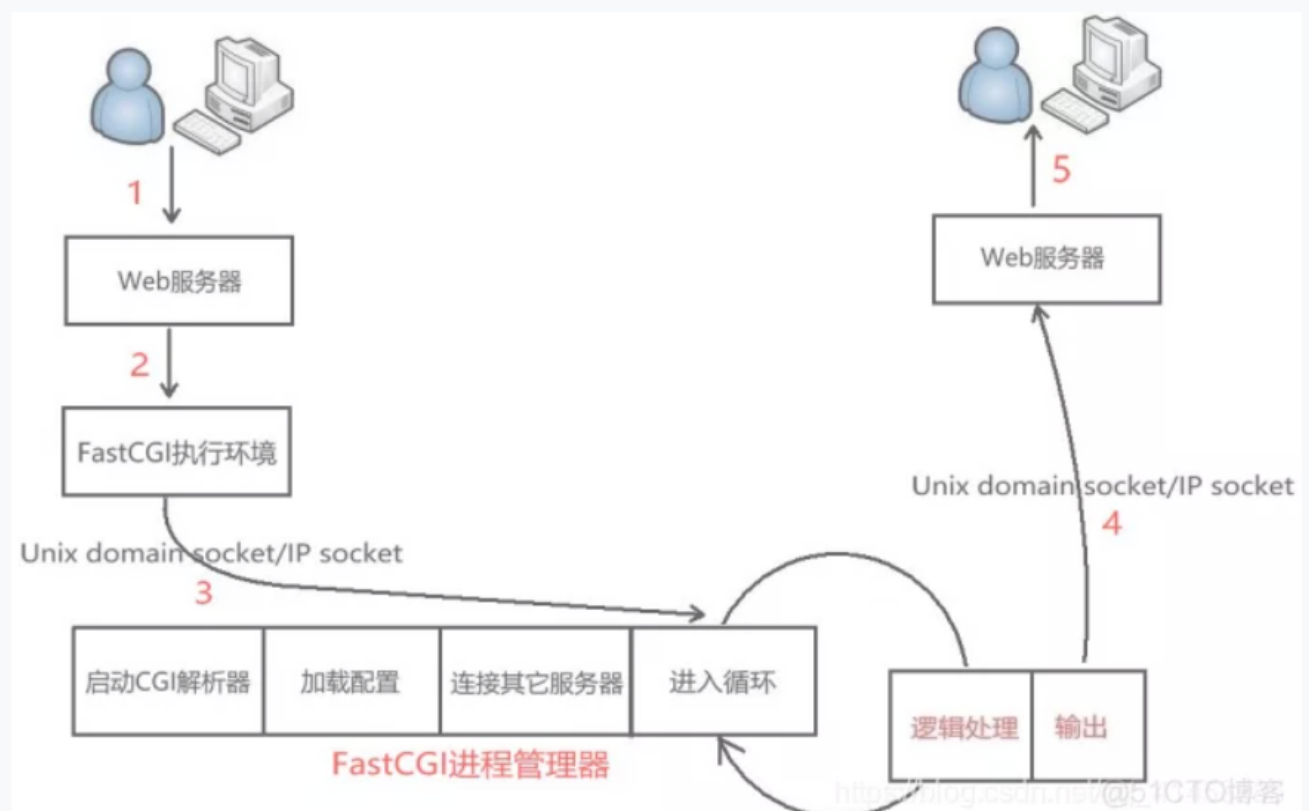
3.14.1 模块介绍以及原理

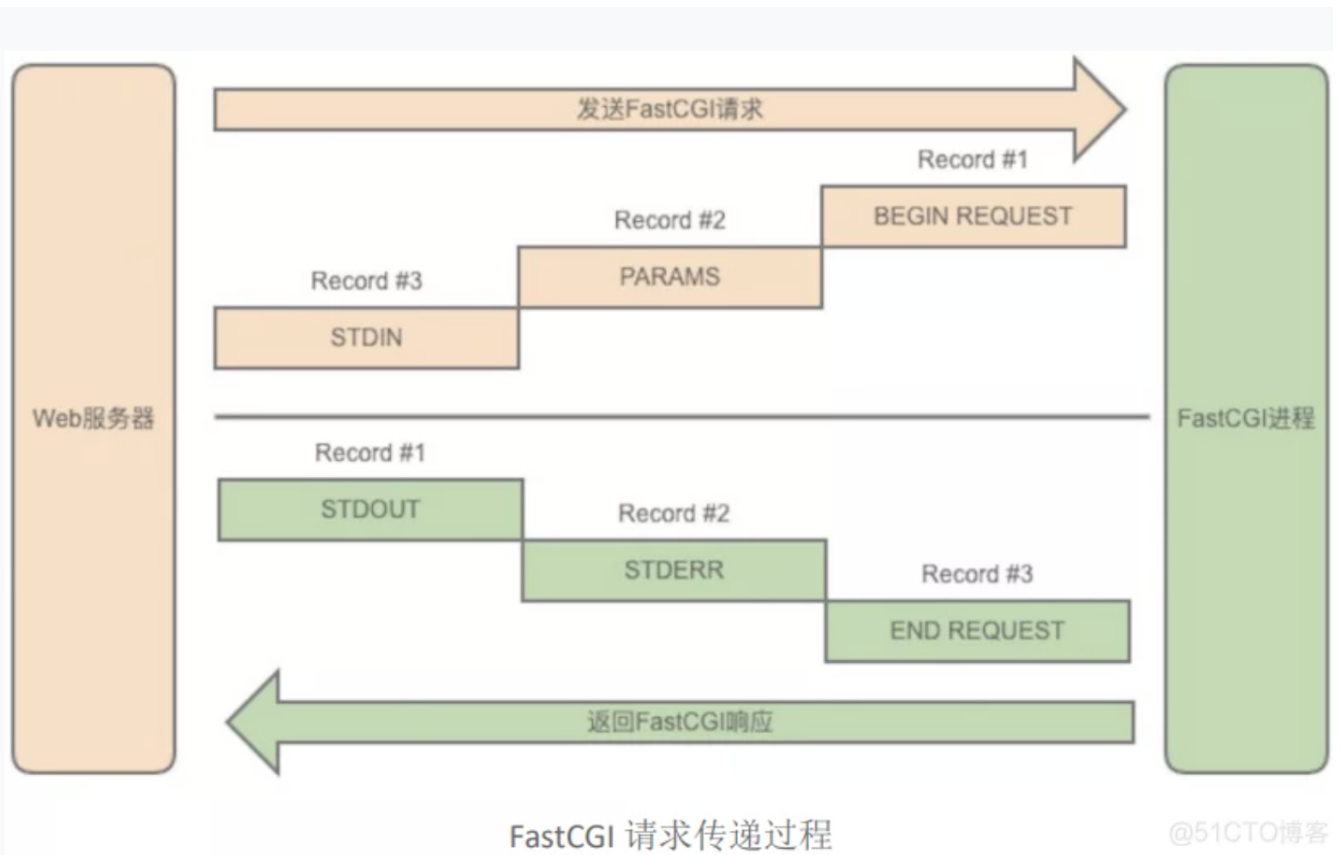
通用网关接口（Common Gateway Interface、CGI）描述了客户端和服务端程序之间传输数据的一种标准，可以让一个客户端，从网页浏览器向执行在网络服务器上的程序请求数据。CGI 独立于任何语言的，CGI 程序可以用任何脚本语言或者是完全独立编程语言实现，只要这个语言可以在这个系统上运行。Unix shell script、Python、Ruby、PHP、perl、Tcl、C/C++ 和 Visual Basic 都可以用来编写 CGI 程序

快速通用网关接口（Fast Common Gateway Interface/FastCGI）是通用网关接口（CGI）的改进，描述了客户端和服务端程序之间传输数据的一种标准。FastCGI 致力于减少 Web 服务器与 CGI 程式之间互动的开销，从而使服务器可以同时处理更多的 Web 请求。与为每个请求创建一个新的进程不同，FastCGI 使用持续的进程来处理一连串的请求。这些进程由 FastCGI 进程管理器管理（例如下面我们要介绍的spawn-fcgi），而不是 web 服务器

3.14.1.1 fastcgi处理流程

FastCGI处理流程





3.14.1.2 常用环境变量

1. SCRIPT_FILENAME \$document_root\$fastcgi_script_name; #脚本文件请求的路径
2. QUERY_STRING \$query_string; #请求的参数;如?app=123
3. REQUEST_METHOD \$request_method; #请求的动作(GET,POST)
4. CONTENT_TYPE \$content_type; #请求头中的 Content-Type 字段
5. CONTENT_LENGTH \$content_length; #请求头中的 Content-length 字段。
6. SCRIPT_NAME \$fastcgi_script_name; #脚本名称
7. REQUEST_URI \$request_uri; #请求的地址不带参数
8. DOCUMENT_URI \$document_uri; #与\$uri 相同。
9. DOCUMENT_ROOT \$document_root; #网站的根目录。在 server 配置中 root 指令中指定的值
10. SERVER_PROTOCOL \$server_protocol; #请求使用的协议，通常是 HTTP/1.0 或 HTTP/1.1。
11. GATEWAY_INTERFACE CGI/1.1; #cgi 版本
12. SERVER_SOFTWARE nginx/\$nginx_version; #nginx 版本号，可修改、隐藏
13. REMOTE_ADDR \$remote_addr; #客户端 IP
14. REMOTE_PORT \$remote_port; #客户端端口
15. SERVER_ADDR \$server_addr; #服务器 IP 地址

- 16. SERVER_PORT \$server_port; #服务器端口
- 17. SERVER_NAME \$server_name; #服务器名，域名在 server 配置中指定的
server_name
- 18. PATH_INFO \$path_info; #可自定义变量

3.14.2 功能列表

官方详细指令介绍请查看 https://nginx.org/en/docs/http/nginx_http_fastcgi_module.html

指令列表(50个，其中15个指令是关于缓存cache，其他是一些buffer设置、超时设置、header设置等基础功能)

- fastcgi_bind
- fastcgi_buffer_size
- fastcgi_buffering
- fastcgi_buffers
- fastcgi_busy_buffers_size

#cache 功能

- fastcgi_cache
- fastcgi_cache_background_update
- fastcgi_cache_bypass
- fastcgi_cache_key
- fastcgi_cache_lock
- fastcgi_cache_lock_age
- fastcgi_cache_lock_timeout
- fastcgi_cache_max_range_offset
- fastcgi_cache_methods
- fastcgi_cache_min_uses
- fastcgi_cache_path
- fastcgi_cache_revalidate
- fastcgi_cache_use_stale
- fastcgi_cache_valid
- fastcgi_catch_stderr
- fastcgi_connect_timeout
- fastcgi_force_ranges
- fastcgi_hide_header
- fastcgi_ignore_client_abort
- fastcgi_ignore_headers
- fastcgi_index
- fastcgi_intercept_errors
- fastcgi_keep_conn

fastcgi_limit_rate
fastcgi_max_temp_file_size
fastcgi_next_upstream
fastcgi_next_upstream_timeout
fastcgi_next_upstream_tries
fastcgi_no_cache
fastcgi_param

#pass功能， 注意pass 地址支持两种， unix socket以及ip端口形式

fastcgi_pass
fastcgi_pass_header
fastcgi_pass_request_body
fastcgi_pass_request_headers
fastcgi_read_timeout
fastcgi_request_buffering
fastcgi_send_lowat
fastcgi_send_timeout
fastcgi_socket_keepalive
fastcgi_split_path_info
fastcgi_store
fastcgi_store_access
fastcgi_temp_file_write_size
fastcgi_temp_path

3.14.3 测试样例

3.14.3.13 环境搭建

Fcgi进程管理器（有多种，此处使用Php-fpm测试）

spawn-fcgi是一个通用的 FastCGI 进程管理器，简单小巧，原先是属于 lighttpd 的一部分， 管理C语言编写的FastCGI程序

php-fpm 即 PHP FastCGI 进程管理器(管理PHP的)，并且提供进程 管理的功能。进程包括 master进程和worker进程。master进程只有一个，负责监听端口，接受来自 web server的请求。worker进程一般会有多个，每个进程中会嵌入一个PHP解析器，进行PHP代码的处理
进程管理器，只是创建一些进程池进行处理，真正的具体业务需要由对应的业务处理程序处理

```
[root@localhost ~]# ps -elf|grep php
\4 s root      8193      1 0 80 0 - 58155 ep_pol 15:45 ?        00:00:00 php-fpm: master process (/etc/php-fpm.conf)
1 s root      8194      8193 0 80 0 - 58155 inet_c 15:45 ?        00:00:00 php-fpm: pool www
1 s root      8195      8193 0 80 0 - 58155 inet_c 15:45 ?        00:00:00 php-fpm: pool www
1 s root      8196      8193 0 80 0 - 58155 inet_c 15:45 ?        00:00:00 php-fpm: pool www
1 s root      8197      8193 0 80 0 - 58155 inet_c 15:45 ?        00:00:00 php-fpm: pool www
1 s root      8198      8193 0 80 0 - 58155 inet_c 15:45 ?        00:00:00 php-fpm: pool www
1 s root      8417      8193 0 80 0 - 58155 inet_c 15:45 ?        00:00:00 php-fpm: pool www
0 s root      32543     1819 0 80 0 - 28208 pipe_w 16:54 pts/1    00:00:00 grep --color=auto php
[root@localhost ~]#
```

本次测试环境（可根据测试需要自行安装, 推荐njet代理与php-fpm部署在不同机器测试， unix地址测试可采用本机测试）：

安装： centos下可通过命令安装 php-fpm： `yum install php-fpm.x86_64`

配置： 配置文件位于 `/etc/php-fpm.conf` 以及 `/etc/php-fpm.d/www.conf`

`vim /etc/php-fpm.d/www.conf` 编辑该文件可设置监听ip以及端口或者unix地址，此处设置为ip: `127.0.0.1:9000`

```
;  
;       specific port;  
;       '/path/to/████/socket' - to listen on a █████ socket.  
;       Note: This value is mandatory.  
listen = 127.0.0.1:9000  
  
; Set listen(2) backlog. A value of '-1' means unlimited.  
; Default value: -1
```

启动: `service php-fpm restart` 也支持 (stop/restart)

具体可参考 <https://zhuanlan.zhihu.com/p/134029988>

3.14.4 njet fastcgi配置示例

njet.conf

```
1 http {  
2     include      mime.types;  
3     default_type  application/octet-stream;  
4  
5     #log_format   main '$remote_addr - $remote_user [$time_local] "$request  
6     #              '$status $body_bytes_sent "$http_referer" '  
7     #              '"$http_user_agent" "$http_x_forwarded_for"';  
8  
9     #access_log   logs/access.log main;  
10  
11     log_format   scripts '$document_root$fastcgi_script_name > $request';  
12     access_log   logs/scripts.log scripts;  
13  
14  
15     sendfile      on;  
16     #tcp_nopush    on;  
17  
18     #keepalive_timeout 0;  
19     keepalive_timeout 65;  
20  
21     #gzip on;  
22  
23     server {  
24         listen      80;  
25         server_name  localhost;
```

```

26
27     # pass the PHP scripts to FastCGI server listening on 127.0.0.1:9000
28     #
29     location ~ /\.php$ {
30         root /usr/share/njet/html;
31         fastcgi_pass    127.0.0.1:9000;
32         fastcgi_index   index.php;
33         include         fastcgi.conf;
34     }
35 }
36

```

fastcgi.conf (官方提供的一个默认的配置文件，可根据需要改动)

```

1 fastcgi_param SCRIPT_FILENAME    $document_root$fastcgi_script_name;
2 fastcgi_param QUERY_STRING       $query_string;
3 fastcgi_param REQUEST_METHOD     $request_method;
4 fastcgi_param CONTENT_TYPE       $content_type;
5 fastcgi_param CONTENT_LENGTH     $content_length;
6
7 fastcgi_param SCRIPT_NAME        $fastcgi_script_name;
8 fastcgi_param REQUEST_URI        $request_uri;
9 fastcgi_param DOCUMENT_URI       $document_uri;
10 fastcgi_param DOCUMENT_ROOT     $document_root;
11 fastcgi_param SERVER_PROTOCOL    $server_protocol;
12 fastcgi_param REQUEST_SCHEME     $scheme;
13 fastcgi_param HTTPS              $https if_not_empty;
14
15 fastcgi_param GATEWAY_INTERFACE  CGI/1.1;
16 fastcgi_param SERVER_SOFTWARE    njet/$njet_version;
17
18 fastcgi_param REMOTE_ADDR        $remote_addr;
19 fastcgi_param REMOTE_PORT        $remote_port;
20 fastcgi_param SERVER_ADDR        $server_addr;
21 fastcgi_param SERVER_PORT        $server_port;
22 fastcgi_param SERVER_NAME        $server_name;
23
24 # PHP only, required if PHP was built with --enable-force-cgi-redirect
25 fastcgi_param REDIRECT_STATUS    200;

```

index.php

```
1 <?php
2 echo "This is my first PHP page!";
3 ?>
```

3.15 正向代理

配置在客户端，客户端所有流量都通过正向代理去访问

3.15.1 指令介绍

3.15.1.1 proxy_connect

Syntax: **proxy_connect**

Default: `none`

Context: `server`

Enable "CONNECT" HTTP method support.

3.15.1.2 proxy_connect_allow

Syntax: **proxy_connect_allow** `all` | `[port ...]` | `[port-range ...]`

Default: `443 563`

Context: `server`

This directive specifies a list of port numbers or ranges to which the proxy CONNECT method may connect.

By default, only the default https port (443) and the default snwps port (563) are enabled.

Using this directive will override this default and allow connections to the listed ports only.

The value `all` will allow all ports to proxy.

The value `port` will allow specified port to proxy.

The value `port-range` will allow specified range of port to proxy, for example:

```
1 proxy_connect_allow 1000-2000 3000-4000; # allow range of port from 1000 to 2
```

3.15.1.3 proxy_connect_connect_timeout

Syntax: **proxy_connect_connect_timeout** `time`

Default: none

Context: server

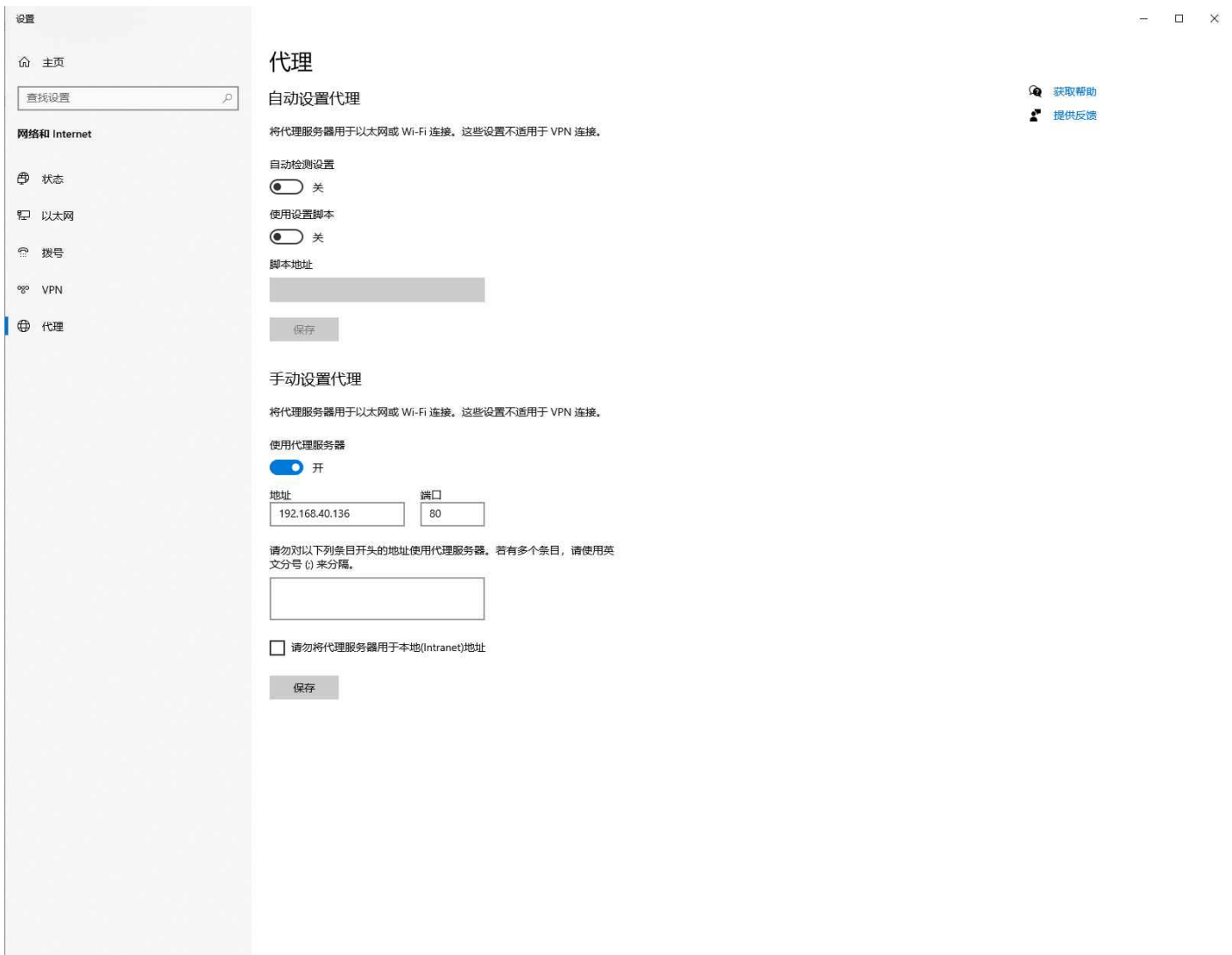
Defines a timeout for establishing a connection with a proxied server.

3.15.2 代理配置示例

Example

```
1 server {
2     listen 80;    //端口设定
3     resolver 114.114.114.114; # dns解析服务器
4
5     proxy_connect;
6     proxy_connect_allow      443, 456;
7     proxy_connect_connect_timeout  10s;
8
9     location /{
10        proxy_pass $scheme://$host$request_uri; #proxy_pass 用来要代理的,
11        # $scheme是客户端请求的协议 (如http, https) ;
12        # $host是客户端请求的域名 (如baidu.com) ;
13        # $request_uri是客户端访问的url地址 (如/baidu?s=12345) 。
14        # 他们拼接成就是http://baidu.com/baidu?s=12345
15    }
16 }
17 }
```

3.15.3 客户端配置 (win10系统为例)



3.16 JWT认证

3.16.1 简介

该模块用于检查JWT-token 是否有效。

3.16.2 指令说明



配置语法: `auth_jwt $variable / on / off;`

默认值: `auth_jwt off;`

允许配置位置: http、http>server、http>location

启用 JWT 的验证，仅当变量值为on时或者配置为on时开启验证。



配置语法: `auth_jwt_key value [encoding];`

默认值: 无

允许配置位置: http、http>server、http>location

指定用于验证 JWT 签名的密钥。

编码可以是 hex | utf8 | base64 | file（默认值为 utf8）。

如果是文件该选项要求该值为有效的文件路径（指向 PEM 编码的公钥）。

 **配置语法:** `auth_jwt_alg any / HS256 / HS384 / HS512 / RS256 / RS384 / RS512 / ES256 / ES384 / ES512;`

默认值: `auth_jwt_alg any;`

允许配置位置: http、http>server、http>location

指定服务器期望在接收的 JWT 的算法。

3.16.3 配置示例

```
1 server {
2     auth_jwt_key "0123456789abcdef" hex; # Your key as hex string
3     auth_jwt off;
4     location /secured-by-cookie/ {
5         auth_jwt $cookie_MyCookieName;
6     }
7     location /secured-by-auth-header/ {
8         auth_jwt on;
9     }
10    location /secured-by-auth-header-too/ {
11        auth_jwt_key "another-secret"; # Your key as utf8 string
12        auth_jwt on;
13    }
14    location /secured-by-rsa-key/ {
15        auth_jwt_key /etc/keys/rsa-public.pem file; # Your key from a PEM
16        auth_jwt on;
17    }
18 }
```

3.17 Mirror

镜像原始请求 通过创建后台镜像子请求。对镜像子请求的响应将被忽略。

3.17.1 指令说明



配置语法: mirror uri | off;

默认值: mirror off;

允许配置位置: http、http>server、http>location

设置原始请求将镜像到的 URI。可以在同一配置级别上指定多个镜像。



配置语法: mirror_request_body on| off;

默认值: mirror_request_body on;

允许配置位置: http、http>server、http>location

指示是否镜像客户端请求正文。启用后，将读取客户端请求正文 在创建镜像子请求之前。在这种情况下，无缓冲的客户端请求正文代理，proxy_request_buffering、fastcgi_request_buffering、scgi_request_buffering、uwsgi_request_buffering指令将被禁用。

3.17.2 配置示例

```
1 location / {  
2     mirror /mirror;  
3     proxy_pass http://backend;  
4 }  
5  
6 location = /mirror {  
7     internal;  
8     proxy_pass http://test_backend$request_uri;  
9 }
```

3.18 服务发现

3.18.1 简介

http, stream upstream 配置域名格式的server，定期查询dns server进行域名解析，实现对upstream server 的添加，删除，修改。

3.18.2 配置说明：

支持http 和 stream 的upstream 配置。

要开启该功能, 需在njet.conf 的 http 或 stream 的upstream block 修改配置:

```
user root root;

worker_processes 1;

error_log /home/njet/logs/error.log debug;

events {
    worker_connections 1024;
}

http {
    dyn_kv_conf conf/iot.conf;
    error_log /home/njet/logs/error.log debug;
    access_log /home/njet/logs/access.log combined;

    upstream http1{
        zone upstream_http 64k;
        resolver 192.168.40.101 valid=10s;
        state zyg11.conf;
    }

    upstream http2{
        zone upstream_http 64k;
        resolver 192.168.40.101 valid=10s;
        server www.dev.test.com resolve;
    }

    server {
        listen 8000;
        server_name cluster1;
        location / {
            proxy_pass http://http1;
        }
    }
}
```

```

}
server {
    listen 8899;
    server_name cluster2;
}
}
stream{

    upstream stream_upstream{
        zone upstream_stream 64k;
        resolver 192.168.40.101 valid=10s;
        state zyg_stream.conf;
    }
    server {
        listen    9000;
        proxy_pass stream_upstream;
    }
}

helper ctrl /home/njet/modules/njt_helper_ctrl_module.so /home/njet/conf/njet_ctrl.conf;
helper broker /home/njet/modules/njt_helper_broker_module.so conf/mqtt.conf;
cluster_name helper;
node_name node1;

```

配置项	必须修改	配置说明
zone	是	upstream 的zone 的名字，及大小。例如：zone upstream_stream 64k;
resolver	是	dns 服务器的地址。resolver 192.168.40.101 vali
valid	否	域名失效的周期，也就是查询dns 的周期。支持标 192.168.40.101 valid=10s;
resolver_timeout	否	查询dns 超时时间。支持标准的NJet时间字段。

state	否	state zyg_stream.conf; 通过upstream api 接口路径。reload 之后不会丢失。 不配置的话，不保存。 reload 之后丢失。
resolve	否	配置静态的动态域名。删除后reload 之后，还会加 server www.dev.test.com resolve; 注意：server 字段和state 字段冲突。 不能静态动

3.18.3 依赖：

- a. Upstream api 接口， 动态解析的ip 可以通过upstream api 查看信息。
- b. 需要搭建dns 服务器，进行域名解析ip， 的添加，删除，修改

3.19 动态开关功能

3.19.1 简介

该模块使用声明式API实现access log 等模块的部分功能动态开关。其通过控制面Copilot:njet_ctrl实现接口，借助Copilot: broker, sendmsg, kv等模块构建的消息总线，传递到worker中，worker即数据面通过实现特定模块的动态配置接口从而实现整体的动态控制。

3.19.2 控制面配置说明

依赖模块



njt_http_sendmsg_module.so
njt_ctrl_config_api_module.so

指令说明

config_req_pool_size



语法: config_req_pool_size number;
默认值: 500
允许配置位置: http->main

配置查询请求（GET）等待池的大小，要求大于查询接口请求最大qps。

config_api



语法: config_api;

默认值: 无

允许配置位置: http->location

配置开启动态config api。

配置示例

控制面配置

```
1 load_module /mnt/f/workspace/c/njet_main/objs/njt_http_sendmsg_module.so;
2 load_module /mnt/f/workspace/c/njet_main/objs/njt_ctrl_config_api_module.s
3 error_log stderr info;# 注意修改为具体的文件路径, 此配置为输出到控制台
4
5 events {
6     worker_connections 1024;
7 }
8 cluster_name helper;
9 node_name node1;
10 http {
11
12     dyn_sendmsg_conf conf/iot_ctrl.conf;
13     access_log logs/access_ctrl.log combined;
14     config_req_pool_size 1000;
15
16     server {
17         listen 8443;
18
19         location /api {
20             config_api;
21         }
22     }
23 }
```

conf/iot_ctrl.conf 文件内容

```
1 log_type error
```

3.19.3 数据面配置说明

3.19.3.1 access log 数据面配置说明

依赖模块

控制面参见上文控制面配置说明

 njt_agent_dynlog_module.so

3.19.3.2 配置指令

无

3.19.3.3 调用样例

3.19.3.3.1 查询access log 当前状态

该接口可以查看所有HTTP location 块内access log 当前的开关状态

请求

```
1 GET http://127.0.0.1:8443/config/1/config/http_log
```

返回值

```
1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Fri, 24 Feb 2023 06:52:19 GMT
4 Content-Type: application/json
5 Content-Length: 480
6 Connection: keep-alive
7
8 {
9   "servers": [
10     {
11       "listens": [
12         "0.0.0.0:1081"
13       ],
14       "serverNames": [
15         ""
16       ],
17       "locations": [
18         {
19           "location": "/",
20           "accessLogOn": true,
21           "locations": [
```

```

22         {
23             "location": "/demo",
24             "accessLogOn": false
25         }
26     ]
27 },
28 {
29     "location": "= /test/demo",
30     "accessLogOn": true
31 }
32 ]
33 },
34 {
35     "listens": [
36         "0.0.0.0:1080"
37     ],
38     "serverNames": [
39         ""
40     ],
41     "locations": [
42         {
43             "location": "/",
44             "accessLogOn": true
45         },
46         {
47             "location": "= /test/demo",
48             "accessLogOn": true
49         },
50         {
51             "location": "/demo",
52             "accessLogOn": true
53         },
54         {
55             "location": "/demos",
56             "accessLogOn": true
57         },
58         {
59             "location": "/named",
60             "accessLogOn": true
61         }
62     ]
63 }
64 ]
65 }

```

3.19.3.3.2 修改access log 开关状态



此处请求body可以支持发送仅局部json

报错提示在error log中，http不做参数校验

请求

```
1 PUT http://127.0.0.1:8443/config/1/config/http_log
2 Content-Type: application/json
3
4 {
5   "servers": [
6     {
7       "listens": [
8         "0.0.0.0:1081"
9       ],
10      "serverNames": [
11        ""
12      ],
13      "locations": [
14        {
15          "location": "/",
16          "accessLogOn": true,
17          "locations": [
18            {
19              "location": "/demo",
20              "accessLogOn": false
21            }
22          ]
23        }
24      ]
25    }
26  ]
27 }
```

返回值

```
1 HTTP/1.1 204 No Content
2 Server: njet/1.23.1
3 Date: Thu, 23 Feb 2023 11:33:08 GMT
4 Connection: keep-alive
```

完成效果

logOff字段为true的location块处理请求时不再打印accesslog。

logOff字段为false的location块处理请求时打印accesslog。

3.19.3.4 动态黑白名单配置

3.17.3.4.1 依赖模块

控制面参见上文控制面配置说明

数据面需要加载如下模块

 njt_helper_broker_module
njt_http_dyn_bwlist_module.so

3.17.3.4.2 配置指令

动态配置无特殊的指令，动态黑白名单的配置指令及格式与静态配置一致，使用 allow, deny 指定允许或拒绝的IP地址。

 语法: allow address | `CIDR` /all;
默认值: --
允许配置位置: http, server, location

 语法: deny address | `CIDR` /all;
默认值: --
允许配置位置: http, server, location

3.17.3.4.3 配置示例

```
1 worker_processes 1;  
2 error_log logs/error.log info;  
3 load_module /home/njet/modules/njt_http_dyn_bwlist_module.so;  
4 helper broker /home/njet/modules/njt_helper_broker_module.so conf/mqtt.conf;  
5 helper broker /home/njet/modules/njt_helper_ctrl_module.so conf/njet_ctrl.conf;  
6  
7 events {  
8     worker_connections 1024;  
9 }  
10 upstream backend {
```

```

11  server 10.10.12.1:8080;
12  }
13
14  http{
15      dyn_kv_conf conf/iot.conf;
16      server {
17          listen      18888;
18          location / {
19              proxy_pass http://${backend};
20          }
21          location /test_bwlist {
22              allow 192.168.1.0/24;
23              deny all;
24              proxy_pass http://${backend};
25          }
26      }
27  }
28  cluster_name helper;
29  node_name node1;

```

3.17.3.4.4 接口示例

使用GET方法获取当前各location的黑白名单配置，使用PUT方法更新黑白名单配置。PUT提交时报文格式与GET获取到的报文格式一致，可以只提交需要改动的location。

```

1  GET http://127.0.0.1:8443/api/1/config/http_dyn_bwlist
2
3  {
4      "servers": [{
5          "listens": ["0.0.0.0:18888"],
6          "serverNames": [""],
7          "locations": [{
8              "location": "/"
9          }, {
10             "location": "/test_bwlist",
11             "accessIpv4": [{
12                 "rule": "allow",
13                 "addr": "192.168.1.0",
14                 "mask": "255.255.255.0"
15             }, {
16                 "rule": "deny",
17                 "addr": "0.0.0.0",
18                 "mask": "0.0.0.0"
19             }]
20         }]
21     }]

```

3.19.3.5 动态分流

3.17.3.5.1 依赖模块

控制面参见上文控制面配置说明

数据面需要加载如下模块

 njt_helper_broker_module
njt_http_split_clients_2_module.so

3.17.3.5.2 配置指令

动态配置无特殊的指令，split_clients_2 的配置指令及格式与静态配置一致。

3.17.3.5.3 配置示例

```

1 worker_processes 1;
2 error_log logs/error.log info;
3 load_module /home/njet/modules/njt_http_split_clients_2_module.so;
4 helper broker /home/njet/modules/njt_helper_broker_module.so conf/mqtt.conf;
5 helper broker /home/njet/modules/njt_helper_ctrl_module.so conf/njet_ctrl.conf;
6
7 events {
8     worker_connections 1024;
9 }
10
11 http{
12     dyn_kv_conf conf/iot.conf;
13     split_clients_2 $backend {
14         10% backend1;
15         * backend2;
16     }
17     upstream backend1 {
18         server 127.0.0.1:18080;
19     }
20     upstream backend2 {
21         server 127.0.0.1:18081;
22     }
23     server {
24         listen 18888;
25         location / {
26             proxy_pass http://${backend};

```

```

27     }
28 }
29 server {
30     listen    18080;
31     location / {
32         return 200 "welcome to 18080 http server\n";
33     }
34 }
35 server {
36     listen    18081;
37     location / {
38         return 200 "welcome to 18081 http server\n";
39     }
40 }
41 }
42 cluster_name helper;
43 node_name node1;

```

3.17.3.5.4 接口示例

使用GET方法获取当前split clients 2 模块的流量配比设置，使用PUT 方法更新流量配比设置。
PUT时使用的报文格式与GET获取到的报文格式一致。

```

1 GET http://127.0.0.1:8443/api/1/config/http_split_clients_2
2
3 {
4     "http": {
5         "split_clients_2": {
6             "backend1": 10,
7             "backend2": 90
8         }
9     }
10 }

```

3.19.3.6 动态metric

3.19.3.6.1 依赖模块

控制面参见上文控制面配置说明。

数据面需要加载如下模块

 njt_helper_broker_module
njt_http_vtsc_module.so

3.19.3.6.2 配置指令

动态配置无特殊的指令。

3.19.3.6.3 配置示例

主要配置示例

```
1 load_module objs/njt_http_vtsc_module.so;
2
3 helper ctrl objs/njt_helper_ctrl_module.so conf/njet_ctrl.conf;
4 helper broker objs/njt_helper_broker_module.so conf/mqtt.conf;
5
6 worker_processes 1;
7 error_log logs/error.log info;
8
9 events {
10     worker_connections 1024;
11 }
12
13 http {
14     dyn_kv_conf conf/iot.conf;
15
16     access_log logs/access.log combined;
17     variables_hash_max_size 2048;
18
19     proxy_cache_path cache levels=1:2 keys_zone=cache:20m max_size=1g inac
20     proxy_cache_valid any 10m ;
21     proxy_cache_revalidate on;
22     proxy_cache_lock on;
23     proxy_cache_key $scheme$host$request_uri$slice_range;
24
25     vhost_traffic_status_zone;
26     vhost_traffic_status_filter_by_set_key "$request_uri" "$server_name";
27
28     upstream backgroup {
29         zone backcenter 128k;
30         server 192.168.30.222:80;
31     }
32
33     upstream errgroup {
34         zone backcenter 128k;
35         server 192.168.40.108:28000;
36     }
37
38     server {
```

```
39     listen      8081;
40     server_name  server1;
41     server_name  server11;
42
43     location / {
44         add_header x-Cache-Status $upstream_cache_status;
45         add_header x-Cache-Key $scheme$host$request_uri$slice_range;
46         proxy_cache cache;
47
48         proxy_pass http://backgroup;
49     }
50
51     location /a {
52         return 200 "welcome to 8081 http server a\n";
53     }
54 }
55
56 server {
57     listen      8082;
58     server_name server2;
59
60     location / {
61         proxy_pass http://errgroup;
62     }
63
64     location /c {
65         root html;
66         index index.html;
67     }
68
69     location /d {
70         root html;
71         index index.html;
72     }
73
74 }
75
76 server {
77     listen      8083;
78     server_name server3;
79
80     location / {
81         root html;
82         index index.html;
83     }
84
85     location /a {
```

```
86         root html;
87         index index.html;
88     }
89
90     location /b {
91         root html;
92         index index.html;
93     }
94 }
95 }
96
```

3.19.3.6.4 接口示例

使用GET方法获取当前VTS模块的设置，使用PUT 方法更新VTS模块的设置。PUT时使用的报文格式与GET获取到的报文格式一致。

查询当前VTS模块的设置：

```
1  curl -X GET http://127.0.0.1:8443/api/1/config/http_vts
2
3  返回：
4  {
5      "vhost_traffic_status_filter_by_set_key": "\"$request_uri\" \"$server_name\"
6      "servers": [
7          {
8              "listens": [
9                  "0.0.0.0:8081"
10             ],
11             "serverNames": [
12                 "server1",
13                 "server11"
14             ],
15             "locations": [
16                 {
17                     "location": "/",
18                     "vhost_traffic_status": true
19                 },
20                 {
21                     "location": "/a",
22                     "vhost_traffic_status": true
23                 }
24             ]
25         },
```

```
26     {
27         "listens": [
28             "0.0.0.0:8082"
29         ],
30         "serverNames": [
31             "server2"
32         ],
33         "locations": [
34             {
35                 "location": "/",
36                 "vhost_traffic_status": true
37             },
38             {
39                 "location": "/c",
40                 "vhost_traffic_status": true
41             },
42             {
43                 "location": "/d",
44                 "vhost_traffic_status": true
45             }
46         ]
47     },
48     {
49         "listens": [
50             "0.0.0.0:8083"
51         ],
52         "serverNames": [
53             "server3"
54         ],
55         "locations": [
56             {
57                 "location": "/",
58                 "vhost_traffic_status": true
59             },
60             {
61                 "location": "/a",
62                 "vhost_traffic_status": true
63             },
64             {
65                 "location": "/b",
66                 "vhost_traffic_status": true
67             }
68         ]
69     }
70 ]
71 }
```


动态配置：

```
1 curl -X GET http://127.0.0.1:8443/api/1/config/http_vts
2
3 返回：
4 {
5   "vhost_traffic_status_filter_by_set_key": "\"$status\" \"$server_name::$rec
6   "servers": [
7     {
8       "listens": [
9         "0.0.0.0:8081"
10      ],
11      "serverNames": [
12        "server1"
13      ],
14      "locations": [
15        {
16          "location": "/a",
17          "vhost_traffic_status": false
18        }
19      ]
20    }
21  ]
22 }
```

再次查询当前的配置：

```
1 PUT http://127.0.0.1/api/1/config/http_vts HTTP/1.1
2 content-type: application/json
3
4 {
5   "vhost_traffic_status_filter_by_set_key": "\"$status\" \"$server_name::$rec
6   "servers": [
7     {
8       "listens": [
9         "0.0.0.0:8081"
10      ],
11      "serverNames": [
12        "server1",
13        "server11"
14      ],
```

```
15     "locations": [  
16         {  
17             "location": "/",  
18             "vhost_traffic_status": true  
19         },  
20         {  
21             "location": "/a",  
22             "vhost_traffic_status": false  
23         }  
24     ],  
25 },  
26 {  
27     "listens": [  
28         "0.0.0.0:8082"  
29     ],  
30     "serverNames": [  
31         "server2"  
32     ],  
33     "locations": [  
34         {  
35             "location": "/",  
36             "vhost_traffic_status": true  
37         },  
38         {  
39             "location": "/c",  
40             "vhost_traffic_status": true  
41         },  
42         {  
43             "location": "/d",  
44             "vhost_traffic_status": true  
45         }  
46     ],  
47 },  
48 {  
49     "listens": [  
50         "0.0.0.0:8083"  
51     ],  
52     "serverNames": [  
53         "server3"  
54     ],  
55     "locations": [  
56         {  
57             "location": "/",  
58             "vhost_traffic_status": true  
59         },  
60         {  
61             "location": "/a",
```

```

62         "vhost_traffic_status": true
63     },
64     {
65         "location": "/b",
66         "vhost_traffic_status": true
67     }
68 ]
69 }
70 ]
71 }

```

3.19.3.7 动态telemetry

3.19.3.7.1 依赖模块

上文控制面配置说明

 `njt_helper_broker_module`
`njt_ctrl_config_api_module.so;`

本功能需要模块，因为本功能采用外部build，所以版本发布的时候提供对应的so

服务间追踪模块(Otel)依赖:

`njt_otel_module.so`
`njt_agent_dyn_otel_module.so`

3.19.3.7.2 配置指令

无

3.19.3.7.3 配置示例

主要配置示例

```

1
2  # 分布式telemetry 动态开关需要加载这两个模块
3  load_module /root/project/telemetry/njet_opentelemetry/njet_otel_module/build
4  load_module /root/project/telemetry/njet_opentelemetry/njet_otel_module/build
5

```

```
6 worker_processes 1;
7
8 error_log stderr info; # 注意修改为具体的文件路径，此配置为输出到控制台
9
10 helper broker /mnt/f/workspace/c/njet_main/objs/njt_helper_broker_module.so c
11 helper ctrl /mnt/f/workspace/c/njet_main/objs/njt_helper_ctrl_module.so conf/
12
13 events {
14     worker_connections 1024;
15 }
16 cluster_name helper;
17 node_name node1;
18
19 http {
20     dyn_kv_conf conf/iot.conf;
21     access_log logs/access.log combined; ## 此处为必要配置，日志格式可以修改
22
23     # 分布式telemetry模块配置文件
24     opentelemetry_config /home/njet/clbtest/conf/otel.toml;
25
26     upstream demo {
27         zone demo 128k;
28         server 192.168.40.141:8080;
29     }
30
31     upstream demos {
32         zone demos 128k;
33         server 192.168.40.141:443;
34     }
35
36     server {
37         listen 1081;
38
39         location / {
40             # 分布式telemetry 配置
41             opentelemetry on;
42             opentelemetry_operation_name my_example_backend;
43             opentelemetry_propagate;
44
45             root html;
46             index index.html;
47             location /demo {
48                 proxy_pass http://demo;
49             }
50         }
51
52         location = /test/demo {
```

```

53         # 分布式telemetry 配置
54         opentelemetry on;
55         opentelemetry_operation_name my_example_backend;
56         opentelemetry_propagate;
57
58         proxy_pass http://demo;
59     }
60 }
61 server {
62     listen      1080;
63
64     location / {
65         root html;
66         index index.html;
67     }
68
69     location = /test/demo {
70         limit_conn addr 100;
71         proxy_pass http://demo;
72     }
73
74     location /demo {
75         add_header test api;
76         proxy_pass http://demo;
77     }
78     location /demos {
79         limit_conn addr 100;
80         proxy_ssl_certificate nginx.pem;
81         proxy_ssl_certificate_key nginx-key.pem;
82         proxy_pass https://demos;
83     }
84 }
85
86 }

```

mqtt.conf

```

1 log_type error

```

iot.conf

```

1 log_type error

```

3.19.3.8 动态证书

3.19.3.8.1 简介

该模块支持在运行中替换http server的服务端证书。

3.19.3.8.2 依赖模块

 njt_dyn_ssl_module.so

3.19.3.8.3 配置指令

无

3.19.3.8.4 配置示例

主要配置示例

 注意主配置文件中需要修改so路径，log路径，替换ssl证书

```
1 load_module /mnt/f/workspace/c/njet_main/objs/njt_dyn_ssl_module.so;
2 worker_processes 1;
3 daemon off;
4 master_process on;
5 error_log stderr info;
6 helper broker /mnt/f/workspace/c/njet_main/objs/njt_helper_broker_module.so c
7 helper ctrl /mnt/f/workspace/c/njet_main/objs/njt_helper_ctrl_module.so conf/
8
9 events {
10     worker_connections 1024;
11 }
12 cluster_name helper;
13 node_name node1;
14 http {
15     access_log logs/access.log combined;
16     dyn_kv_conf conf/iot.conf;
17     upstream demo {
18         zone demo 128k;
19         server 192.168.40.141:8080;
20         keepalive 10240;
21     }
22     server {
23         listen 1080 ssl;
24         ssl_certificate_key 40.206-key.pem;
```

```

25         ssl_certificate      40.206.pem;
26
27         location / {
28             add_header test api;
29             proxy_set_header Conection "";
30             proxy_http_version 1.1;
31             proxy_pass http://demo;
32         }
33     }
34 }

```

mqtt.conf

```

1 log_type error

```

iot.conf

```

1 log_type error

```

3.19.3.8.5 接口文档

字段详细说明参见：<http://192.168.40.108:9000/#7>

3.19.3.8.6 接口示例

3.19.3.8.6.1 查询http server ssl 当前配置

该接口可以查看HTTP 块下所有server的证书配置情况

请求

```

1 GET http://127.0.0.1:8443/config/1/config/http_ssl

```

返回值

```

1 HTTP/1.1 200 OK
2 Server: njet/1.23.1
3 Date: Fri, 03 Mar 2023 08:42:22 GMT
4 Content-Type: application/json
5 Content-Length: 282

```

```

6 Connection: keep-alive
7
8 {
9   "servers": [
10    {
11      "listens": [
12        "0.0.0.0:1080"
13      ],
14      "serverNames": [
15        ""
16      ],
17      "certificates": [
18        {
19          "certificate": "./conf/40.206.pem",
20          "certificateKey": "./conf/40.206-key.pem"
21        }
22      ]
23    },
24    {
25      "listens": [
26        "0.0.0.0:1090"
27      ],
28      "serverNames": [
29        ""
30      ],
31      "certificates": [
32        {
33          "certificate": "$host.pem",
34          "certificateKey": "$host-key.pem"
35        }
36      ]
37    }
38  ]
39 }

```

3.19.3.8.6.2 更新http server ssl当前证书

请求



注意事项

该接口不是声明式全量配置，多次配置证书会存在内存泄露，所以更新json内容应为最小变更

1 PUT http://127.0.0.1:8443/config/1/config/http_ssl


```
2 Content-Type: application/json
3
4 {
5   "servers": [
6     {
7       "listens": [
8         "0.0.0.0:1080"
9       ],
10      "serverNames": [
11        ""
12      ],
13      "certificates": [
14        {
15          "certificate": "data:-----BEGIN CERTIFICATE-----\r\nMIIEITCCAwmGAWI
16          "certificateKey": "data:-----BEGIN RSA PRIVATE KEY-----\r\nMIIEogIE
17        }
18      ]
19    }
20  ]
21 }
```

3.20 边车

3.18.1 概述

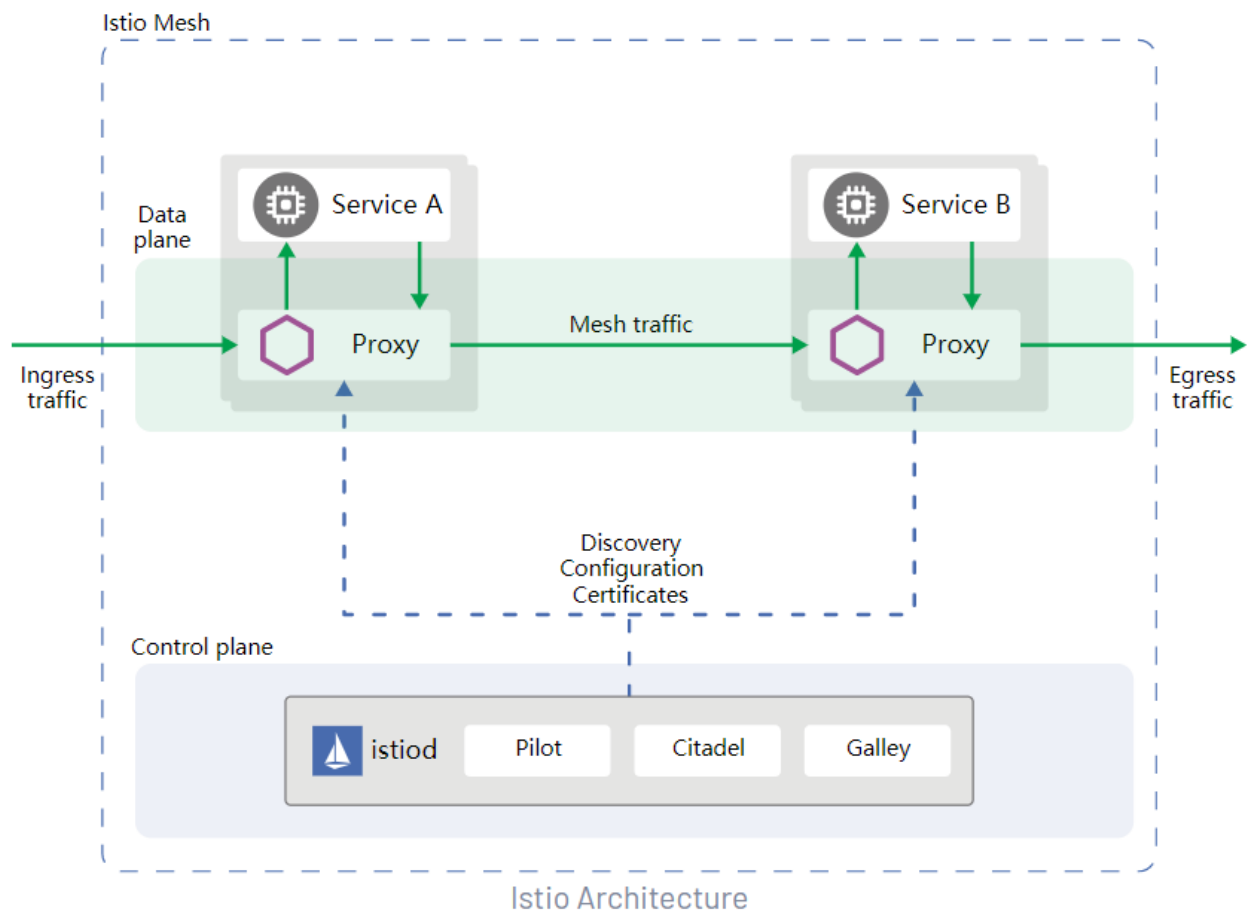
sidecar njet作为istio 架构中的数据面，旨在替换原生数据面sidecar envoy。本文档基于 istio 1.13.3 版本编写，主要针对sidecar njet对istio proxy功能特性的支持进行了说明。

3.18.1.1 istio架构

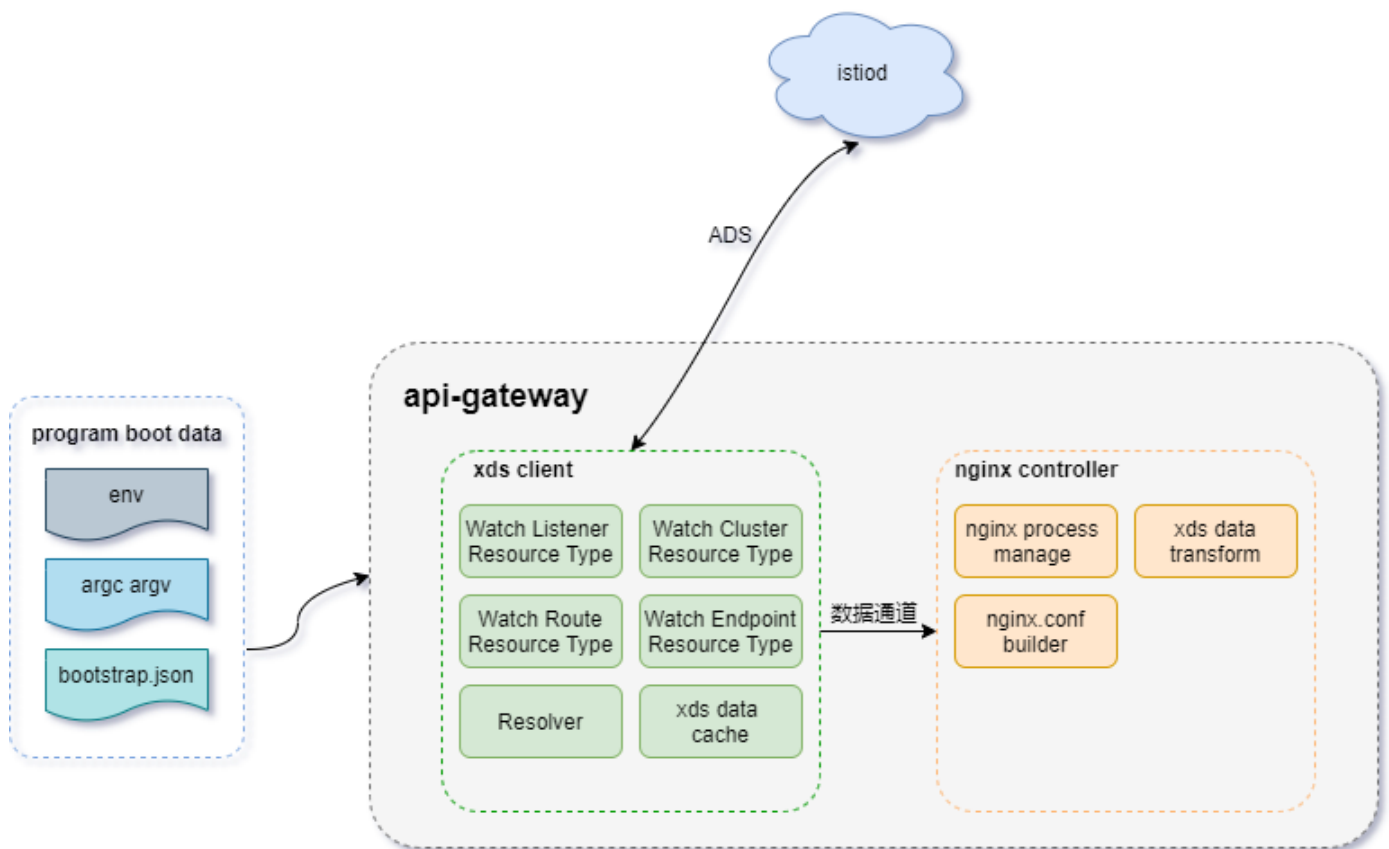
Istio服务网格在逻辑上分为数据平面和控制平面。

- 数据平面由一组作为sidecars部署的智能代理(Envoy)组成。这些代理调解和控制微服务之间的所有网络通信。他们还收集并报告所有网格流量的遥测数据。
- 控制平面对代理进行路由管理和配置。

下图显示了组成每个平面的不同组件:



3.18.1.2 sidecar njet架构



api-gateway有两个主要模块：

- xds 客户端
主要与istiod xds服务端通信，进行xds资源的订阅，基于grpc-go xds实现，grpc支持SotW ADS
- NJet控制器
 - 管理NJet进程，包括启动，reload
 - xds资源类型数据进行解析、处理，生成NJet server、upstream、loction元数据，通过go tpl技术生成NJet.conf文件
 - 集成lua

3.18.2 镜像构建

通过git submodule，下载njet_main和lua_module源码，这两个仓库是njet proxy源码仓库。

```
1 cd api-gateway
2 git submodule init
3 git submodule update
```

执行成功后，源码被下载到istio-agent/docker/njet_lua_src/

 cd 到istio-agent/docker/njet_lua_src/目录，目录下为lua源码与njet源码子目录，进入到子目录，切换到对应的分支，确保njet源码是您期望的。

```
1 cd api-gateway/istio-agent/docker
```

执行

```
1 make docker_api_gateway
```

执行上述命令后，sidecar image开始构建，等待片刻，image会被构建生成。出现如下提示代表成功。

```

---> 41a58fb4ccf6
Step 19/24 : ENV API_GATEWAY_LOG_SEVERITY_LEVEL=error
---> Running in 611e3ddef179
Removing intermediate container 611e3ddef179
---> 850351843088
Step 20/24 : ENV GRPC_GO_LOG_SEVERITY_LEVEL=info
---> Running in 1553a6c16a9d
Removing intermediate container 1553a6c16a9d
---> 9e9e3d0f080f
Step 21/24 : ENV API_GATEWAY_BOOTSTRAP=$WORKPATH/bootstrap.json
---> Running in d9b1d3a1cdb0
Removing intermediate container d9b1d3a1cdb0
---> 13a1ae03e6d4
Step 22/24 : WORKDIR /
---> Running in 3d2a6901a680
Removing intermediate container 3d2a6901a680
---> 9122c596da76
Step 23/24 : USER 1337:1337
---> Running in a2020973d9b6
Removing intermediate container a2020973d9b6
---> f11097bc860c
Step 24/24 : CMD ["/home/tongminghu/api-gateway/api-gateway"]
---> Running in c18b9c9d52c0
Removing intermediate container c18b9c9d52c0
---> e13f7ab233cf
Successfully built e13f7ab233cf
Successfully tagged tmlake/sidecar-njet-httprouting_ubuntu:v1.0
root@ubuntu:/home/lq/work/tongmingzhiyun/api-gateway/istio-agent/docker#
root@ubuntu:/home/lq/work/tongmingzhiyun/api-gateway/istio-agent/docker#
root@ubuntu:/home/lq/work/tongmingzhiyun/api-gateway/istio-agent/docker#

```

3.18.3 部署sidecar

使用sidecar njet需要定制化部署istio，自动注入njet sidecar。定制化部署istio需要使用istio-install仓库中的文件，此仓库是基于istio-1.13.3 release修改的，需要修改sidecar镜像名称，修改sidecar 容器模板。

3.18.3.1 安装istio

拉取istio-install仓库，修改default-istio.yaml文件values.global.proxy.image字段，设置为njet的镜像名称，必须为包含“/”。执行如下命令安装istiod：

```
1 ./istio-1.13.3/bin/istioctl install --manifests=istio-1.13.3/manifests/ -f d
```

3.18.3.1注入sidecar njet

自动注入njet sidecar(支持namespace、pod级别)，在pod被创建时被触发。

Pod level

给pod打标贴：sidecar.istio.io/inject: "true"，以productpage为例，修改deploy，增加标贴。

```
1 kubectl edit deploy productpage-v1
```

```
5 apiVersion: apps/v1
6 kind: Deployment
7 metadata:
8   annotations:
9     deployment.kubernetes.io/revision: "6"
10    kubectl.kubernetes.io/last-applied-configuration: |
11      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotation
12      etadata":{"labels":{"app":"productpage","version":"v1"},"spec":{"contain
13      volumeMounts":[{"mountPath":"/tmp","name":"tmp"}]}],"serviceName":
14    creationTimestamp: "2022-10-24T06:16:35Z"
15    generation: 6
16    labels:
17      app: productpage
18      version: v1
19    name: productpage-v1
20    namespace: default
21    resourceVersion: "6526320"
22    uid: 2d90c75c-a545-4b01-b691-d861388de033
23 spec:
24   progressDeadlineSeconds: 600
25   replicas: 1
26   revisionHistoryLimit: 10
27   selector:
28     matchLabels:
29       app: productpage
30       version: v1
31   strategy:
32     rollingUpdate:
33       maxSurge: 25%
34       maxUnavailable: 25%
35     type: RollingUpdate
36   template:
37     metadata:
38       creationTimestamp: null
39     labels:
40       app: productpage
41       sidecar.istio.io/inject: "true"
42     spec:
43       containers:
44         - image: docker.io/istio/examples-bookinfo-productpage-v1:1.16.2
45           imagePullPolicy: IfNotPresent
46           name: productpage
47           ports:
```

等待一会

```

root@ubuntu:/home/lq/work/istio# kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-5498c86cf5-6kcpw        1/1     Running   0           100m
productpage-v1-778c664bbd-4ddj9    2/2     Running   0           89m
ratings-v1-b477cf6cf-m4fz4        1/1     Running   0           100m
reviews-v1-79d546878f-bcrqs        1/1     Running   0           6d3h
reviews-v2-548c57f459-tbpj4        1/1     Running   0           6d3h
reviews-v3-6dd79655b9-wmldb        1/1     Running   0           6d3h

```

Namespace level

添加namespace label,以便后续在此ns部署应用时,通知istio自动注入nginx sidecar, 下面为default ns添加标贴。

```
1 kubectl label namespace default istio-injection=enabled
```

查看ns标贴

```
1 kubectl get ns --show-labels
```

```

root@ubuntu:/home/lq/work/istio# kubectl get ns --show-labels
NAME          STATUS   AGE   LABELS
api-gateway   Active   6d23h  kubernetes.io/metadata.name=api-gateway
default       Active   7d4h   istio-injection=enabled,kubernetes.io/metadata.name=default
istio-system  Active   133m   kubernetes.io/metadata.name=istio-system
kube-node-lease Active   7d4h   kubernetes.io/metadata.name=kube-node-lease
kube-public   Active   7d4h   kubernetes.io/metadata.name=kube-public
kube-system   Active   7d4h   kubernetes.io/metadata.name=kube-system
kubernetes-dashboard Active   7d4h   addonmanager.kubernetes.io/mode=Reconcile,kubernetes.io/metadata.name=kubernetes-dashboard,kubernetes.io/minikube-addons=dashboard
root@ubuntu:/home/lq/work/istio#
root@ubuntu:/home/lq/work/istio#

```

查看default ns pod状态

```

root@ubuntu:/home/lq/work/istio# kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-5498c86cf5-6kcpw        1/1     Running   0           100m
productpage-v1-778c664bbd-4ddj9    2/2     Running   0           89m
ratings-v1-b477cf6cf-m4fz4        1/1     Running   0           100m
reviews-v1-79d546878f-bcrqs        1/1     Running   0           6d3h
reviews-v2-548c57f459-tbpj4        1/1     Running   0           6d3h
reviews-v3-6dd79655b9-wmldb        1/1     Running   0           6d3h
root@ubuntu:/home/lq/work/istio#

```

删除details pod, 触发sidecar注入

```
1 kubectl delete pod details-v1-5498c86cf5-6kcpw
```

查看default ns pod状态, details pod注入NJet sidecar

```
root@ubuntu:/home/lq/work/istio# kubectl get pod
NAME                                READY    STATUS    RESTARTS    AGE
details-v1-5498c86cf5-m4qwj        2/2     Running   0            8s
productpage-v1-778c004bb0-4ddj9    2/2     Running   0           99m
ratings-v1-b477cf6cf-m4fz4         1/1     Running   0          110m
reviews-v1-79d546878f-bcrqs        1/1     Running   0           6d4h
reviews-v2-548c57f459-tbpj4        1/1     Running   0           6d4h
reviews-v3-6dd79655b9-wmldb        1/1     Running   0           6d4h
root@ubuntu:/home/lq/work/istio#
```

3.18.3.2 卸载istio

官方使用如下命令卸载istio,但是部分cm和sa没有被删除

```
1 istioctl x uninstall --purge
```



```
root@ubuntu:/home/lq/work/tongmingzhiyun/istio-install# istioctl x uninstall --purge
All Istio resources will be pruned from the cluster
Proceed? (y/N) y
Removed IstioOperator:istio-system:installed-state.
Removed HorizontalPodAutoscaler:istio-system:istiod.
Removed PodDisruptionBudget:istio-system:istiod.
Removed Deployment:istio-system:istiod.
Removed Service:istio-system:istiod.
Removed ConfigMap:istio-system:istio.
Removed ConfigMap:istio-system:istio-sidecar-injector.
Removed Pod:istio-system:istiod-8488b9bdc7-mxcxj.
Removed ServiceAccount:istio-system:istio-reader-service-account.
Removed ServiceAccount:istio-system:istiod.
Removed ServiceAccount:istio-system:istiod-service-account.
Removed RoleBinding:istio-system:istiod.
Removed RoleBinding:istio-system:istiod-istio-system.
Removed Role:istio-system:istiod.
Removed Role:istio-system:istiod-istio-system.
Removed EnvoyFilter:istio-system:stats-filter-1.11.
Removed EnvoyFilter:istio-system:stats-filter-1.12.
Removed EnvoyFilter:istio-system:stats-filter-1.13.
Removed EnvoyFilter:istio-system:tcp-stats-filter-1.11.
Removed EnvoyFilter:istio-system:tcp-stats-filter-1.12.
Removed EnvoyFilter:istio-system:tcp-stats-filter-1.13.
Removed MutatingWebhookConfiguration::istio-revision-tag-default.
Removed MutatingWebhookConfiguration::istio-sidecar-injector.
Removed ValidatingWebhookConfiguration::istio-validator-istio-system.
Removed ValidatingWebhookConfiguration::istiod-default-validator.
Removed ClusterRole::istio-reader-clusterrole-istio-system.
Removed ClusterRole::istio-reader-istio-system.
Removed ClusterRole::istiod-clusterrole-istio-system.
Removed ClusterRole::istiod-gateway-controller-istio-system.
Removed ClusterRole::istiod-istio-system.
Removed ClusterRoleBinding::istio-reader-clusterrole-istio-system.
Removed ClusterRoleBinding::istio-reader-istio-system.
Removed ClusterRoleBinding::istiod-clusterrole-istio-system.
Removed ClusterRoleBinding::istiod-gateway-controller-istio-system.
Removed ClusterRoleBinding::istiod-istio-system.
Removed CustomResourceDefinition::authorizationpolicies.security.istio.io.
Removed CustomResourceDefinition::destinationrules.networking.istio.io.
Removed CustomResourceDefinition::envoyfilters.networking.istio.io.
Removed CustomResourceDefinition::gateways.networking.istio.io.
Removed CustomResourceDefinition::istiooperators.install.istio.io.
Removed CustomResourceDefinition::peerauthentications.security.istio.io.
Removed CustomResourceDefinition::proxyconfigs.networking.istio.io.
Removed CustomResourceDefinition::requestauthentications.security.istio.io.
Removed CustomResourceDefinition::serviceentries.networking.istio.io.
Removed CustomResourceDefinition::sidecars.networking.istio.io.
Removed CustomResourceDefinition::telemetries.telemetry.istio.io.
Removed CustomResourceDefinition::virtualservices.networking.istio.io.
Removed CustomResourceDefinition::wasmpplugins.extensions.istio.io.
Removed CustomResourceDefinition::workloadentries.networking.istio.io.
Removed CustomResourceDefinition::workloadgroups.networking.istio.io.
✔ Uninstall complete
```

删除istio-system ns，可以删除istio-system ns下的所有资源

```
1 kubectl delete namespace istio-system
```


3.18.3.3 更新sidecar njet镜像

安装istio，sidecar成功注入后，当修改了代码重新构建了不同名称的镜像后，需要替换镜像，istio 通过ConfigMap去更新sidecar镜像，操作如下：

```
1 [root@node151 ~]# kubectl get cm -n istio-system
2 NAME                                DATA   AGE
3 istio                                2       121d
4 istio-ca-root-cert                  1       121d
5 istio-gateway-deployment-leader     0       121d
6 istio-gateway-status-leader         0       121d
7 istio-leader                        0       121d
8 istio-namespace-controller-election 0       121d
9 istio-sidecar-injector              2       121d
10 kube-root-ca.crt                   1       121d
11 [root@node151 ~]#
```

通过修改名为istio-sidecar-injector的 ConfigMap资源中的配置项，来修改sidecar镜像。修改的配置项为data.values.proxy.image字段，格式如下：

通过如下指令进行编辑，保存即可。

```
1 kubectl edit cm -n istio-system istio-sidecar-injector
```

```
values: |-
{
  "global": {
    "caAddress": "",
    "caName": "",
    "configCluster": false,
    "configValidation": true,
    "defaultNodeSelector": {},
    "defaultPodDisruptionBudget": {
      "enabled": true
    },
    "defaultResources": {
      "requests": {
        "cpu": "10m"
      }
    },
    "enabled": true,
    "externalIstiod": false,
    "hub": "docker.io/istio",
    "imagePullPolicy": "",
    "imagePullSecrets": [],
    "istioNamespace": "istio-system",
    "istiod": {
      "enableAnalysis": false
    },
    "jwtPolicy": "third-party-jwt",
    "logAsJson": false,
    "logging": {
```

```

    "level": "default:info"
  },
  "meshID": "",
  "meshNetworks": {},
  "mountMtlsCerts": false,
  "multiCluster": {
    "clusterName": "",
    "enabled": false
  },
  "namespace": "istio-system",
  "network": "",
  "omitSidecarInjectorConfigMap": false,
  "oneNamespace": false,
  "operatorManageWebhooks": false,
  "pilotCertProvider": "istiod",
  "priorityClassName": "",
  "proxy": {
    "autoInject": "enabled",
    "clusterDomain": "cluster.local",
    "componentLogLevel": "misc:error",
    "enableCoreDump": false,
    "excludeIPRanges": "",
    "excludeInboundPorts": "",
    "excludeOutboundPorts": "",
    "holdApplicationUntilProxysStarts": false,
    "image": "tmlake/sidecar-njet-httprounting ubuntu:v1.0",
    "includeIPRanges": "*",
    "includeInboundPorts": "*",
    "includeOutboundPorts": "",
    "logLevel": "warning",
    "privileged": true,
    "readinessFailureThreshold": 30,
    "readinessInitialDelaySeconds": 1,
    "readinessPeriodSeconds": 2,
    "resources": {
      "limits": {
        "cpu": "2000m",
        "memory": "1024Mi"
      },
      "requests": {
        "cpu": "100m",
        "memory": "128Mi"
      }
    },
    "statusPort": 15020,
    "tracer": "zipkin"
  },
  "proxy_init": {
    "image": "docker.io/istio/proxyv2:1.13.3"
  }
}

```

3.18.4 流量管理

3.18.4.1 协议支持

目前支持代理HTTP1。

协议类型	支持情况
HTTP1	支持
HTTP2	不支持
HTTPS	不支持

TCP	不支持
TLS	不支持

3.18.4.2 HTTP路由

功能描述

通过丰富的路由规则对HTTP流量实施细粒度的流量控制。使用路由规则告诉njet如何将虚拟服务的流量发送到适当的目的地，路由目的地可以是相同服务的版本，也可以是完全不同的服务。路由目的地支持按权重分流。

高级的路由可以实现灰度发布，发送到灰度版本的流量是基于请求的属性或者是随机选择的，按照请求的百分比进行。

Istio sidecar njet支持istio 控制面HTTP路由匹配规则，目前支持的路由匹配规则如下表所述：

字段名称	字段类型	Istio njet sidecar支持情况
uri	支持exact、prefix、regex	支持exact、prefix、regex
scheme	支持exact、prefix、regex	支持exact
method	支持exact、prefix、regex	支持exact
authority	支持exact、prefix、regex	不支持
headers	支持exact、prefix、regex	支持exact
queryParams	exact、regex	支持exact、regex
ignoreUriCase	uri大小写忽略flag(bool)， 针对于exact、prefix	不支持
withoutHeaders	支持exact、prefix、regex	支持exact

Istio 控制面HTTP路由匹配接口定义参照：

<https://istio.io/v1.13/docs/reference/config/networking/virtual-service/#HTTPMatchRequest>

Njet HTTP路由实现

njet通过map+rewrite+split_clients实现基于请求内容的高级路由，比如基于请求的:header、method、queryParams。

创建一个基于header的灰度路由

```

1  apiVersion: networking.istio.io/v1beta1
2  kind: VirtualService
3  metadata:
4    name: reviews
5    namespace: default
6  spec:
7    hosts:
8      - reviews
9    http:
10     - match:
11       - uri:
12         exact: /reviews/0
13       route:
14         - destination:
15           host: reviews
16           subset: v3
17     - match:
18       - headers:
19         end-user:
20           exact: jason
21       method:
22         exact: get
23       uri:
24         exact: /reviews/0
25       route:
26         - destination:
27           host: reviews
28           subset: v2

```

- 请求中不携带header->end-user: jason，请求被路由到reviews|v3| upstream，即reviews v3版本，v3会返回红色的星星，在productpage pod中的业务容器中通过curl 访问reviews:9080 host，指令如下：

```
1 curl http://reviews:9080/reviews/0
```



curl请求服务时，主机域名根据实际情况填写，查看njet VirtualHOST 配置的域名列表，即server 块的server_name指令。本测试中名称如下：

```
server_name reviews.default.svc.cluster.local reviews.default.svc.cluster.local:9080
reviews reviews:9080 reviews.default.svc reviews.default.svc:9080 reviews.default
reviews.default:9080 10.233.213.122 10.233.213.122:9080 ;
```

返回结果如下图所示，可知，与期望一致。

```
root@productpage-v1-6c594c7b95-qgg8k:/opt/microservices# curl http://reviews:9080/reviews/0
{"id": "0", "reviews": [{"reviewer": "Reviewer1", "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!", "rating": {"stars": 5, "color": "red"}}, {"reviewer": "Reviewer2", "text": "A decent play, but not as good as the first. The slapstick humour is refreshing!", "rating": {"stars": 4, "color": "red"}}]}root@productpage-v1-6c594c7b95-qgg8k:/opt/microservices#
```

- 请求中携带header->end-user: jason，请求被路由到reviews|v2| upstream，即reviews v2版本，v2会返回5颗黑色的星星，在productpage pod中的业务容器中通过curl 访问 reviews:9080 host，指令如下：

```
1 curl http://reviews:9080/reviews/0 -H "end-user:jason"
```

返回结果如下图所示，可知，与期望一致。

```
root@productpage-v1-6c594c7b95-qgg8k:/opt/microservices# curl http://reviews:9080/reviews/0 -H "end-user:jason"
{"id": "0", "reviews": [{"reviewer": "Reviewer1", "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!", "rating": {"stars": 5, "color": "black"}}, {"reviewer": "Reviewer2", "text": "A decent play, but not as good as the first. The slapstick humour is refreshing!", "rating": {"stars": 4, "color": "black"}}]}root@productpage-v1-6c594c7b95-qgg8k:/opt/microservices#
```

分流

```
1 apiVersion: networking.istio.io/v1beta1
2 kind: VirtualService
3 metadata:
4   name: reviews
5   namespace: default
6 spec:
7   hosts:
8     - reviews
9   http:
10    - match:
11      - method:
12          exact: GET
13        uri:
14          prefix: /productpage
15      route:
16        - destination:
17            host: productpage
18            subset: v1
19            weight: 80
20        - destination:
21            host: reviews
22            subset: v3
23            weight: 20
```

match指明，当请求方法为GET，且uri有/productpage前缀时，请求按权重被路由到productpage|v1|和reviews|v3| upstream。

在productpage pod中的业务容器中通过curl 访问reviews:9080 host，指令如下：

```
1 curl http://reviews:9080/productpage -v
```

返回结果如下图所示，可知，与期望一致。

```
<!-- empty stars: -->
</font>

</blockquote>
<blockquote>
  <p>Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare.</p>
  <small>Reviewer2</small>

  <font color="red">
    <!-- full stars: -->

    <span class="glyphicon glyphicon-star"></span>
    <span class="glyphicon glyphicon-star"></span>
    <span class="glyphicon glyphicon-star"></span>
    <span class="glyphicon glyphicon-star"></span>

    <!-- empty stars: -->

    <span class="glyphicon glyphicon-star-empty"></span>
  </font>

</blockquote>

</div>
</div>
</div>

- Latest compiled and minified JavaScript -->
<script src="static/jquery.min.js"></script>

- Latest compiled and minified JavaScript -->
<script src="static/bootstrap/js/bootstrap.min.js"></script>

<script type="text/javascript">
  ('#login-modal').on('shown.bs.modal', function () {
    $('#username').focus();
  });
</script>
</body>
```

由图可知，成功返回productpage页面。

反复执行如下命令，会出现404响应。

```
1 curl http://reviews:9080/productpage -v
```

```
* Trying 10.233.213.122...
* TCP_NODELAY set
* Expire in 200 ms for 4 (transfer 0x5607ce22ff50)
* Connected to reviews (10.233.213.122) port 9080 (#0)
> GET /productpage HTTP/1.1
> Host: reviews:9080
> User-Agent: curl/7.64.0
> Accept: */*
>
< HTTP/1.1 404 Not Found
< Server: njet/1.23.1
< Date: Wed, 08 Mar 2023 10:12:20 GMT
< Content-Length: 0
< Connection: keep-alive
< X-Powered-By: Servlet/3.1
< Allow: GET,OPTIONS,HEAD
< Content-Language: en-US
< proxy-active-upstream-name: 123
<
* Connection #0 to host reviews left intact
root@productpage-v1-6c594c7b95-qgg8k:/opt/microservices#
```

```
TRAS0036E: The system could not create file /tmp/logs/messages.log because of the following exception: java.security.PrivilegedActionException: java.io.IOException: Permission denied
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
TRAS0036E: The system could not create file /tmp/logs/messages.log because of the following exception: java.security.PrivilegedActionException: java.io.IOException: Permission denied
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*, Please enable FINE/TRACE log level for more details.
```

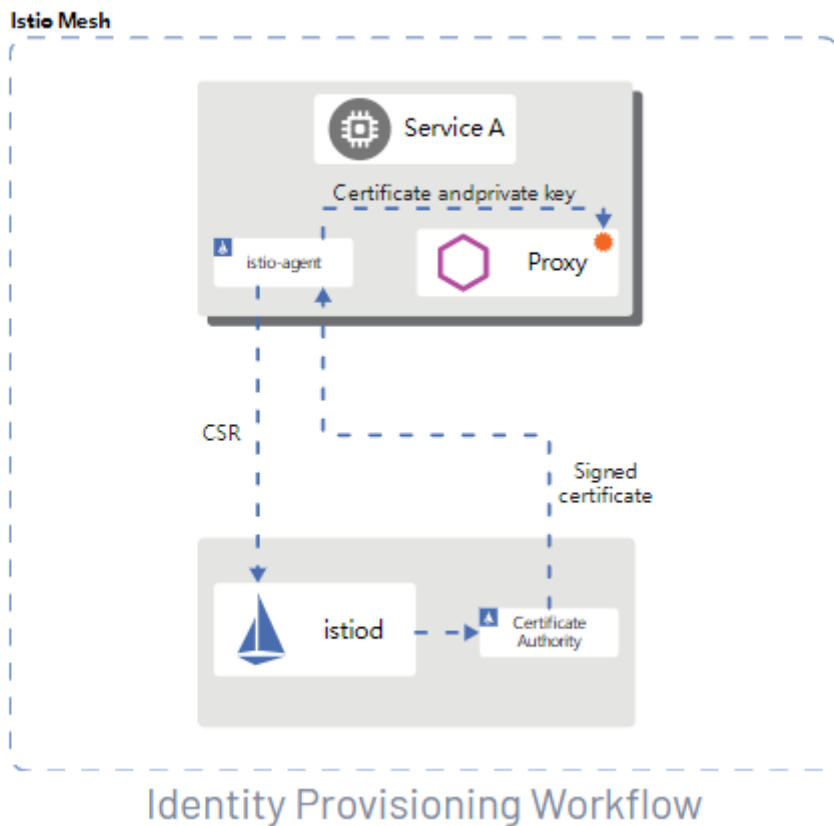
由图可知，请求被路由到reviews v3版本，因为reviews服务没有实现/productpage，所以404属于正常。多次请求中，大部分会返回productpage首页，从而说明按权重分流已生效。

3.18.5 安全

3.18.5.1 证书管理

功能描述

Istio使用X.509证书安全地为每个工作负载提供强身份。与每个njet(Envoy) proxy一起运行的Istio agent与istiod一起工作，实现大规模自动化密钥和证书轮换。下图显示了身份供应流程。



Istio通过以下流程提供密钥和证书:

- **istiod**提供gRPC服务来接收证书签名请求(csr)。
- 启动时, Istio agent创建私钥和CSR, 然后将带有凭证的CSR发送给istiod进行签名。
- istiod中的CA验证CSR中携带的凭据。在成功验证之后, 它签署CSR以生成证书。
- **这一步, njet与Envoy proxy不同**, Envoy支持SDS动态获取, Envoy通过Envoy SDS API从同一个容器中的Istio agent 请求证书和密钥。Istio agent通过Envoy SDS API将从Istio接收到的证书和私钥发送给Envoy。njet的策略是直接在njet.conf中通过指令配置证书文件及私钥文件, 证书文件及私钥文件会被Istio agent写到 /var/lib/istio/data/。
- Istio agent监视证书的过期。对于证书和密钥轮换, 定期重复上述过程。



njet sidecar目前不支持动态证书加载, 即证书轮换后, njet不会动态加载, 此功能目前未实现

看证书定期轮换

Istio 默认签发的证书为24h有效期, 生成的证书链和根证书及key文件被存放到sidecar容器的/var/lib/istio/data目录下, 如下图所示:


```
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$ ls
cert-chain.pem key.pem nginx-cert root-cert.pem
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$ pwd
/var/lib/istio/data
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$
```

a. 可以通过查看文件生成时间，来确认证书已经轮换，如下所示：

```
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$ ll
total 12
drwxrwxrwx. 3 root      root      82 Feb 21 10:02 ./
drwxr-xr-x. 3 root      root      18 Feb 21 10:02 ../
-rw-r--r--. 1 istio-proxy istio-proxy 2397 Feb 21 10:02 cert-chain.pem
-rw-r--r--. 1 istio-proxy istio-proxy 1679 Feb 21 10:02 key.pem
drwxrwxr-x. 4 istio-proxy istio-proxy 110 Nov 16 05:40 nginx-cert/
-rw-r--r--. 1 istio-proxy istio-proxy 1098 Feb 21 10:02 root-cert.pem
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$
```

b. 或者到期后，通过openssl 命令查看证书过期时间是否修改，如下所示：

```
1 openssl x509 -noout -text -in /var/lib/istio/data/cert-chain.pem
```

```

istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$ openssl x509 -noout -text -in /var/lib/istio/data/cert-chain.pem
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            cf:fe:c2:d8:ff:b4:22:4b:fa:db:ea:b8:62:b3:65:0e
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = cluster.local
        Validity
            Not Before: Feb 21 10:00:09 2023 GMT
            Not After : Feb 24 10:02:09 2023 GMT
        Subject: CN = spiffe://cluster.local/ns/default/sa/bookinfo-productpage
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            RSA Public-Key: (2048 bit)
            Modulus:
                00:cd:50:c8:b7:fd:12:7c:64:fa:0b:7c:ca:ae:68:
                14:73:a4:6c:76:61:34:11:35:23:b9:5e:0d:3c:1b:
                5c:72:9f:90:5b:30:90:82:cd:c0:79:af:7d:70:37:
                7e:14:71:31:ca:9b:d2:f7:5b:aa:97:e3:7e:8d:d4:
                62:0d:f6:c7:54:30:74:96:1b:ee:21:be:f4:2a:7d:
                ad:f2:a5:4d:3e:83:9b:5b:45:c9:3a:4d:35:2b:36:
                27:c2:bd:9f:28:9d:d4:1c:3e:22:9f:41:73:c1:b0:
                86:84:74:01:ec:c7:80:aa:6c:9a:30:76:3a:9a:0f:
                5d:00:19:b7:53:c4:67:cd:cd:bb:9b:c6:ed:57:28:
                ee:d0:ab:5d:a6:8e:ee:c5:73:aa:12:d3:d4:d4:20:
                d4:16:56:96:c4:72:66:a8:ae:22:d4:f9:25:ce:d8:
                77:3f:b9:10:62:3a:74:41:23:fc:e3:a9:76:4e:af:
                13:6d:cb:51:e1:0d:ee:07:7e:d3:2a:6c:3a:c4:c5:
                d8:3a:0d:81:4b:37:51:11:02:59:f1:77:e3:32:ea:
                e9:1d:85:87:40:60:1e:9e:45:3f:f1:62:8d:ee:6b:
                a3:a5:25:c6:fa:7a:69:b2:1a:ff:dd:30:e8:5e:44:
                8b:38:26:c4:96:12:46:b5:55:4c:f9:2c:02:67:af:
                50:bd
            Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication, TLS Web Client Authentication
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Authority Key Identifier:
                keyid:28:74:62:8E:4A:F3:1C:4A:44:6D:9B:6A:81:20:B1:29:32:53:79:8E

            X509v3 Subject Alternative Name: critical
                URI:spiffe://cluster.local/ns/default/sa/bookinfo-productpage
        Signature Algorithm: sha256WithRSAEncryption
            87:a5:ba:3e:fd:8b:75:a0:2d:fd:26:af:1a:ac:86:65:9b:b6:
            4c:07:d8:b6:2d:1d:ae:69:3a:13:b3:4c:db:f2:74:f1:60:f8:
            25:4c:23:7c:7d:57:0f:06:1c:47:ca:57:61:2c:9f:fe:03:67:
            54:fb:80:22:a7:fc:1c:5d:3c:e7:2b:ba:54:52:30:ed:c2:68:
            7e:87:fb:b3:55:6d:fa:df:c3:0a:a8:a3:a7:60:88:f5:ae:f2:
            da:02:d5:57:a9:76:10:b9:47:ec:c2:37:83:3d:b7:e9:89:67:
            b9:61:41:3b:62:8c:f7:b0:17:87:3e:8a:86:d8:ca:2b:df:17:
            ee:67:66:e2:e6:58:1c:d9:52:ea:dd:81:39:37:a2:46:f9:1c:
            3b:5b:fa:7f:3b:29:2d:62:22:10:a3:5d:4d:c8:66:0e:70:43:
            bb:9e:07:f1:25:21:9c:7c:09:5f:fc:5e:a3:dc:01:3d:3b:6b:
            fb:6c:e1:81:2f:ce:c6:e1:cc:c0:fb:b3:3a:9b:07:cc:1b:ee:
            7b:a5:d6:0e:4d:07:62:e3:96:81:06:15:39:aa:95:db:c0:b3:
            9b:4c:70:fc:21:9c:a4:12:5e:15:ce:7b:09:f3:63:9b:09:d6:
            d0:d6:0d:de:cf:6f:2f:3b:bc:d3:25:f1:b7:85:11:b4:3d:29:
            9a:43:df:ba
istio-proxy@productpage-v1-7986cd44c7-6wdx5:/var/lib/istio/data$

```

我修改了证书有效期为3天，默认为1天

修改签发证书的有效期时间

通过修改istio-sidecar-injector k8s ConfigMap资源中的配置项，来改变证书的有效期日期。配置项为

data.config.templates.sidecar.spec.containers 里的istio-proxy容器中的env变量SECRET_TTL的value，格式为1h（小时）、10m(分钟)。修改后delete pod，重新注入sidecar，即可生效。执行下面指令修改配置(记得修改后保存)：

```
1 kubectl edit cm -n istio-system istio-sidecar-injector
```

```

{{ end }}
containers:
- name: istio-proxy
  {{- if contains "/" (annotation .ObjectMeta `sidecar.istio.io/proxyImage` .Values
    image: "{{ annotation .ObjectMeta `sidecar.istio.io/proxyImage` .Values.global
  {{- else }}
    image: "{{ .ProxyImage }}"
  {{- end }}
  ports:
    - containerPort: 15090
      protocol: TCP
      name: http-envoy-prom
  {{- if .Values.global.proxy.lifecycle }}
    lifecycle:
      {{ toYaml .Values.global.proxy.lifecycle | indent 6 }}
  {{- else if $holdProxy }}
    lifecycle:
      postStart:
        exec:
          command:
            - pilot-agent
            - wait
  {{- end }}
  env:
    {{- if eq (env "PILOT_ENABLE_INBOUND_PASSTHROUGH" "true") "false" }}
    - name: REWRITE_PROBE_LEGACY_LOCALHOST_DESTINATION
      value: "true"
    {{- end }}
    - name: API_GATEWAY_LOG_SEVERITY_LEVEL
      value: debug
    #- name: API_GATEWAY_LOG_VERBOSITY_LEVEL
    # value: "90"
    - name: JWT_POLICY
      value: {{ .Values.global.jwtPolicy }}
    - name: PILOT_CERT_PROVIDER
      value: {{ .Values.global.pilotCertProvider }}
    - name: OUTPUT_CERTS
      value: /var/lib/istio/data
    - name: SECRET_TTL
      value: 72h
    - name: CA_ADDR
    {{- if .Values.global.caAddress }}
      value: {{ .Values.global.caAddress }}
    {{- else }}
      value: istiod{{- if not (eq .Values.revision "") }}-{{ .Values.revision }}{{-
    {{- end }}
    - name: POD_NAME
      valueFrom:
        fieldRef:
          fieldPath: metadata.name
    - name: POD_NAMESPACE
      valueFrom:
        fieldRef:
          fieldPath: metadata.namespace
    - name: INSTANCE_IP
      valueFrom:

```

3.18.5.2 服务到服务的双向认证(service-to-service mTLS)

功能描述

用于service-to-service身份验证，客户端与服务端服务彼此认证对端身份信息，验证通过后，按照双方配置的安全通信策略进行数据交互。对于传输层身份认证(transport authentication)，Istio 提供mTLS作为全栈解决方案，无需更改服务代码即可启用。mTLS方案提供以下特性：

- 为每个服务提供表示其角色的强标识，以支持跨集群和云的互操作性。
- service-to-service的安全通信。

- 提供一个key管理系统，使key和证书的生成、分发、轮换自动化。

客户端TLS策略

客户端服务，那些发送请求的服务，是有责任的遵循需要的身份认证机制。你可以通过 DestinationRule CR中的ClientTLSSettings去设置与上游服务连接的安全策略。客户端服务根据用户创建的DestinationRule CR策略，给予与上游服务通信的约束，例如明文通信或者密文通信，如果密文通信，验证上游服务身份，来判断其是否可信。

在istio mTLS方案中，客户端TLS策略支持如下模式：

ClientTLSSettings.TLSmode

TLS connection mode

Name	Description
DISABLE	Do not setup a TLS connection to the upstream endpoint.
SIMPLE	Originate a TLS connection to the upstream endpoint.
MUTUAL	Secure connections to the upstream using mutual TLS by presenting client certificates for authentication.
ISTIO_MUTUAL	Secure connections to the upstream using mutual TLS by presenting client certificates for authentication. Compared to Mutual mode, this mode uses certificates generated automatically by Istio for mTLS authentication. When this mode is used, all other fields in ClientTLSSettings should be empty.

istio通过k8s DestinationRule CR配置客户端TLS策略。

njet客户端TLS支持情况如下：

njet客户端TLS配置使用proxy_ssl相关指令进行配置。



njet客户端TLS默认配置为ISTIO_MUTUAL模式。

TLS mode	njet sidecar	描述
DISABLE	支持	明文传输
SIMPLE	非sidecar特性	密文传输
MUTUAL	非sidecar特性	密文传输
ISTIO_MUTUAL	支持	密文传输

详细介绍参照<https://istio.io/v1.13/docs/reference/config/networking/destination-rule/#ClientTLSSettings>

服务端TLS策略

Istio 通过PeerAuthentication CR实现服务端对客户端请求的认证策略配置。

PeerAuthentication 定义流量怎样通往sidecar，支持服务端端口级别的安全认证。在sidecar场景，接受请求的服务端为15006 listener(根据原始目的地port，转发流量，并且给予端口级别的认证策略)，因为istio iptables rule将流量重定向到了15006，所以所有策略会被应用到此 listener。

在istio mTLS方案中，服务端TLS策略支持如下模式：

PeerAuthentication.MutualTLS.Mode

Name	Description
UNSET	Inherit from parent, if has one. Otherwise treated as PERMISSIVE.
DISABLE	Connection is not tunneled.
PERMISSIVE	Connection can be either plaintext or mTLS tunnel.
STRICT	Connection is an mTLS tunnel (TLS with client cert must be presented).

istio通过k8s PeerAuthentication CR配置服务端TLS策略。

njet服务端TLS支持情况如下：

njet服务端TLS配置在stream块中的server块使用 njtmesh_port_mode相关指令进行配置。示例如下所示：

```
server {
    listen 15006 ssl;
    #listen 15006;
    proxy_pass    $njtmesh_dest;
    proxy_bind    127.0.0.6;
    njtmesh_dest on;

    #ssl_certificate      /var/lib/istio/data/cert-chain.pem;
    #ssl_certificate_key  /var/lib/istio/data/key.pem;
    #ssl_client_certificate /var/lib/istio/data/root-cert.pem;
    #ssl_verify_client    on;
    #ssl_verify_depth     10;

    njtmesh_port_mode 15006 DISABLE;
    njtmesh_port_mode 200 PERMISSIVE;
    njtmesh_port_mode 9080 DISABLE;

    ssl_certificate /var/lib/istio/data/cert-chain.pem;
    ssl_certificate_key /var/lib/istio/data/key.pem;
    ssl_verify_client on;
    ssl_client_certificate /var/lib/istio/data/root-cert.pem;
    ssl_verify_depth 10;
```



njet服务端TLS默认配置为PERMISSIVE模式。

TLS mode	njet sidecar	描述
UNSET	支持	继承父作用域的模式，如果最高级别(mesh级别)没有设置，默认使用PERMISSIVE
DISABLE	支持	只接受明文
PERMISSIVE	支持	可以接受密文和明文
STRICT	支持	严格的mTLS，不接受明文

详细介绍参照https://istio.io/v1.13/docs/reference/config/security/peer_authentication/

灰度发布：不同upstream不同安全策略

使用“3.18.4.2 HTTP路由”创建一个基于header的灰度路由“章节的路由规则。我们修改reviews|v2| upstream 为明文传输。

通过下面指令修改：

```
1 kubectl edit dr reviews
```

```
# responses with the relevant features.
#
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1alpha3","kind":"DestinationRule","name":"v3"}}
  creationTimestamp: "2022-11-07T09:44:04Z"
  generation: 18
  name: reviews
  namespace: default
  resourceVersion: "25746101"
  uid: 32a0bcc5-d614-48b0-82f5-f179507331be
spec:
  host: reviews
  subsets:
    - labels:
        version: v1
      name: v1
    - labels:
        version: v2
        name: v2
        trafficPolicy:
          tls:
            mode: DISABLE
      name: v2
    - labels:
        version: v3
      name: v3
```

查看njet配置

reviews|v2| upstream为明文传输

```

location = /internal_location_matches_0_match_0 {
    #log_by_lua_block {
    #balancer.log()
    #}
    set $balancer_ewma_score -1;
    set $proxy_upstream_name "outbound|9080|v2|reviews.default.svc.cluster.local";
    set $proxy_host $proxy_upstream_name;
    set $pass_access_scheme $scheme;

    set $best_http_host $http_host;
    set $pass_server_port $server_port;
    set $pass_port $pass_server_port;

    set $proxy_alternative_upstream_name "";
    set $proxy_active_upstream_name "123";
    set $original_dst_host "original_dst_host";

    proxy_set_header Host $best_http_host;
    proxy_set_header x-njet-original-dst-host $original_dst_host;

    proxy_set_header proxy_active_upstream_name $proxy_active_upstream_name;

    # Allow websocket connections
    #proxy_set_header Upgrade $http_upgrade;
    #proxy_set_header Connection $connection_upgrade;

    proxy_http_version 1.1;

    #configuration.call()
    content_by_lua_block {
        balancer.get_proxy_active_upstream_name()
    }

    header_filter_by_lua_block {
        njet.header.proxy_active_upstream_name = njet.var["proxy_active_upstream_name"]
        local proxy_active_upstream_name = njet.var["proxy_active_upstream_name"]
        njet.log(njet.INFO, "test proxy_active_upstream_name:" , proxy_active_upstream_name)
    }

    if ($flag = "1") {
        proxy_pass http://outbound|9080|v2|reviews.default.svc.cluster.local$request_uri;
    }
    if ($flag = "0") {
        proxy_pass http://upstream-default-backend;
    }

    #set $uri_key $proxy_active_upstream_name;

    #return 200 'njet.var["proxy_active_upstream_name"]';
}

```

reviews|v3| upstream为密文传输


```

location = /internal_location_matches_0_default {
    #log_by_lua_block {
    #balancer.log()
    #}
    set $balancer_ewma_score -1;
    set $proxy_upstream_name "outbound|9080|v3|reviews.default.svc.cluster.local";
    set $proxy_host $proxy_upstream_name;
    set $pass_access_scheme $scheme;

    set $best_http_host $http_host;
    set $pass_server_port $server_port;
    set $pass_port $pass_server_port;

    set $proxy_alternative_upstream_name "";
    set $proxy_active_upstream_name "123";
    set $original_dst_host "original_dst_host";

    proxy_set_header Host $best_http_host;
    proxy_set_header x-njet-original-dst-host $original_dst_host;

    proxy_set_header proxy_active_upstream_name $proxy_active_upstream_name;

    # Allow websocket connections
    #proxy_set_header Upgrade $http_upgrade;
    #proxy_set_header Connection $connection_upgrade;

    proxy_http_version 1.1;

    #configuration.call()
    content_by_lua_block {
        balancer.get_proxy_active_upstream_name()
    }

    header_filter_by_lua_block {
        njet.header.proxy_active_upstream_name = njet.var["proxy_active_upstream_name"]
        local proxy_active_upstream_name = njet.var["proxy_active_upstream_name"]
        njet.log(njet.INFO, "test proxy_active_upstream_name:" , proxy_active_upstream_name)
    }

    if ($flag = "1") {
        proxy_pass https://outbound|9080|v3|reviews.default.svc.cluster.local$request_uri;
    }
    if ($flag = "0") {
        proxy_pass http://upstream-default-backend;
    }

    proxy_ssl_certificate /var/lib/istio/data/cert-chain.pem;
    proxy_ssl_certificate_key /var/lib/istio/data/key.pem;
    proxy_ssl_verify on;
    proxy_ssl_trusted_certificate /var/lib/istio/data/root-cert.pem;
    proxy_ssl_verify_depth 10;
    proxy_ssl_name "spiffe://cluster.local/ns/default/sa/bookinfo-reviews";

    #set $uri_key $proxy_active_upstream_name;

    #return 200 'njet.var["proxy_active_upstream_name"]';
}

```

Njet location path实现如下：

```

map $http_end_user $vs_reviews_default_svc_cluster_local_9080_matches_0_match_0_cond_0 {
    "json" $vs_reviews_default_svc_cluster_local_9080_matches_0_match_0_cond_1;
    default 0;
}

map $request_method $vs_reviews_default_svc_cluster_local_9080_matches_0_match_0_cond_1 {
    "get" 1;
    default 0;
}

map $vs_reviews_default_svc_cluster_local_9080_matches_0_match_0_cond_0 $vs_reviews_default_svc_cluster_local_9080_matches_0 {
    ~^1 /internal_location_matches_0_match_0;
    default /internal_location_matches_0_default;
}

```

```

location = /reviews/0 {
    rewrite ^ $vs_reviews_default_svc_cluster_local_9080_matches_0 last;
}

location ^~ /details {
    rewrite ^ $vs_reviews_default_svc_cluster_local_9080_matches_1 last;
}

location ^~ /productpage {
    rewrite ^ $vs_reviews_default_svc_cluster_local_9080_matches_2 last;
}

location = /ratings/0 {
    rewrite ^ $vs_reviews_default_svc_cluster_local_9080_matches_3 last;
}

```

- 请求中不携带header->end-user: jason，请求被路由到reviews|v3| upstream，即reviews v3版本，v3会返回红色的星星，在productpage pod中的业务容器中通过curl 访问reviews:9080 host，指令如下：

```
1 curl http://reviews:9080/reviews/0
```

返回结果如下图所示，可知，与期望一致。

```

root@productpage-v1-6c594c7b95-qgqk:/opt/microservices# curl http://reviews:9080/reviews/0
{"id": "0", "reviews": [{"reviewer": "Reviewer1", "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!", "rating": {"stars": 5, "color": "red"}, {"reviewer": "Reviewer2", "text": "Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare.", "rating": {"stars": 4, "color": "red"}}]}
root@productpage-v1-6c594c7b95-qgqk:/opt/microservices#

```

在reviews v3 pod中sidecar容器抓包

```

istio-proxy@reviews-v3-6f979bc579-pzltl:/#
istio-proxy@reviews-v3-6f979bc579-pzltl:/# $ sudo tcpdump -i eth0 port 9080 -n -nn -X -w ./reviews-v3.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C36 packets captured
36 packets received by filter
0 packets dropped by kernel
istio-proxy@reviews-v3-6f979bc579-pzltl:/# pwd
/
istio-proxy@reviews-v3-6f979bc579-pzltl:/# ls

```

```

[root@node151 istio]#
[root@node151 istio]# kubectl get pod -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
details-v1-6bf94bc757-2vwk2	2/2	Running	0	9h	10.234.190.197	node153	<none>	<none>
netchecker-agent-6jcbf	1/1	Running	3 (144d ago)	149d	10.234.190.40	node153	<none>	<none>
netchecker-agent-hostnet-skxx	1/1	Running	3 (144d ago)	149d	192.168.40.153	node153	<none>	<none>
netchecker-agent-hostnet-tj69b	1/1	Running	7 (133d ago)	149d	192.168.40.151	node151	<none>	<none>
netchecker-agent-z2pmj	1/1	Running	7 (133d ago)	149d	10.234.96.60	node151	<none>	<none>
productpage-v1-6c594c7b95-qgqk	2/2	Running	0	9h	10.234.190.196	node153	<none>	<none>
ratings-v1-546f59d6f6-gljkp	2/2	Running	0	9h	10.234.96.131	node151	<none>	<none>
reviews-v1-5bc56fbbd8-qxtzp	2/2	Running	0	9h	10.234.190.194	node153	<none>	<none>
reviews-v2-5cc6595bb6-blxv9	2/2	Running	0	9h	10.234.96.132	node151	<none>	<none>
reviews-v2-ratings-bf9569d7-bm6dj	3/3	Running	0	8h	10.234.190.198	node153	<none>	<none>
reviews-v3-6f979bc579-pzltl	2/2	Running	0	9h	10.234.190.195	node153	<none>	<none>

```

[root@node151 istio]#

```

reviews-v3.pcap

文件(F) 编辑(E) 视图(V) 脚本(S) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

应用显示过滤器 ... <Ctrl- />

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.234.190.196	10.234.190.195	TCP	74	49978 → 9080 [SYN] Seq=0 Win=28800 Len=0 MSS=1440 SACK_PERM=1 TSval=3934749982 TSecr=0 WS=128
2	0.000032	10.234.190.195	10.234.190.196	TCP	74	9080 → 49978 [SYN, ACK] Seq=0 Ack=1 Win=28560 Len=0 MSS=1440 SACK_PERM=1 TSval=3934749982 TSecr=3934749982
3	0.000056	10.234.190.196	10.234.190.195	TCP	66	49978 → 9080 [ACK] Seq=1 Ack=1 Win=28800 Len=0 TSval=3934749982 TSecr=3934749982
4	0.000375	10.234.190.196	10.234.190.195	TLSv1.2	264	Client Hello
5	0.000391	10.234.190.195	10.234.190.196	TCP	66	9080 → 49978 [ACK] Seq=1 Ack=199 Win=29696 Len=0 TSval=3934749983 TSecr=3934749983
6	0.002015	10.234.190.195	10.234.190.196	TLSv1.2	2240	Server Hello, Certificate, Server Key Exchange, Certificate Request, Server Hello Done
7	0.002055	10.234.190.196	10.234.190.195	TCP	66	49978 → 9080 [ACK] Seq=199 Ack=2175 Win=33152 Len=0 TSval=3934749984 TSecr=3934749984
8	0.004408	10.234.190.196	10.234.190.195	TLSv1.2	2132	Certificate, Client Key Exchange, Certificate Verify, Change Cipher Spec, Encrypted Handshake Message
9	0.004421	10.234.190.195	10.234.190.196	TCP	66	9080 → 49978 [ACK] Seq=2175 Ack=2265 Win=33792 Len=0 TSval=3934749987 TSecr=3934749987
10	0.005052	10.234.190.195	10.234.190.196	TLSv1.2	1236	New Session Ticket, Change Cipher Spec, Encrypted Handshake Message
11	0.005242	10.234.190.196	10.234.190.195	TLSv1.2	277	Application Data
12	0.020813	10.234.190.195	10.234.96.131	TCP	74	49038 → 9080 [SYN] Seq=0 Win=28800 Len=0 MSS=1440 SACK_PERM=1 TSval=3934750003 TSecr=0 WS=128
13	0.021134	10.234.96.131	10.234.190.195	TCP	74	9080 → 49038 [SYN, ACK] Seq=0 Ack=1 Win=28560 Len=0 MSS=1440 SACK_PERM=1 TSval=2906976249 TSecr=3934750003
14	0.021153	10.234.190.195	10.234.96.131	TCP	66	49038 → 9080 [ACK] Seq=1 Ack=1 Win=28800 Len=0 TSval=3934750004 TSecr=2906976249
15	0.021315	10.234.190.195	10.234.96.131	TLSv1.2	1400	Client Hello
16	0.021435	10.234.96.131	10.234.190.195	TCP	66	9080 → 49038 [ACK] Seq=1 Ack=1335 Win=31488 Len=0 TSval=2906976249 TSecr=3934750004
17	0.021857	10.234.96.131	10.234.190.195	TLSv1.2	207	Server Hello, Change Cipher Spec, Encrypted Handshake Message
18	0.021864	10.234.190.195	10.234.96.131	TCP	66	49038 → 9080 [ACK] Seq=1335 Ack=142 Win=29952 Len=0 TSval=3934750004 TSecr=2906976249
19	0.022000	10.234.190.195	10.234.96.131	TLSv1.2	117	Change Cipher Spec, Encrypted Handshake Message
20	0.022044	10.234.190.195	10.234.96.131	TLSv1.2	333	Application Data
21	0.022350	10.234.96.131	10.234.190.195	TCP	66	9080 → 49038 [ACK] Seq=142 Ack=1653 Win=34176 Len=0 TSval=2906976250 TSecr=3934750004
22	0.023703	10.234.96.131	10.234.190.195	TLSv1.2	289	Application Data
23	0.023856	10.234.190.195	10.234.96.131	TLSv1.2	97	Encrypted Alert
24	0.023909	10.234.190.195	10.234.96.131	TCP	66	49038 → 9080 [FIN, ACK] Seq=1684 Ack=365 Win=30976 Len=0 TSval=3934750006 TSecr=2906976251

由图可知，路由到reviews v3服务的请求发送的是密文。

- 请求中携带header->end-user: jason，请求被路由到reviews|v2| upstream，即reviews v2版本，v2会返回黑色的星星，在productpage pod中的业务容器通过curl 访问reviews:9080 host，指令如下：

```
1 curl http://reviews:9080/reviews/0 -H "end-user:jason"
```

返回结果如下图所示，可知，与期望一致。

```
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices# curl http://reviews:9080/reviews/0 -H "end-user:jason"
{"id": "0", "reviews": [{"reviewer": "Reviewer1", "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!", "rating": {"stars": 5, "color": "black"}}, {"reviewer": "Reviewer2", "text": "Absolutely fun and entertaining. It was a great experience watching this play. The acting was superb and the set design was amazing."}], "rating": {"stars": 4, "color": "black"}}
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
```

在reviews v2 pod中中sidecar容器抓包

```
[root@node151 istio]#
[root@node151 istio]# kubectl get pod -owide
NAME                                READY    STATUS    RESTARTS   AGE    IP                NODE             NOMINATED NODE    READINESS GATES
details-v1-6bf94bc757-2vwk2        2/2     Running   0           9h     10.234.190.197    node153          <none>             <none>
netchecker-agent-6jcbf              1/1     Running   3 (144d ago)  149d   10.234.190.40     node153          <none>             <none>
netchecker-agent-hostnet-skxxx      1/1     Running   3 (144d ago)  149d   192.168.40.153    node153          <none>             <none>
netchecker-agent-hostnet-tj69b      1/1     Running   7 (133d ago)  149d   192.168.40.151    node151          <none>             <none>
netchecker-agent-z2pmj              1/1     Running   7 (133d ago)  149d   10.234.96.60      node151          <none>             <none>
productpage-v1-6c594c7b95-qgq8k    2/2     Running   0           9h     10.234.190.196    node153          <none>             <none>
ratings-v1-546f59d6f6-gljkp        2/2     Running   0           9h     10.234.96.131     node151          <none>             <none>
reviews-v1-5bc56fbbd8-qxtzp        2/2     Running   0           9h     10.234.190.194    node153          <none>             <none>
reviews-v2-5cc6595bb6-blxv9        2/2     Running   0           9h     10.234.96.132     node151          <none>             <none>
reviews-v2-ratings-bf9569d7-bm6dj  3/3     Running   0           8h     10.234.190.198    node153          <none>             <none>
reviews-v3-6f979bc579-pzltl        2/2     Running   0           9h     10.234.190.195    node153          <none>             <none>
[root@node151 istio]#
```

```
istio-proxy@reviews-v2-5cc6595bb6-blxv9:/# sudo tcpdump port 9080 -i eth0 -n -vv -nn -X
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes

11:35:13.623172 IP (tos 0x0, ttl 62, id 38018, offset 0, flags [DF], proto TCP (6), length 60)
10.234.190.190.37818 > 10.234.96.132.9080: Flags [S], cksum 0xf8e (correct), seq 1924169979, win 28800, options [mss 1440,sackOK,TS val 3935310969 ecr 0,nop,wscale 7], length 0
0x0000: 4500 0072 9402 4000 3e06 721d 0aea bec4 E...@>S.....
0x0010: 0aea bec4 93ba 2378 72b0 80fc 0000 0000 ...#Xr.....
0x0020: a002 7080 f88e 0000 0204 05a0 0402 080a ..p.....
0x0030: ea90 1479 0000 0000 0103 0307 ...Y.....
11:35:13.623220 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60)
10.234.96.132.9080 > 10.234.190.190.37818: Flags [S], cksum 0x354b (incorrect -> 0x9ab6), seq 2314577972, ack 1924169980, win 28560, options [mss 1440,sackOK,TS val 2907537215 ecr 3935310969]
0x0000: 4500 0072 9402 4000 3e06 721d 0aea bec4 E...@>S.....
0x0010: 0aea bec4 2378 93ba 89f5 a834 72b0 80fc ...#Xr.....
0x0020: a012 6f90 354b 0000 0204 05a0 0402 080a ..o5K.....
0x0030: ad4d 7f3f ea90 1479 0103 0307 ..M?...y....
11:35:13.623321 IP (tos 0x0, ttl 62, id 38019, offset 0, flags [DF], proto TCP (6), length 52)
10.234.190.190.37818 > 10.234.96.132.9080: Flags [S], cksum 0x381e (correct), seq 1, ack 1, win 225, options [nop,nop,TS val 3935310969 ecr 2907537215], length 0
0x0000: 4500 0034 9482 4000 3a06 7224 0aea bec4 E....@>S.....
0x0010: 0aea 6084 93ba 2378 72b0 80fc 89f5 a835 ...#Xr.....
0x0020: 8010 00e1 381e 0000 0101 080a ea90 1479 ...8.....y
0x0030: ad4d 7f3f ...M.?
11:35:13.623362 IP (tos 0x0, ttl 62, id 38020, offset 0, flags [DF], proto TCP (6), length 251)
10.234.190.190.37818 > 10.234.96.132.9080: Flags [P], cksum 0x04be (correct), seq 1:200, ack 1, win 225, options [nop,nop,TS val 3935310969 ecr 2907537215], length 199
0x0000: 4500 00fb 9484 4000 3a06 7225 0aea bec4 E....@>P.....
0x0010: 0aea 6084 93ba 2378 72b0 80fc 89f5 a835 ...#Xr.....
0x0020: 8018 00e1 04be 0000 0101 080a ea90 1479 ...Y.....
0x0030: ad4d 7f3f 4745 5420 2f72 6576 6965 7773 ..M.7GET./reviews
0x0040: 2f30 2048 5454 502f 312e 310d 0a48 6f73 /0.HTTP/1.1..Hos
0x0050: 743a 2072 6576 6965 7773 3a39 3038 300d t..reviews.9080.
0x0060: 0a78 2d6e 6a65 742d 6f72 6967 696e 616c .X-njet-original
0x0070: 2d64 7374 2d68 6f73 743a 206f 7269 6769 .dst-host:.origi
0x0080: 6e61 6c5f 6473 745f 686f 7374 0d0a 7072 nal_dst.host..pr
0x0090: 6f78 795f 6163 7469 7665 5f75 7073 7472 oxy_active_upstr
0x00a0: 6561 6d5f 6e61 6d65 3a20 3132 330d 0a43 eam_name:.I23..C
0x00b0: 6f6e 6a65 6374 696f 6e3a 2063 6e6f 7365 onnection:close
0x00c0: 0d0a 5573 6572 2d41 6765 6e74 3a20 6375 .User-Agent:cu
0x00d0: 726c 2f3f 2e36 342e 300d 0a41 6363 6570 lV2-S4.O..Access
0x00e0: 743a 202a 2f2a 0d0a 656e 642d 7573 6572 t:/*.*.end-user
0x00f0: 3a20 6a61 736f 6e0d 0a0d 0a ..:json....
11:35:13.623373 IP (tos 0x0, ttl 64, id 55050, offset 0, flags [DF], proto TCP (6), length 52)
10.234.96.132.9080 > 10.234.190.190.37818: Flags [S], cksum 0x3543 (incorrect -> 0x3750), seq 1, ack 200, win 232, options [nop,nop,TS val 2907537215 ecr 3935310969], length 0
0x0000: 4500 0034 d70a 4000 2e0d 0aea 6084 E...4..@.....
0x0010: 0aea bec4 2378 93ba 89f5 a835 72b0 81c3 ...#Xr.....
0x0020: 8010 00e8 3543 0000 0101 080a ad4d 7f3f ...5C.....M.?
0x0030: ea90 1479 ...Y.....
```

明文

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.234.190.196	10.234.96.132	TCP	74	38256 → 9080 [SYN] Seq=0 Win=28800 Len=0 MSS=1440 SACK_PERM=1 TSval=3935412096 TSecr=0 WS=128
2	0.000061	10.234.96.132	10.234.190.196	TCP	74	9080 → 38256 [SYN, ACK] Seq=0 Ack=1 Win=28560 Len=0 MSS=1440 SACK_PERM=1 TSval=2907638342 TSecr=3935412096 WS=128
3	0.000252	10.234.190.196	10.234.96.132	TCP	66	38256 → 9080 [ACK] Seq=1 Ack=1 Win=28800 Len=0 TSval=3935412096 TSecr=2907638342
4	0.000257	10.234.190.196	10.234.96.132	HTTP	265	GET /reviews/0 HTTP/1.1
5	0.000302	10.234.96.132	10.234.190.196	TCP	66	9080 → 38256 [ACK] Seq=1 Ack=200 Win=29696 Len=0 TSval=2907638342 TSecr=3935412096
6	0.021046	10.234.96.132	10.234.96.131	TCP	74	53398 → 9080 [SYN] Seq=0 Win=28800 Len=0 MSS=1440 SACK_PERM=1 TSval=2907638363 TSecr=0 WS=128
7	0.021112	10.234.96.131	10.234.96.132	TCP	74	9080 → 53398 [SYN, ACK] Seq=0 Ack=1 Win=28560 Len=0 MSS=1440 SACK_PERM=1 TSval=2907638363 TSecr=2907638363 WS=128
8	0.021120	10.234.96.132	10.234.96.131	TCP	66	53398 → 9080 [ACK] Seq=1 Ack=1 Win=28800 Len=0 TSval=2907638363 TSecr=2907638363

由图可知，路由到reviews v2服务的请求发送的是明文。

4. 标准配置文件

4.1 njet.conf（数据面）

```
1 helper broker modules/njt_helper_broker_module.so conf/mqtt.conf; # 公共
2 helper ctrl modules/njt_helper_ctrl_module.so conf/ctrl.conf; # 公共 (注意
3
4 load_module modules/njt_http_split_clients_2_module.so; # split_clients_2
5 load_module modules/njt_agent_dynlog_module.so; # accesslog
6 load_module modules/njt_http_location_module.so; # location
7 load_module modules/njt_http_dyn_bwlist_module.so; # 黑白名单
8
9 load_module modules/njt_otel_module.so;
10 load_module modules/njt_agent_dyn_otel_module.so;
11 load_module modules/njt_dyn_ssl_module.so;
12 load_module modules/njt_http_vtsc_module.so;
13
14
```

```

15 cluster_name helper;
16 node_name node1;
17
18
19 worker_processes auto;
20 user root;
21
22 error_log logs/error.log info;
23 pid logs/njet.pid;
24
25 events {
26     worker_connections 1024;
27 }
28
29 http {
30
31     dyn_kv_conf conf/iot-work.conf;
32     include mime.types;
33     include conf.d/*.conf;
34 }

```

4.2 ctrl.conf（控制面）

```

1 load_module modules/njt_http_sendmsg_module.so;
2 load_module modules/njt_ctrl_config_api_module.so;
3 load_module modules/njt_helper_health_check_module.so;
4 load_module modules/njt_http_upstream_api_module.so;
5 load_module modules/njt_http_location_api_module.so;
6 load_module modules/njt_doc_module.so;
7 load_module modules/njt_http_vtsd_module.so;
8
9 cluster_name helper;
10 node_name node1;
11
12 error_log logs/error-ctrl.log info;
13
14 events {
15     worker_connections 1024;
16 }
17
18 http {
19
20     dyn_sendmsg_conf conf/iot-ctrl.conf;

```

```

21     config_req_pool_size 1000;
22     access_log logs/access_ctrl.log combined;
23     include conf/mime.types;
24
25     server {
26         listen      8443;
27
28         location /kv {
29             dyn_sendmsg_kv;
30         }
31
32         location /config {
33             config_api;
34         }
35
36         location /hc {
37             health_check_api;
38         }
39
40         location /api {
41             api write=on;
42         }
43
44         location /doc {
45             doc_api;
46         }
47         location /dyn_loc {
48             dyn_location_api;
49         }
50
51         location /metrics {
52             vhost_traffic_status_display;
53             vhost_traffic_status_display_format html;
54         }
55     }
56
57 }

```

4.3 mime.types

```

1
2 types {
3     text/html                                html htm shtml;

```

4	text/css	css;
5	text/xml	xml;
6	image/gif	gif;
7	image/jpeg	jpeg jpg;
8	application/javascript	js;
9	application/atom+xml	atom;
10	application/rss+xml	rss;
11		
12	text/mathml	mml;
13	text/plain	txt;
14	text/vnd.sun.j2me.app-descriptor	jad;
15	text/vnd.wap.wml	wml;
16	text/x-component	htc;
17		
18	image/avif	avif;
19	image/png	png;
20	image/svg+xml	svg svgz;
21	image/tiff	tif tiff;
22	image/vnd.wap.wbmp	wbmp;
23	image/webp	webp;
24	image/x-icon	ico;
25	image/x-jng	jng;
26	image/x-ms-bmp	bmp;
27		
28	font/woff	woff;
29	font/woff2	woff2;
30		
31	application/java-archive	jar war ear;
32	application/json	json;
33	application/mac-binhex40	hqx;
34	application/msword	doc;
35	application/pdf	pdf;
36	application/postscript	ps eps ai;
37	application/rtf	rtf;
38	application/vnd.apple.mpegurl	m3u8;
39	application/vnd.google-earth.kml+xml	kml;
40	application/vnd.google-earth.kmz	kmz;
41	application/vnd.ms-excel	xls;
42	application/vnd.ms-fontobject	eot;
43	application/vnd.ms-powerpoint	ppt;
44	application/vnd.oasis.opendocument.graphics	odg;
45	application/vnd.oasis.opendocument.presentation	odp;
46	application/vnd.oasis.opendocument.spreadsheet	ods;
47	application/vnd.oasis.opendocument.text	odt;
48	application/vnd.openxmlformats-officedocument.presentationml.presentation	pptx;
49		
50	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	

51		xlsx;
52	application/vnd.openxmlformats-officedocument.wordprocessingml.documer	
53		docx;
54	application/vnd.wap.wmlc	wmlc;
55	application/wasm	wasm;
56	application/x-7z-compressed	7z;
57	application/x-cocoa	cco;
58	application/x-java-archive-diff	jardiff;
59	application/x-java-jnlp-file	jnlp;
60	application/x-makeself	run;
61	application/x-perl	pl pm;
62	application/x-pilot	prc pdb;
63	application/x-rar-compressed	rar;
64	application/x-redhat-package-manager	rpm;
65	application/x-sea	sea;
66	application/x-shockwave-flash	swf;
67	application/x-stuffit	sit;
68	application/x-tcl	tcl tk;
69	application/x-x509-ca-cert	der pem crt;
70	application/x-xpinstall	xpi;
71	application/xhtml+xml	xhtml;
72	application/xspf+xml	xspf;
73	application/zip	zip;
74		
75	application/octet-stream	bin exe dll;
76	application/octet-stream	deb;
77	application/octet-stream	dmg;
78	application/octet-stream	iso img;
79	application/octet-stream	msi msp msm;
80		
81	audio/midi	mid midi kar;
82	audio/mpeg	mp3;
83	audio/ogg	ogg;
84	audio/x-m4a	m4a;
85	audio/x-realaudio	ra;
86		
87	video/3gpp	3gpp 3gp;
88	video/mp2t	ts;
89	video/mp4	mp4;
90	video/mpeg	mpeg mpg;
91	video/quicktime	mov;
92	video/webm	webm;
93	video/x-flv	flv;
94	video/x-m4v	m4v;
95	video/x-mng	mng;
96	video/x-ms-asf	asx asf;
97	video/x-ms-wmv	wmv;


```
98     video/x-msvideo                                avi;
99 }
100
```

4.4 iot-ctrl.conf

配置为空即可

```
1
```

4.5 iot-work.conf

配置为空即可

```
1
```

4.6 mqtt.conf

配置为空即可

```
1
```

4.7 otel.conf

```
1 exporter = "otlp"
2 processor = "batch"
3
4 [exporters.otlp]
5 #collector server address
6 # Alternatively the OTEL_EXPORTER_OTLP_ENDPOINT environment variable can also
7 host = "192.168.40.136"
8 port = 4317
9
10 [processors.batch]
11 max_queue_size = 2048
12 schedule_delay_millis = 5000
```

```
13 max_export_batch_size = 512
14
15 [service]
16 # Can also be set by the OTEL_SERVICE_NAME environment variable.
17 name = "njet-proxy" # Opentelemetry resource name
18
19 [sampler]
20 name = "AlwaysOn" # Also: AlwaysOff, TraceIdRatioBased
21 ratio = 0.1
22 parent_based = false
```

4.8 重要：部署所需额外配置

提测单里除了有流水线出的包 + 还要有如下三个文件



1) 将njet_otel_module.so、njet_agent_dyn_otel_module.so 放到与其他动态模块同目录，即放置在modules目录下。

2) dependlib.tar 解压后，启动njet时将该dependlib目录加入到LD_LIBRARY_PATH 环境变量里

eg: export LD_LIBRARY_PATH=`pwd`/dependlib:\$LD_LIBRARY_PATH

vi /etc/profile

source /etc/profile



dependlib.tar

16.00MB



njet_otel_module.so

2.95MB



njet_agent_dyn_otel_module.so

24.23KB



5. 常见问题

a. reload后，需要等待3秒左右，动态查询接口，才可用。

```

[root@master110 conf.d]# ../../sbin/njet -p /home/njet_1.2 -c conf/njet.conf -s reload
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
0    0    0    0    0    0    0  0 0 0 0 0 0 0 0 0 0 0 0 0 0
curl: (7) Failed to connect to 127.0.0.1 port 8081 after 0 ms: Couldn't connect to server
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
0    0    0    0    0    0    0  0 0 0 0 0 0 0 0 0 0 0 0 0 0
curl: (7) Failed to connect to 127.0.0.1 port 8081 after 0 ms: Couldn't connect to server
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
0    0    0    0    0    0    0  0 0 0 0 0 0 0 0 0 0 0 0 0 0
curl: (7) Failed to connect to 127.0.0.1 port 8081 after 0 ms: Couldn't connect to server
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100 1052 100 1052    0    0 15762    0 0 0 0 0 0 0 0 0 0 0 0 0 0
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:80"
      ],
      "serverNames": [
        "www.crm.com"
      ],
      "locations": [
        {
          "location": "/cust",
          "accessLogOn": true,
          "accessLogs": [
            {
              "path": "logs/accesslog_2345.conf",
              "formatName": "combined"
            }
          ]
        }
      ],
    },
    {
      "location": "/order"
    }
  ]
}

```

b. 访问不存在的接口，会报500的错误。

```

[root@master110 conf.d]# ../../sbin/njet -p /home/njet_1.2 -c conf/njet.conf -s reload
[root@master110 conf.d]# ../../sbin/njet -p /home/njet_1.2 -c conf/njet.conf -s reload
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/split_clients_2 |jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100    41    100    41    0    0    20      0  0:00:02  0:00:02 --:--:--   20
{
  "code": 500,
  "msg": "rpc timeout"
}
[root@master110 conf.d]# cd ../logs
-bash: cd: ../logs: 没有那个文件或目录
[root@master110 conf.d]# cd ../../logs
[root@master110 logs]# ll
总用量 164
-rw-r--r--. 1 root root 5243 3月 29 13:45 access_ctrl.log
-rw-r--r--. 1 root root 0 3月 29 10:39 access.log
-rw-r--r--. 1 root root 0 3月 29 10:39 accesslog_1.conf
-rw-r--r--. 1 root root 0 3月 29 10:50 accesslog_2345.conf
-rw-r--r--. 1 root root 0 3月 29 10:39 accesslog_2.conf
-rw-r--r--. 1 root root 20779 3月 29 13:45 error_ctrl.log
-rw-r--r--. 1 root root 74459 3月 29 13:45 error.log
-rw-r--r--. 1 root root 0 3月 29 10:39 mosquito.log
-rw-r--r--. 1 root root 0 3月 29 11:14 new.log
-rw-r--r--. 1 root root 6 3月 29 11:00 njet.pid
-rw-rw-rw-. 1 root root 0 3月 29 11:00 sendmsg_1
-rw-rw-rw-. 1 root root 0 3月 29 10:39 sendmsg_2
-rw-rw-rw-. 1 root root 0 3月 29 10:39 work_iot_0
[root@master110 logs]# tail error.log
2023/03/29 13:45:18 [notice] 23757#0: start copilot process 4075
2023/03/29 13:45:18 [notice] 23757#0: start copilot process 4076
2023/03/29 13:45:18 [notice] 23757#0: signal 29 (SIGIO) received
2023/03/29 13:45:18 [error] 4070#0: Event fired!,try connect again, 000060820010EE40
2023/03/29 13:45:18 [notice] 4070#0: connect to broker failed:4
2023/03/29 13:45:18 [notice] 4076#0: helper ctrl started
2023/03/29 13:45:18 [notice] 4076#0: ctrl reconfiguring
2023/03/29 13:45:19 [error] 4070#0: Event fired!,try connect again, 000060820010EE40
2023/03/29 13:45:19 [notice] 4070#0: connect ok, register io
2023/03/29 13:45:19 [notice] 4070#0: kv module connect ok, register socket:5
[root@master110 logs]# curl 127.0.0.1:8081/config/1/config/split_clients_2 |jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100    41    100    41    0    0    20      0  0:00:02  0:00:02 --:--:--   20
{
  "code": 500,
  "msg": "rpc timeout"
}
[root@master110 logs]#

```

c. ctrl.conf中自定义了接口location，curl命令可以正常使用，但是doc gui无法使用。

njt_ctrl标准配置文件：需要包含如下location：

- /api 提供upstream member维护
- /config 提供声明式api
- /kv 提供kv维护接口
- /dyn_loc 提供动态location配置接口
- /hc 主动健康检查接口
- /metrics 指标输出接口
- /doc/swagger openapi接口
- /doc/gui 简单界面维护

6. 附录

健康检查API说明

openapi: 3.0.3

info:

title: njet-api

description: njet-api

version: 1.0.0

servers:

- url: '<http://192.168.40.140/api/1>'

paths:

/hc/:

get:

tags:

- Health Check

summary: Returns a list of health check

description: Returns a list of health check.

operationId: getHealthCheckUpstreams

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/HealthCheck'

'500':

description: 服务器内部错误

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/hc/{typeName}/{upstreamName}:

parameters:

- name: typeName

type: string

in: path

description: 健康检查的类型.

required: true

example: HTTP

enum:

- HTTP

- name: upstreamName

type: string
in: path
description: upstream 名称.
required: true
example: demo

get:

tags:

- Health Check

summary: 查询健康检查详细配置

description: 查询健康检查详细配置

operationId: getHealthCheckConf

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/HealthCheckConf'

'404':

description: upstream 或者 type 未找到

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

'500':

description: 服务器内部错误

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

post:

tags:

- Health Check

summary: 创建健康检查

description: 创建健康检查

operationId: postHealthCheck

parameters:

- in: body

name: postHealthCheck

description: 创建新的健康检查

required: true

schema:

\$ref: '#/components/schemas/HealthCheckConf'

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

'400':

description: 请求body 错误

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

'404':

description: 健康检查类型或upstream 未找到

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

'409':

description: 健康检查已经定义过

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

'500':

description: 服务器内部错误

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

delete:

tags:

- Health Check

summary: 删除健康检查

description: 删除健康检查

operationId: deleteHealthCheck

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

'404':

description: 健康检查不存在

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

###DEFINITIONS

components:

schemas:

HealthCheckItem:

type: object

properties:

upstream:

type: string

description: name of upstream.

type:

type: string

description: type of health check.

HealthCheck:

title: healthCheck

description: health Check 简易信息

type: array

items:

\$ref: '#/components/schemas/HealthCheckItem'

CommonMsg:

title: commonMsg

description: 公共提示信息

type: object

properties:

code:
 type: string
msg:
 type: string

HealthCheckConf:

title: healthCheckConf
description: health Check 完整配置
type: object
properties:
 interval:
 type: string
 description: 健康检查间隔
 default: 5s
 example: 5s
 jitter:
 type: string
 description: 健康检查间隔
 default: 0s
 example: 1s
 timeout:
 type: string
 description: 健康检查超时时间
 default: 5s
 example: 5s
 port:
 type: number
 description: 健康检查的端口
 example: 80
 minimum: 1
 maximum: 65535
 passes:
 type: number
 description: 健康检查转变为健康，需要检查连续通过的次数
 minimum: 1
 default: 1
 fails:
 type: number
 description: 健康检查转变为不健康，需要健康检查连续不通过的次数
 minimum: 1

default: 1

http:

type: object

description: HTTP upstream 健康检查相关配置

properties:

grpcService:

type: string

description: grpc健康检查请求的 service 名称

default: demo

grpcStatus:

type: number

default: 13

description: grpc健康检查标记为通过的状态码

uri:

type: string

description: http 健康检查请求的uri

default: /

header:

type: array

description: http 健康检查校验的header

items:

type: string

description:

body:

type: string

description: http 健康检查检查body的内容是否满足表达式

status:

type: string

description: http 健康检查检查返回状态码是否满足

ssl:

type: object

description: ssl客户端相关配置

properties:

enable:

type: boolean

description: 是否开启https 健康检查

default: false

protocols:

type: string

description: 指定健康检查使用的ssl协议版本，支持的版本：[SSLv2] [SSLv3] [TLSv1] [TLSv1.1] [TLSv1.2] [TLSv1.3]
default: TLSv1 TLSv1.1 TLSv1.2
ciphers:
type: string
description: 对健康检查请求开启 ciphers，ciphers以openssl 标准格式配置。"openssl ciphers" 指令可以查看ciphers列表
default: DEFAULT
name:
type: string
description: 设置健康检查请求时携带的Host header
default: upstream name
serverName:
type: boolean
description: 启用或禁用通过TLS服务器名称指示扩展（SNI，RFC 6066）传递服务器名称。
default: false
verify:
type: boolean
description: 启用或禁用对连接的HTTPS服务器证书的验证。（需配置trustedCertificate）
default: false
verifyDepth:
type: number
description: HTTPS服务器证书链中设置验证深度。
default: 1
minimum: 1
trustedCertificate:
type: string
description: 指定一个具有PEM格式的受信任CA证书，用于验证指定HTTPS服务器的证书。
crl:
type: string
description: 以PEM格式指定具有撤销证书（CRL）的文件，用于验证代理HTTPS服务器的证书。
certificate:
type: string
description: 指定一个带有PEM格式证书的文件，用于对HTTPS服务器进行身份验证(客户端证书)。
certificateKey:
type: string

description: 指定一个带有PEM格式的密钥的文件，用于对HTTPS服务器进行身份验证(客户端证书key)。

6.1 动态access log配置API说明

openapi: 3.0.3

info:

title: dyn config api

description: 动态config 开关

version: 1.0.0

servers:

- url: '/config/1'

paths:

/config/:

get:

tags:

- dyn config module

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/ModuleList'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/config/http_log:

put:

tags:

- dyn log config

summary: set module conf

description: set module conf

operationId: setDynModuleConf

parameters:

- in: body

name: access log conf

description: access log 配置清单

required: true

schema:

\$ref: '#/components/schemas/MainConf'

responses:

'204':

description: Success

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

get:

tags:

- dyn log config

summary: return dynamic module conf

description: get module conf

operationId: getDynModuleConf

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/MainConf'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

###DEFINITIONS

components:

schemas:

CommonMsg:

title: commonMsg

description: 公共提示信息

type: object

properties:

code:

type: string

description: 状态码

msg:

type: string

description: 提示信息

ModuleList:

title: module list

description: 支持动态配置的模块列表

type: array

items:

type: string

MainConf:

title: main Conf

description: 动态access log main 级别配置

type: object

properties:

servers:

type: array

items:

\$ref: '#/components/schemas/ServerConf'

accessLogFormats:

type: array

items:

\$ref: '#/components/schemas/FormatConf'

FormatConf:

title: FormatConf

description: 动态access format 配置

type: object
properties:
 name:
 type: string
 description: access log format 名称
 example: test
 format:
 type: string
 description: access log 格式字符串
 example: \$remote_addr - \$remote_user [\$time_local] \"\$request\" \$status
\$body_bytes_sent \"\$http_referer\" \"\$http_user_agent\"
 escape:
 type: string
 description: access log 转义格式
 example: default

ServerConf:

title: ServerConf
description: 动态access log server 级别配置
type: object
properties:
 listens:
 type: array
 description: server listen 端口列表
 items:
 type: string
 serverNames:
 type: array
 description: server Name 列表.
 items:
 type: string
 locations:
 \$ref: '#/components/schemas/LocationLogConf'

AccessLogConf:

title: AccessLogConf
description: access log 配置
type: object
properties:
 path:

type: string
description: access log 写入的位置
formatName:
type: string
description: access log 格式名称

LocationLogConf:

title: LocationLogConf
description: location 级别 access log 配置开关信息
type: object
properties:
 location:
 type: string
 description: location 名称 .
 example: /demo
 accessLogOn:
 type: boolean
 description: access log 开关状态 .
 example: false
 accessLogs:
 type: array
 items:
 \$ref: '#/components/schemas/AccessLogConf'
 locations:
 \$ref: '#/components/schemas/LocationLogConf'

6.2 动态telemetry配置API说明

openapi: 3.0.3

info:

title: dyn config api
description: 动态config 开关
version: 1.0.0

servers:

- url: '/config/1'

paths:

/config/:

get:

tags:

- dyn config module

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/ModuleList'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/config/otel:

put:

tags:

- dyn telemetry config

summary: set module conf

description: set module conf

operationId: setDynModuleConf

parameters:

- in: body

name: telemetry conf

description: telemetry 配置清单

required: true

schema:

\$ref: '#/components/schemas/MainConf'

responses:

'204':

description: Success

'500':

description: server error

content:

application/json:

```
    schema:
      $ref: '#/components/schemas/CommonMsg'
get:
  tags:
    - dyn telemetry config
  summary: return dynamic module conf
  description: get module conf
  operationId: getDynModuleConf
  responses:
    '200':
      description: Success
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/MainConf'
    '500':
      description: server error
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CommonMsg'
```

###DEFINITIONS

```
components:
  schemas:
    CommonMsg:
      title: commonMsg
      description: 公共提示信息
      type: object
      properties:
        code:
          type: string
          description: 状态码
        msg:
          type: string
          description: 提示信息
```

ModuleList:

title: module list

description: 支持动态配置的模块列表

type: array

items:

type: string

MainConf:

title: main Conf

description: 动态telemetry main 级别配置

type: object

properties:

servers:

type: array

items:

\$ref: '#/components/schemas/ServerConf'

ServerConf:

title: ServerConf

description: 动态telemetry server 级别配置

type: object

properties:

listens:

type: array

description: server listen 端口列表

items:

type: string

serverNames:

type: array

description: server Name 列表.

items:

type: string

locations:

\$ref: '#/components/schemas/LocationLogConf'

LocationLogConf:

title: LocationLogConf

description: location 级别 telemetry 配置开关信息

type: object
properties:
 location:
 type: string
 description: location 名称 .
 example: /demo
 opentelemetry:
 type: boolean
 description: telemetry 开关状态 .
 example: false
 locations:
 \$ref: '#/components/schemas/LocationLogConf'

6.3 动态vts配置API说明

openapi: 3.0.3

info:

 title: dyn config api

 description: 动态config 开关

 version: 1.0.0

servers:

 - url: '/config/1'

paths:

 /config/:

 get:

 tags:

 - dyn config module

 summary: Returns a list of allow dynamic conf module

 description: Returns a list of allow dynamic conf module

 operationId: getDynConfModuleList

 responses:

 '200':

 description: Success

 content:

 application/json:

 schema:

 \$ref: '#/components/schemas/ModuleList'

 '500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/config/http_vts:

put:

tags:

- dyn vts config

summary: set module conf

description: set module conf

operationId: setDynModuleConf

parameters:

- in: body

name: vts conf

description: vts 配置清单

required: true

schema:

\$ref: '#/components/schemas/MainConf'

responses:

'204':

description: Success

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

get:

tags:

- dyn vts config

summary: return dynamic module conf

description: get module conf

operationId: getDynModuleConf

responses:

'200':

description: Success

content:

```
  application/json:
    schema:
      $ref: '#/components/schemas/MainConf'
'500':
  description: server error
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CommonMsg'
```

###DEFINITIONS

```
components:
  schemas:
    CommonMsg:
      title: commonMsg
      description: 公共提示信息
      type: object
      properties:
        code:
          type: string
          description: 状态码
        msg:
          type: string
          description: 提示信息
```

```
ModuleList:
  title: module list
  description: 支持动态配置的模块列表
  type: array
  items:
    type: string
```

```
MainConf:
  title: main Conf
  description: 动态vts main 级别配置
  type: object
  properties:
```

vhost_traffic_status_filter_by_set_key:
type: string
description: vts filter set .
example: \"\$request_uri\" \"\$server_name\"
servers:
type: array
items:
\$ref: '#/components/schemas/ServerConf'

ServerConf:
title: ServerConf
description: 动态vts server 级别配置
type: object
properties:
listens:
type: array
description: server listen 端口列表
items:
type: string

serverNames:
type: array
description: server Name 列表.
items:
type: string
locations:
\$ref: '#/components/schemas/LocationConf'

LocationConf:
title: LocationConf
description: location 级别 vts 配置开关信息
type: object
properties:
location:
type: string
description: location 名称 .
example: /demo
vhost_traffic_status:
type: boolean

description: vts 开关状态 .
example: false
locations:
\$ref: '#/components/schemas/LocationConf'

6.4 动态split_clients2配置API说明

openapi: 3.0.3

info:

title: dyn config api
description: 动态config 开关
version: 1.0.0

servers:

- url: '/config/1'

paths:

/config/:

get:

tags:

- dyn config module

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/ModuleList'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/config/http_split_clients_2:

put:

tags:

- dyn split clients 2 config

summary: set module conf

description: set module conf

operationId: setDynModuleConf

requestBody:

content:

application/json:

schema:

\$ref: '#/components/schemas/SplitConf'

required: true

responses:

'204':

description: Success

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

get:

tags:

- dyn split clients 2 config

summary: return dynamic module conf

description: get module conf

operationId: getDynModuleConf

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/SplitConf'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

components:

schemas:

CommonMsg:

title: commonMsg

description: 公共提示信息

type: object

properties:

code:

type: string

description: 状态码

msg:

type: string

description: 提示信息

ModuleList:

title: module list

description: 支持动态配置的模块列表

type: array

items:

type: string

SplitConf:

title: Split Clients 2 Conf

description: 动态Split Clients 2 配置

type: object

properties:

http:

type: object

properties:

split_clients_2:

type: object

properties:

\${backend_name1}:

type: integer

example: 20

minimum: 0

maximum: 100

\${backend_name2}:

type: integer

example: 80

minimum: 0

maximum: 100

6.5 动态黑白名单配置API说明

openapi: 3.0.3

info:

title: dyn config api

description: 动态config 开关

version: 1.0.0

servers:

- url: '/config/1'

paths:

/config/:

get:

tags:

- dyn config module

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/ModuleList'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/config/http_dyn_bwlist:

put:

tags:

- dyn black white list config

summary: set module conf

description: set module conf

operationId: setDynModuleConf

requestBody:
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/MainConf'
 required: true

responses:
 '204':
 description: Success
 '500':
 description: server error
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/CommonMsg'

get:
 tags:
 - dyn black white list config
 summary: return dynamic module conf
 description: get module conf
 operationId: getDynModuleConf
 responses:
 '200':
 description: Success
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/MainConf'
 '500':
 description: server error
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/CommonMsg'

components:
 schemas:
 CommonMsg:
 title: commonMsg
 description: 公共提示信息
 type: object

properties:

code:

type: string

description: 状态码

msg:

type: string

description: 提示信息

ModuleList:

title: module list

description: 支持动态配置的模块列表

type: array

items:

type: string

MainConf:

title: main Conf

description: 动态 main 级别配置

type: object

properties:

servers:

type: array

items:

\$ref: '#/components/schemas/ServerConf'

ServerConf:

title: ServerConf

description: 动态 server 级别配置

type: object

properties:

listens:

type: array

description: server listen 端口列表

items:

type: string

serverNames:

type: array

description: server Name 列表.

items:

type: string
locations:
\$ref: '#/components/schemas/LocationConf'

LocationConf:

title: LocationConf
description: location 级别 vts 配置开关信息

type: object

properties:

location:

type: string

description: location 名称 .

example: /demo

accessIpv4:

type: object

description: access 黑白名单

properties:

rule:

type: string

enum: [allow, deny]

addr:

type: string

example: "192.168.1.0"

mask:

type: string

example: "255.255.255.0"

locations:

\$ref: '#/components/schemas/LocationConf'

6.6 动态http server ssl配置API说明

openapi: 3.0.3

info:

title: dyn server ssl api

description: 动态server ssl配置

version: 1.0.0

servers:

- url: '/config/1'

paths:

/config/:

get:

tags:

- dyn config module

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/ModuleList'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/config/http_ssl:

put:

tags:

- dyn http server ssl config

summary: set http server ssl conf

description: set http server ssl conf

operationId: setDynHttpServerSsl

parameters:

- in: body

name: http server ssl conf

description: http server ssl conf 单条配置

required: true

schema:

\$ref: '#/components/schemas/MainConf'

responses:

'204':

description: Success

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

get:

tags:

- dyn http server ssl config

summary: return http server ssl config

description: get http server ssl config

operationId: getDynHttpServerSsl

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/MainConf'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

###DEFINITIONS

components:

schemas:

CommonMsg:

title: commonMsg

description: 公共提示信息

type: object

properties:

code:

type: string

description: 状态码

msg:

type: string
description: 提示信息

ModuleList:

title: module list
description: 支持动态配置的模块列表
type: array
items:
 type: string

MainConf:

title: main Conf
description: 动态http server ssl main 级别配置
type: object
properties:
 servers:
 type: array
 items:
 \$ref: '#/components/schemas/ServerConf'

ServerConf:

title: ServerConf
description: 动态 http server ssl server 级别配置
type: object
properties:
 listens:
 type: array
 description: server listen 端口列表
 items:
 type: string
 serverNames:
 type: array
 description: server Name 列表.
 items:
 type: string
 certificates:
 type: array
 items:
 \$ref: '#/components/schemas/CertGroup'

CertGroup:

title: CertGroupConf
description: 证书组配置信息
type: object
properties:

certificate:

type: string

description: 证书文件内容base64(注意需要配置'data:')

example: "data:-----BEGIN CERTIFICATE-----

```
\r\nMIIETCCAwmgAwIBAgIUUK6l+HK4KR\hLosx2XqNTaHLUkO0wDQYJKoZIhvcNAQEL\r\nBQAwZTELMAkGA1UEBhMCQ04xEDAOBgNVBAGTB0JlaUppbmcmcxEDAOBgNVBACTB0Jl\r\nnaUppbmcmcxDjAMBgNVBAoTBW5naW54MQ8wDQYDVQQLEwZ0bWxha2UxETAPBgNVBAMT\r\n\r\nCG5naW54LWNhMCAXDTIyMDgwOTA5NDEwMFOYDzIxMjIwNzE2MDk0MTAwWjBiMQsw\r\n\r\nCQYDVQQGEwJDTjEQMA4GA1UECBMHQmVpSmluZzEQMA4GA1UEBxMHQmVpSmluZzEO\r\n\r\nMAwGA1UEChMFbmdpbngxDzANBgNVBASTBnRtbGFrZTEOMAwGA1UEAxMFbmdpbngw\r\n\r\nnggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQC0hs5E8ZZT+ZU\DVvIRrSL\r\n\r\nnokk6OE5EA4URo6zawqguKRITc1USgfMdZFwR2znCxjFcsk7W\FSWqA5k5zyePYL8\r\n\r\n\r\n0lhn3i\GQGczfa9dMZZEtN9EMD6TPOiDI\CaWAFc3U7qE+SYckUHLu\lMX8LshR\r\n\r\nndNkGecq41Ck5Sftm3xwy4cmpcClyMpNumHrjJyxkp3S\QGewgZRQypntG9R34EKj\r\n\r\nnQNFovsn39TeeG+DBjS5m1+roth0JJ6Dbe mB47hy\uaEwAQpobN1etx20wCwxRO7\r\n\r\nn7Huuh4kmWXjUol0nvzXM3ZZX2QnHn4jlg\ALS7O6p86z+m4vEHK4hRfWf52xGjMv\r\n\r\nnAgMBAAGjgckwgcYwDgYDVR0PAQH\BAQDAgWgMB0GA1UdJQQWMBQGCCsGAQUFBwMB\r\n\r\nBgggBgEFBQcDAjAMBgNVHRMBAf8EAjAAMB0GA1UdDgQWBBQz6z5AIFRrP+KD1gBt\r\n\r\nnW6j5NEMzwDAfBgNVHSMEGDAWgBQ64+C++PsR6pDZoQ7EO RozSJ8d9DBHBgNVHREE\r\n\r\nnQDA+ggkxOTluMTY4LiqCDiouY2x1c3RlcjEuY29tgg0qLmNsdXN0Z XluY29tggwq\r\n\r\nnLnRtbGFrZS5jb22HBH8AAAEwDQYJKoZIhvcNAQELBQADggEBADouKA9fFfInAs2u\r\n\r\nnu7yj4cGNG+sC8r1wDWyGiXGDzvaB7C\sPTZiCgh8hl5gxzOSC+Se51kNFYymN9g4\r\n\r\nnBPfB\SZ+CxO60BOt5eFF4PxIW7+d8gGFJr7Yx55QGA9DrvGi\LEiWjV3bQaezjN\r\n\r\nnoXUkfFgyKv79AfNFShkUPIHCKlInfyqcCF+GEKxNC1q4wyCIUt2EGtwfiO4YR85s0\r\n\r\nn7jQ2BKJOSw3LLd8pZSvOXYu\I9DoO5w3gM5TgjnJlQPq+bmgqJ7bfWbSpFo70cdF\r\n\r\nnFLfE9MLpwhDjVQEJ6DOmZqU1dVdNtwLrUrm0\8xkkmac0\8BaEYyPy1SNMGJTMGE\r\n\r\nnW8psxto=\r\n\r\n-----END CERTIFICATE-----\r\n"
```

certificateKey:

type: string

description: 证书key文件内容base64(注意需要配置'data:')

example: "data:-----BEGIN RSA PRIVATE KEY-----

```
\r\nMIIEogIBAAKCAQEAtIbORPGWU\mVPw77yEa0i6JJOjhORAOFEaOs2sKoLikZU3NV\r\n\r\nnEoHzHWRcEds5wsYxXLJO1vxUlqgOZOc8nj2C\P9JYZ94vxkBnM32vXTGWRLTfRDA\r\n\r\nn+kzzogyPwmlgBXN1O6hPkmHJFBy7v5TF\C7IUXTZBnnKuNQpOU n7Zt8cMuHJqXAI\r\n\r\nnMjKtBph64ycsZKd0v0BnsIGUUMqZ7RvUd+BCo0DRZ6L7J9\U3nhvgwY0uZtfq6LY\r\n\r\nndCseg23pgeO4cv7mhMAEKaGzdXrcdtMAsMUTu+x7roeJll41KJdJ781zN2WV9kJ\r\n\r\nnx5+lylPwC0uzuqfOs\puLxByulUX1n+dsRozLwIDAQABAoIBAFh+VJLbUnOrvwuA\r\n\r\nnTtBoSlzCat8NRuB0UUDKWSuLjGHEZ9POj39Z
```

EFHyJmfibTgba4sjJR6h5t1LWHMC\r\nH2b6hEF86v3d7JTQr0esdy18FtcHMYD3O4H3Qt7HBZm
pihZh+K\b29XH9YfUZfyN\r\n81ehnzS+8LwJ6+QarHKW35QX\|ny5+pCw+G97cjdE7sClZnryd1
KseWubAXDLSgmQ\r\nJcjXMMkwjtd2x\|w23ZT97WMEcToL1wMw4hF\|tHtu9u3UHMhzLHgrfr
kW4QmACbLF\r\n8MJR94wzscJrUEviz5fV+3ZTmK+lujGmO\|KkOpNAPGf8QAeBfa\|qyBTQCQg
CdxRo\r\n8hrDT4ECgYEA5vLsDDWiAROvvPaRkCv0y0UWkdgyBxv2T6pjDJZzMxb0yKZ48E6l\r\nh0PI9bfn0KbI1ZZviiZg20oZ\|Keo2wLYa+W00rZhWljWUh0lvd\|5vTPHBmpsXl6+\r\nrRK49aNVkf
qlleherhfrYUkHKKDeg4lKVY3fwAlAQeg\|y4Ds\|8k3HUuECgYEAyBu8\r\nns7KuO7sAaJVj+QFfRr
R6u2eJD48HPB4UoeDYMaKAuPTiM5\|AUjH\|YdD4E2lGAYJJ\r\nnJVN413yu3Dk9UIAivgqCROep
AcnnZ1BwE2ql\|gfr2GD95o8JQSphze0pC\|6WS\|jG\r\nniBR+j7MGWIkHJHEIUjOrf8eqtVLAywjx
z9rPWA8CgYBTkuzgrjfl893Qn9mlNoLr\r\nnXCECvh28fN3xjIMxpvAhONl0Euol7CzyehEq+lylJ3Xd
9QaAE8tRD8u\|plxwhOMU\r\nniJea+OzZ6PQF2wPi0j5pvWb0Z2a378kiyXrniPFI9LwlJrCnV1MY0
T36t8a8n+33\r\nnhNuRuq97vHHDuy003fiXgQKBgGKzM6cauc+iU\|hBvzbBk4HnYSXwUm1HKd
VgLOMP\r\nnaPNKaN1RhATchdrE6GcR0FqasTq4fYWYn2ECEalz3idHnFtKCaj87qKAOM\|\|n9gj\r
\n0wAhXhWy+WjwlitvQSB2Gqnc37sHML1MBoTQU4\|1vn0d93G8JJn5HN0kvQ0oE0Vn\r\nncp\|
HAoGAGCxpM24rHBK0HmlABZd8ZfLW\|sW4Y7bMsQmrOVuPZcTWcO5HzkkiGzTc\r\nn8DCDDe
GdfJUC9NaM0DnVuMwp8\|f5uZv4dTS7Hzf8le5vR9oQx\|z73iakiKUIpDQc\r\nn0\|sUJSKijOQRT
1xZQmkw5+QClmbjLLS3ZXn9kMt0bbxZRPkcV2A=\r\nn-----END RSA PRIVATE KEY-----\r\nn"

6.7 动态upstream配置API说明

swagger: '2.0'

info:

version: '8.0'

title: NJET REST API

description: NJET REST

[API](https://njet.org/en/docs/http/nginx_http_api_module.html)

provides access to NJET status information,

on-the-fly configuration of upstream servers and

key-value pairs management for

[http](https://njet.org/en/docs/http/nginx_http_keyval_module.html) and

[stream](https://njet.org/en/docs/stream/nginx_stream_keyval_module.html).

basePath: /api/8

servers:

- url: '<http://192.168.40.140//api/8>'

tags:

- name: HTTP Upstreams

- name: Stream Upstreams

- name: Method GET

- name: Method POST

- name: Method PATCH
- name: Method DELETE

schemes:

- http
- https

paths:

/http/upstreams/:

get:

tags:

- HTTP Upstreams
- Method GET

summary: Return status of all HTTP upstream server groups

description: Returns status of each HTTP upstream server group and its servers.

operationId: getHttpUpstreams

produces:

- application/json

parameters:

- name: fields

in: query

type: string

description: Limits which fields of upstream server groups will be output.
If the “<literal>fields</literal>” value is empty,
only names of upstreams will be output.

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxHTTPUpstreamMap'

'404':

description: Unknown version (*UnknownVersion*)

schema:

\$ref: '#/definitions/NginxError'

/http/upstreams/{httpUpstreamName}/:

parameters:

- name: httpUpstreamName

in: path

description: The name of an HTTP upstream server group.

required: true

type: string

get:

tags:

- HTTP Upstreams
- Method GET

summary: Return status of an HTTP upstream server group

description: Returns status of a particular HTTP upstream server group and its servers.

operationId: getHttpUpstreamName

produces:

- application/json

parameters:

- name: fields

in: query

type: string

description: Limits which fields of the upstream server group will be output.

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxHTTPUpstream'

'400':

description: Upstream is static (*UpstreamStatic*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),

upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

delete:

tags:

- HTTP Upstreams
- Method DELETE

summary: Reset statistics of an HTTP upstream server group

description: Resets the statistics for each upstream server in an upstream server group and queue statistics.

operationId: deleteHttpUpstreamStat

produces:

- application/json

responses:

'204':

description: Success

'400':

description: Upstream is static (*UpstreamStatic*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),

upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

'/http/upstreams/{httpUpstreamName}/servers/':

parameters:

- name: httpUpstreamName

in: path

description: The name of an upstream server group.

required: true

type: string

get:

tags:

- HTTP Upstreams

- Method GET

summary: Return configuration of all servers in an HTTP upstream server group

description: Returns configuration of each server

in a particular HTTP upstream server group.

operationId: getHttpUpstreamServers

produces:

- application/json

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServerMap'

'400':

description: Upstream is static (*UpstreamStatic*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),

upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

post:

tags:

- HTTP Upstreams

- Method POST

summary: Add a server to an HTTP upstream server group

description: Adds a new server to an HTTP upstream server group.

Server parameters are specified in the JSON format.

operationId: postHttpUpstreamServer

produces:

- application/json

parameters:

- in: body

name: postHttpUpstreamServer

description: Address of a new server and other optional parameters
in the JSON format.

The “*ID*” , “*backup*” , and “*service*” parameters
cannot be changed.

required: true

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

responses:

'201':

description: Created

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),

invalid “**parameter**” value (*UpstreamConfFormatError*),

missing “*server*” argument (*UpstreamConfFormatError*),

unknown parameter “**name**” (*UpstreamConfFormatError*),

nested object or list (*UpstreamConfFormatError*),
“*error*” while parsing (*UpstreamBadAddress*),
service upstream “*host*” may not have port (*UpstreamBadAddress*),
service upstream “*host*” requires domain name (*UpstreamBadAddress*),
invalid “*weight*” (*UpstreamBadWeight*),
invalid “*max_conns*” (*UpstreamBadMaxConns*),
invalid “*max_fails*” (*UpstreamBadMaxFails*),
invalid “*fail_timeout*” (*UpstreamBadFailTimeout*),
invalid “*slow_start*” (*UpstreamBadSlowStart*),
reading request body failed *BodyReadError*),
route is too long (*UpstreamBadRoute*),
“*service*” is empty (*UpstreamBadService*),
no resolver defined to resolve (*UpstreamConfNoResolver*),
upstream “**name**” has no backup (*UpstreamNoBackup*),
upstream “**name**” memory exhausted (*UpstreamOutOfMemory*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

'409':

description: Entry exists (*EntryExists*)

schema:

\$ref: '#/definitions/NginxError'

'415':

description: JSON error (*JsonError*)

schema:

\$ref: '#/definitions/NginxError'

'/http/upstreams/{httpUpstreamName}/servers/{httpUpstreamServerId}':

parameters:

- name: httpUpstreamName

in: path

description: The name of the upstream server group.

required: true
type: string
- name: httpUpstreamServerId
in: path
description: The ID of the server.
required: true
type: string

get:

tags:

- HTTP Upstreams
- Method GET

summary: Return configuration of a server in an HTTP upstream server group

description: Returns configuration of a particular server
in the HTTP upstream server group.

operationId: getHttpUpstreamPeer

produces:

- application/json

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid server ID (*UpstreamBadServerId*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Server with ID “**id**” does not exist (*UpstreamServerNotFound*),
unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

patch:

tags:

- HTTP Upstreams
- Method PATCH

summary: Modify a server in an HTTP upstream server group

description: Modifies settings of a particular server
in an HTTP upstream server group.

Server parameters are specified in the JSON format.

operationId: patchHttpUpstreamPeer

produces:

- application/json

parameters:

- in: body

name: patchHttpUpstreamServer

description: Server parameters, specified in the JSON format.

The “*ID*” , “*backup*” , and “*service*” parameters
cannot be changed.

required: true

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid “**parameter**” value (*UpstreamConfFormatError*),
unknown parameter “**name**” (*UpstreamConfFormatError*),
nested object or list (*UpstreamConfFormatError*),
“*error*” while parsing (*UpstreamBadAddress*),
invalid “*server*” argument (*UpstreamBadAddress*),
invalid server ID (*UpstreamBadServerId*),
invalid “*weight*” (*UpstreamBadWeight*),
invalid “*max_conns*” (*UpstreamBadMaxConns*),
invalid “*max_fails*” (*UpstreamBadMaxFails*),
invalid “*fail_timeout*” (*UpstreamBadFailTimeout*),
invalid “*slow_start*” (*UpstreamBadSlowStart*),
reading request body failed *BodyReadError*),
route is too long (*UpstreamBadRoute*),
“*service*” is empty (*UpstreamBadService*),
server “**ID**” address is immutable (*UpstreamServerImmutable*),
server “*ID*” weight is immutable (*UpstreamServerWeightImmutable*),
upstream “*name*” memory exhausted (*UpstreamOutOfMemory*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Server with ID “**id**” does not exist (*UpstreamServerNotFound*),
unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

'415':

description: JSON error (*JsonError*)

schema:

\$ref: '#/definitions/NginxError'

delete:

tags:

- HTTP Upstreams
- Method DELETE

summary: Remove a server from an HTTP upstream server group

description: Removes a server from an HTTP upstream server group.

operationId: deleteHttpUpstreamServer

produces:

- application/json

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServerMap'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid server ID (*UpstreamBadServerId*),
server “**id**” not removable (*UpstreamServerImmutable*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Server with ID “**id**” does not exist (*UpstreamServerNotFound*),
unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

/stream/upstreams/:

get:

tags:

- Stream Upstreams
- Method GET

summary: Return status of all stream upstream server groups

description: Returns status of each stream upstream server group
and its servers.

operationId: getStreamUpstreams

produces:

- application/json

parameters:

- name: fields

in: query

type: string

description: Limits which fields of upstream server groups will be output.
If the “<literal>fields</literal>” value is empty,
only names of upstreams will be output.

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstreamMap'

'404':

description: Unknown version (*UnknownVersion*)

schema:

\$ref: '#/definitions/NginxError'

/stream/upstreams/{streamUpstreamName}/:

parameters:

- name: streamUpstreamName
- in: path

description: The name of a stream upstream server group.

required: true

type: string

get:

tags:

- Stream Upstreams
- Method GET

summary: Return status of a stream upstream server group

description: Returns status of a particular stream upstream server group and its servers.

operationId: getStreamUpstream

produces:

- application/json

parameters:

- name: fields

in: query

type: string

description: Limits which fields of the upstream server group will be output.

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstream'

'400':

description: Upstream is static (*UpstreamStatic*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),

upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

delete:

tags:

- Stream Upstreams
- Method DELETE

summary: Reset statistics of a stream upstream server group

description: Resets the statistics for each upstream server in an upstream server group.

operationId: deleteStreamUpstreamStat

produces:

- application/json

responses:

'204':

description: Success

'400':

description: Upstream is static (*UpstreamStatic*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),

upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

'/stream/upstreams/{streamUpstreamName}/servers/':

parameters:

- name: streamUpstreamName

in: path

description: The name of an upstream server group.

required: true

type: string

get:

tags:

- Stream Upstreams

- Method GET

summary: Return configuration of all servers in a stream upstream server group

description: Returns configuration of each server

in a particular stream upstream server group.

operationId: getStreamUpstreamServers

produces:

- application/json

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServerMap'

'400':

description: Upstream is static (*UpstreamStatic*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),

upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

post:

tags:

- Stream Upstreams

- Method POST

summary: Add a server to a stream upstream server group

description: Adds a new server to a stream upstream server group.

Server parameters are specified in the JSON format.

operationId: postStreamUpstreamServer

produces:

- application/json

parameters:

- in: body

name: postStreamUpstreamServer

description: Address of a new server and other optional parameters
in the JSON format.

The “*ID*” , “*backup*” , and “*service*” parameters
cannot be changed.

required: true

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

responses:

'201':

description: Created

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),

invalid **“parameter”** value (*UpstreamConfFormatError*),
missing **“*server*”** argument (*UpstreamConfFormatError*),
unknown parameter **“name”** (*UpstreamConfFormatError*),
nested object or list (*UpstreamConfFormatError*),
 “*error*” while parsing (*UpstreamBadAddress*),
no port in server **“*host*”** (*UpstreamBadAddress*),
service upstream **“*host*”** may not have port (*UpstreamBadAddress*),
service upstream **“*host*”** requires domain name (*UpstreamBadAddress*),
invalid **“*weight*”** (*UpstreamBadWeight*),
invalid **“*max_conns*”** (*UpstreamBadMaxConns*),
invalid **“*max_fails*”** (*UpstreamBadMaxFails*),
invalid **“*fail_timeout*”** (*UpstreamBadFailTimeout*),
invalid **“*slow_start*”** (*UpstreamBadSlowStart*),
 “*service*” is empty (*UpstreamBadService*),
no resolver defined to resolve (*UpstreamConfNoResolver*),
upstream **“name”** has no backup (*UpstreamNoBackup*),
upstream **“name”** memory exhausted (*UpstreamOutOfMemory*),
reading request body failed *BodyReadError*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

'409':

description: Entry exists (*EntryExists*)

schema:

\$ref: '#/definitions/NginxError'

'415':

description: JSON error (*JsonError*)

schema:

\$ref: '#/definitions/NginxError'

'/stream/upstreams/{streamUpstreamName}/servers/{streamUpstreamServerId}':

parameters:

- name: streamUpstreamName
in: path
description: The name of the upstream server group.
required: true
type: string
- name: streamUpstreamServerId
in: path
description: The ID of the server.
required: true
type: string

get:

tags:

- Stream Upstreams
- Method GET

summary: Return configuration of a server in a stream upstream server group

description: Returns configuration of a particular server
in the stream upstream server group.

operationId: getStreamUpstreamServer

produces:

- application/json

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid server ID (*UpstreamBadServerId*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*),
server with ID “**id**” does not exist (*UpstreamServerNotFound*)

schema:

\$ref: '#/definitions/NginxError'

patch:

tags:

- Stream Upstreams
- Method PATCH

summary: Modify a server in a stream upstream server group

description: Modifies settings of a particular server
in a stream upstream server group.

Server parameters are specified in the JSON format.

operationId: patchStreamUpstreamServer

produces:

- application/json

parameters:

- in: body

name: patchStreamUpstreamServer

description: Server parameters, specified in the JSON format.

The “*ID*”, “*backup*”, and “*service*” parameters cannot be changed.

required: true

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid “**parameter**” value (*UpstreamConfFormatError*),
unknown parameter “**name**” (*UpstreamConfFormatError*),
nested object or list (*UpstreamConfFormatError*),
“*error*” while parsing (*UpstreamBadAddress*),
invalid “*server*” argument (*UpstreamBadAddress*),
no port in server “*host*” (*UpstreamBadAddress*),
invalid server ID (*UpstreamBadServerId*),
invalid “*weight*” (*UpstreamBadWeight*),
invalid “*max_conns*” (*UpstreamBadMaxConns*),
invalid “*max_fails*” (*UpstreamBadMaxFails*),
invalid “*fail_timeout*” (*UpstreamBadFailTimeout*),
invalid “*slow_start*” (*UpstreamBadSlowStart*),
reading request body failed *BodyReadError*),
“*service*” is empty (*UpstreamBadService*),

server “ID” address is immutable (*UpstreamServerImmutable*),
server “ID” weight is immutable (*UpstreamServerWeightImmutable*),
upstream “*name*” memory exhausted (*UpstreamOutOfMemory*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Server with ID “id” does not exist (*UpstreamServerNotFound*),
unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

'415':

description: JSON error (*JsonError*)

schema:

\$ref: '#/definitions/NginxError'

delete:

tags:

- Stream Upstreams
- Method DELETE

summary: Remove a server from a stream upstream server group

description: Removes a server from a stream server group.

operationId: deleteStreamUpstreamServer

produces:

- application/json

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServerMap'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid server ID (*UpstreamBadServerId*),
server “id” not removable (*UpstreamServerImmutable*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Server with ID “**id**” does not exist (*UpstreamServerNotFound*),
unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

###

###DEFINITIONS

###

definitions:

ArrayOfStrings:

title: Array

description: |

An array of strings.

type: array

items:

type: string

NginxHTTPUpstreamMap:

title: HTTP Upstreams

description: |

Status information of all HTTP

dynamically
configurable

groups.

type: object

additionalProperties:

\$ref: '#/definitions/NginxHTTPUpstream'

example:

trac-backend:

peers:

- id: 0

server: 10.0.0.1:8088
name: 10.0.0.1:8088
backup: false
weight: 5
state: up
active: 0
ssl:
 handshakes: 620311
 handshakes_failed: 3432
 session_reuses: 36442
 no_common_protocol: 4
 handshake_timeout: 0
 peer_rejected_cert: 0
 verify_failures:
 expired_cert: 2
 revoked_cert: 1
 hostname_mismatch: 2
 other: 1
requests: 667231
header_time: 20
response_time: 36
responses:
 1xx: 0
 2xx: 666310
 3xx: 0
 4xx: 915
 5xx: 6
codes:
 200: 666310
 404: 915
 503: 6
total: 667231
sent: 251946292
received: 19222475454
fails: 0
unavail: 0
health_checks:
 checks: 26214
 fails: 0
 unhealthy: 0

last_passed: true
downtime: 0
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z
- id: 1
server: 10.0.0.1:8089
name: 10.0.0.1:8089
backup: true
weight: 1
state: unhealthy
active: 0
requests: 0
responses:
 1xx: 0
 2xx: 0
 3xx: 0
 4xx: 0
 5xx: 0
 codes: {}
 total: 0
sent: 0
received: 0
fails: 0
unavail: 0
health_checks:
 checks: 26284
 fails: 26284
 unhealthy: 1
 last_passed: false
downtime: 262925617
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z
keepalive: 0
zombies: 0
zone: trac-backend
hg-backend:
peers:
 - id: 0
 server: 10.0.0.1:8088
 name: 10.0.0.1:8088

backup: false
weight: 5
state: up
active: 0
ssl:
 handshakes: 620311
 handshakes_failed: 3432
 session_reuses: 36442
 no_common_protocol: 4
 handshake_timeout: 0
 peer_rejected_cert: 0
 verify_failures:
 expired_cert: 2
 revoked_cert: 1
 hostname_mismatch: 2
 other: 1
requests: 667231
header_time: 20
response_time: 36
responses:
 1xx: 0
 2xx: 666310
 3xx: 0
 4xx: 915
 5xx: 6
codes:
 200: 666310
 404: 915
 503: 6
total: 667231
sent: 251946292
received: 19222475454
fails: 0
unavail: 0
health_checks:
 checks: 26214
 fails: 0
 unhealthy: 0
 last_passed: true
downtime: 0

downstart: 2022-06-28T11:09:21.602Z

selected: 2022-06-28T15:01:25Z

- id: 1

server: 10.0.0.1:8089

name: 10.0.0.1:8089

backup: true

weight: 1

state: unhealthy

active: 0

requests: 0

responses:

1xx: 0

2xx: 0

3xx: 0

4xx: 0

5xx: 0

codes: {}

total: 0

sent: 0

received: 0

fails: 0

unavail: 0

health_checks:

checks: 26284

fails: 26284

unhealthy: 1

last_passed: false

downtime: 262925617

downstart: 2022-06-28T11:09:21.602Z

selected: 2022-06-28T15:01:25Z

keepalive: 0

zombies: 0

zone: hg-backend

NginxHTTPUpstream:

title: HTTP Upstream

type: object

properties:

peers:

\$ref: '#/definitions/NginxHTTPUpstreamPeerMap'

keepalive:

type: integer

description: The current number of idle

keepalive

connections.

zombies:

type: integer

description: The current number of servers removed

from the group but still processing active client requests.

zone:

type: string

description: The name of the shared memory

zone

that keeps the group's configuration and run-time state.

queue:

type: object

description: >

For the requests

queue,

the following data are provided:

properties:

size:

type: integer

description: The current number of requests in the queue.

max_size:

type: integer

description: The maximum number of requests that can be in the queue
at the same time.

overflows:

type: integer

description: The total number of requests rejected due to the queue overflow.

example:

upstream_backend:

peers:

- id: 0

server: 10.0.0.1:8088

name: 10.0.0.1:8088

backup: false
weight: 5
state: up
active: 0
ssl:
 handshakes: 620311
 handshakes_failed: 3432
 session_reuses: 36442
 no_common_protocol: 4
 handshake_timeout: 0
 peer_rejected_cert: 0
 verify_failures:
 expired_cert: 2
 revoked_cert: 1
 hostname_mismatch: 2
 other: 1
max_conns: 20
requests: 667231
header_time: 20
response_time: 36
responses:
 1xx: 0
 2xx: 666310
 3xx: 0
 4xx: 915
 5xx: 6
 codes:
 200: 666310
 404: 915
 503: 6
 total: 667231
sent: 251946292
received: 19222475454
fails: 0
unavail: 0
health_checks:
 checks: 26214
 fails: 0
 unhealthy: 0
last_passed: true

downtime: 0
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z

- id: 1

server: 10.0.0.1:8089

name: 10.0.0.1:8089

backup: true

weight: 1

state: unhealthy

active: 0

max_conns: 20

requests: 0

responses:

1xx: 0

2xx: 0

3xx: 0

4xx: 0

5xx: 0

codes: {}

total: 0

sent: 0

received: 0

fails: 0

unavail: 0

health_checks:

checks: 26284

fails: 26284

unhealthy: 1

last_passed: false

downtime: 262925617

downstart: 2022-06-28T11:09:21.602Z

selected: 2022-06-28T15:01:25Z

keepalive: 0

zombies: 0

zone: upstream_backend

NginxHTTPUpstreamPeerMap:

title: HTTP Upstream Servers

description: |

An array of HTTP

`<a href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#upstream">upstream servers</a>.`

type: array

items:

- `$ref: '#/definitions/NginxHTTPUpstreamPeer'`

NginxHTTPUpstreamPeer:

title: HTTP Upstream Server

type: object

properties:

- id:
 - type: integer
 - description: The ID of the server.
 - readOnly: true
- server:
 - type: string
 - description: An `<a href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#server">address</a>` of the server.
- service:
 - type: string
 - description: The `<a href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#service">service</a>` parameter value of the `<a href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#server">server</a>` directive.
- name:
 - type: string
 - description: The name of the server specified in the `<a href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#server">server</a>` directive.
- readOnly: true
- backup:
 - type: boolean
 - description: A boolean value indicating whether the server is a `<a href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#backup">backup</a>`

server.

weight:

type: integer

description: Weight

of the server.

state:

type: string

enum:

- up
- draining
- down
- unavail
- checking
- unhealthy

description: Current state, which may be one of

“<code>up</code>” , “<code>draining</code>” , “<code>down</code>” ,
“<code>unavail</code>” , “<code>checking</code>” ,
and “<code>unhealthy</code>” .

active:

type: integer

description: The current number of active connections.

readOnly: true

ssl:

type: object

readOnly: true

properties:

handshakes:

type: integer

description: The total number of successful SSL handshakes.

readOnly: true

handshakes_failed:

type: integer

description: The total number of failed SSL handshakes.

readOnly: true

session_reuses:

type: integer

description: The total number of session reuses during SSL handshake.

readOnly: true

no_common_protocol:

type: integer

description: The number of SSL handshakes failed because of no common protocol.

handshake_timeout:

type: integer

description: The number of SSL handshakes failed because of a timeout.

peer_rejected_cert:

type: integer

description: The number of failed SSL handshakes when njet presented the certificate to the upstream server but it was rejected with a corresponding alert message.

verify_failures:

type: object

description: SSL certificate verification errors

properties:

expired_cert:

type: integer

description: An expired or not yet valid certificate was presented by an upstream server.

revoked_cert:

type: integer

description: A revoked certificate was presented by an upstream server.

hostname_mismatch:

type: integer

description: Server's certificate doesn't match the hostname.

other:

type: integer

description: Other SSL certificate verification errors.

max_conns:

type: integer

description: The

max_conns

limit for the server.

requests:

type: integer

description: The total number of client requests forwarded to this server.

readOnly: true

responses:

type: object

readOnly: true

properties:

1xx:

type: integer

description: The number of responses with “<code>1xx</code>” status codes.

readOnly: true

2xx:

type: integer

description: The number of responses with “<code>2xx</code>” status codes.

readOnly: true

3xx:

type: integer

description: The number of responses with “<code>3xx</code>” status codes.

readOnly: true

4xx:

type: integer

description: The number of responses with “<code>4xx</code>” status codes.

readOnly: true

5xx:

type: integer

description: The number of responses with “<code>5xx</code>” status codes.

readOnly: true

codes:

type: object

description: The number of responses per each status code.

readOnly: true

properties:

codeNumber:

type: integer

description: The number of responses with this particular status code.

readOnly: true

total:

type: integer

description: The total number of responses obtained from this server.

readOnly: true

sent:

type: integer

description: The total number of bytes sent to this server.

readOnly: true

received:

type: integer

description: The total number of bytes received from this server.

readOnly: true

fails:

type: integer

description: The total number of unsuccessful attempts to communicate with the server.

readOnly: true

unavail:

type: integer

description: How many times the server became unavailable for client requests (state “`unavail`”) due to the number of unsuccessful attempts reaching the

max_fails

threshold.

readOnly: true

health_checks:

type: object

readOnly: true

properties:

checks:

type: integer

description: The total number of

h

ealth check

requests made.

fails:

type: integer

description: The number of failed health checks.

unhealthy:

type: integer

description: How many times the server became unhealthy (state “`unhealthy`”).

last_passed:

type: boolean

description: Boolean indicating if the last health check request was successful and passed

tests

.

downtime:

type: integer

readOnly: true

description: Total time the server was in the “<code>unavail</code>” , “<code>checking</code>” , and “<code>unhealthy</code>” states.

downstart:

type: string

format: date-time

readOnly: true

description: The time when the server became “<code>unavail</code>” , “<code>checking</code>” , or “<code>unhealthy</code>” , in the ISO 8601 format with millisecond resolution.

selected:

type: string

format: date-time

readOnly: true

description: The time when the server was last selected to process a request, in the ISO 8601 format with millisecond resolution.

header_time:

type: integer

readOnly: true

description: The average time to get the

response header

from the server.

response_time:

type: integer

readOnly: true

description: The average time to get the

full response

from the server.

NginxHTTPUpstreamConfServerMap:

title: HTTP Upstream Servers

description: An array of HTTP upstream servers for dynamic configuration.

type: array

items:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

example:

- id: 0

server: 10.0.0.1:8088

weight: 1

max_conns: 0

max_fails: 0

fail_timeout: 10s

slow_start: 10s

route: ''

backup: false

down: false

- id: 1

server: 10.0.0.1:8089

weight: 4

max_conns: 0

max_fails: 0

fail_timeout: 10s

slow_start: 10s

route: ''

backup: true

down: true

NginxHTTPUpstreamConfServer:

title: HTTP Upstream Server

description: |

Dynamically configurable parameters of an HTTP upstream

server:

type: object

properties:

id:

type: integer

description: The ID of the HTTP upstream server.

The ID is assigned automatically and cannot be changed.

readOnly: true

server:

type: string

description: Same as the

address

>

parameter of the HTTP upstream server.

When adding a server, it is possible to specify it as a domain name.

In this case, changes of the IP addresses

that correspond to a domain name will be monitored and automatically

applied to the upstream configuration

without the need of restarting njet.

This requires the

resolver

directive in the “<code>http</code>” block.

See also the

resolve

parameter of the HTTP upstream server.

service:

type: string

description: Same as the

service

parameter of the HTTP upstream server.

This parameter cannot be changed.

readOnly: true

weight:

type: integer

description: Same as the

weight

parameter of the HTTP upstream server.

max_conns:

type: integer

description: Same as the

max_co

nns

parameter of the HTTP upstream server.

max_fails:

type: integer

description: Same as the

max_fails

parameter of the HTTP upstream server.

fail_timeout:

type: string

description: Same as the

fail_t
imeout

parameter of the HTTP upstream server.

slow_start:

type: string

description: Same as the

slow_sta
rt

parameter of the HTTP upstream server.

route:

type: string

description: Same as the

route

parameter of the HTTP upstream server.

backup:

type: boolean

description: When <code>>true</code>, adds a

backup
server.

This parameter cannot be changed.

readOnly: true

down:

type: boolean

description: Same as the

down

parameter of the HTTP upstream server.

drain:

type: boolean

description: Same as the

drain

parameter of the HTTP upstream server.

parent:

type: string

description: Parent server ID of the resolved server.

The ID is assigned automatically and cannot be changed.

readOnly: true

host:

type: string

description: Hostname of the resolved server.

The hostname is assigned automatically and cannot be changed.

readOnly: true

example:

id: 1

server: 10.0.0.1:8089

weight: 4

max_conns: 0

max_fails: 0

fail_timeout: 10s

slow_start: 10s

route: ''

backup: true

down: true

NginxStreamUpstreamMap:

title: Stream Upstreams

description: Status information of stream upstream server groups.

type: object

additionalProperties:

\$ref: '#/definitions/NginxStreamUpstream'

example:

mysql_backends:

peers:

- id: 0

server: 10.0.0.1:12345
name: 10.0.0.1:12345
backup: false
weight: 5
state: up
active: 0
ssl:
 handshakes: 1045
 handshakes_failed: 89
 session_reuses: 321
 no_common_protocol: 4
 handshake_timeout: 0
 peer_rejected_cert: 0
 verify_failures:
 expired_cert: 2
 revoked_cert: 1
 hostname_mismatch: 2
 other: 1
max_conns: 30
connecions: 1231
sent: 251946292
received: 19222475454
fails: 0
unavail: 0
health_checks:
 checks: 26214
 fails: 0
 unhealthy: 0
 last_passed: true
downtime: 0
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z

- id: 1

server: 10.0.0.1:12346
name: 10.0.0.1:12346
backup: true
weight: 1
state: unhealthy
active: 0
max_conns: 30

connections: 0
sent: 0
received: 0
fails: 0
unavail: 0
health_checks:
 checks: 26284
 fails: 26284
 unhealthy: 1
 last_passed: false
downtime: 262925617
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z
zombies: 0
zone: mysql_backends
dns:
peers:
 - id: 0
 server: 10.0.0.1:12347
 name: 10.0.0.1:12347
 backup: false
 weight: 5
 state: up
 active: 0
 ssl:
 handshakes: 5268
 handshakes_failed: 121
 session_reuses: 854
 no_common_protocol: 4
 handshake_timeout: 0
 peer_rejected_cert: 0
 verify_failures:
 expired_cert: 2
 revoked_cert: 1
 hostname_mismatch: 2
 other: 1
max_conns: 30
connections: 667231
sent: 251946292
received: 19222475454

fails: 0
unavail: 0
health_checks:
 checks: 26214
 fails: 0
 unhealthy: 0
 last_passed: true
downtime: 0
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z

- id: 1

server: 10.0.0.1:12348
name: 10.0.0.1:12348
backup: true
weight: 1
state: unhealthy
active: 0
connections: 0
max_conns: 30
sent: 0

received: 0
fails: 0
unavail: 0
health_checks:
 checks: 26284
 fails: 26284
 unhealthy: 1
 last_passed: false
downtime: 262925617
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z

zombies: 0
zone: dns

NginxStreamUpstream:

 title: Stream Upstream

 type: object

 properties:

 peers:

 \$ref: '#/definitions/NginxStreamUpstreamPeerMap'

 zombies:

type: integer

description: The current number of servers removed from the group
but still processing active client connections.

zone:

type: string

description: The name of the shared memory

zone

that keeps the group's configuration and run-time state.

example:

dns:

peers:

- id: 0

server: 10.0.0.1:12347

name: 10.0.0.1:12347

backup: false

weight: 5

state: up

active: 0

ssl:

handshakes: 200

handshakes_failed: 4

session_reuses: 189

no_common_protocol: 4

handshake_timeout: 0

peer_rejected_cert: 0

verify_failures:

expired_cert: 2

revoked_cert: 1

hostname_mismatch: 2

other: 1

max_conns: 50

connections: 667231

sent: 251946292

received: 19222475454

fails: 0

unavail: 0

health_checks:

checks: 26214

fails: 0
unhealthy: 0
last_passed: true
downtime: 0
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z

- id: 1

server: 10.0.0.1:12348

name: 10.0.0.1:12348

backup: true

weight: 1

state: unhealthy

active: 0

max_conns: 50

connections: 0

sent: 0

received: 0

fails: 0

unavail: 0

health_checks:

checks: 26284

fails: 26284

unhealthy: 1

last_passed: false

downtime: 262925617

downstart: 2022-06-28T11:09:21.602Z

selected: 2022-06-28T15:01:25Z

zombies: 0

zone: dns

NginxStreamUpstreamPeerMap:

title: Stream Upstream Servers

description: Array of stream upstream servers.

type: array

items:

\$ref: '#/definitions/NginxStreamUpstreamPeer'

NginxStreamUpstreamPeer:

title: Stream Upstream Server

type: object

properties:

id:

type: integer

description: The ID of the server.

readOnly: true

server:

type: string

description: An

address

of the server.

service:

type: string

description: The

service

parameter value of the

server

directive.

name:

type: string

format: hostname

description: The name of the server specified in the

server

directive.

readOnly: true

backup:

type: boolean

description: A boolean value indicating whether the server is a

backup

server.

weight:

type: integer

description: Weight

of the server.

state:

type: string

readOnly: true

enum:

- up
- down
- unavail
- checking
- unhealthy

description: Current state, which may be one of

“<code>up</code>” , “<code>down</code>” , “<code>unavail</code>” , “<code>checking</code>” , or “<code>unhealthy</code>” .

active:

type: integer

description: The current number of connections.

readOnly: true

ssl:

type: object

readOnly: true

properties:

handshakes:

type: integer

description: The total number of successful SSL handshakes.

readOnly: true

handshakes_failed:

type: integer

description: The total number of failed SSL handshakes.

readOnly: true

session_reuses:

type: integer

description: The total number of session reuses during SSL handshake.

readOnly: true

no_common_protocol:

type: integer

description: The number of SSL handshakes failed because of no common protocol.

handshake_timeout:

type: integer

description: The number of SSL handshakes failed because of a timeout.

peer_rejected_cert:

type: integer

description: The number of failed SSL handshakes when njet presented the certificate to the upstream server but it was rejected with a corresponding alert message.

verify_failures:

type: object

description: SSL certificate verification errors

properties:

expired_cert:

type: integer

description: An expired or not yet valid certificate was presented by an upstream server.

revoked_cert:

type: integer

description: A revoked certificate was presented by an upstream server.

hostname_mismatch:

type: integer

description: Server's certificate doesn't match the hostname.

other:

type: integer

description: Other SSL certificate verification errors.

max_conns:

type: integer

description: The

m

ax_conns

limit for the server.

connections:

type: integer

description: The total number of client connections forwarded to this server.

readOnly: true

connect_time:

type: integer

description: The average time to connect to the upstream server.

readOnly: true

first_byte_time:

type: integer

description: The average time to receive the first byte of data.

readOnly: true

response_time:

type: integer

description: The average time to receive the last byte of data.

readOnly: true

sent:

type: integer

description: The total number of bytes sent to this server.

readOnly: true

received:

type: integer

description: The total number of bytes received from this server.

readOnly: true

fails:

type: integer

description: The total number of unsuccessful attempts
to communicate with the server.

readOnly: true

unavail:

type: integer

description: How many times the server became unavailable for client connections
(state “<code>unavail</code>”) due to the number of unsuccessful
attempts reaching the
max
_fails

threshold.

readOnly: true

health_checks:

type: object

readOnly: true

properties:

checks:

type: integer

description: The total number of

[health check](https://njet.org/en/docs/stream/nginx_stream_upstream_hc_module.html#health_check)
requests made.
readOnly: true
fails:
type: integer
description: The number of failed health checks.
readOnly: true
unhealthy:
type: integer
description: How many times the server became unhealthy (state “`unhealthy`”).
readOnly: true
last_passed:
type: boolean
description: Boolean indicating whether the last health check request was successful and passed
[test](https://njet.org/en/docs/stream/nginx_stream_upstream_hc_module.html#match)
s.
readOnly: true
downtime:
type: integer
description: Total time the server was in the “`unavail`”, “`checking`”, and “`unhealthy`” states.
readOnly: true
downstart:
type: string
format: date-time
description: The time when the server became “`unavail`”, “`checking`”, or “`unhealthy`”, in the ISO 8601 format with millisecond resolution.
readOnly: true
selected:
type: string
format: date-time
description: The time when the server was last selected

to process a connection,
in the ISO 8601 format with millisecond resolution.

readOnly: true

NginxStreamUpstreamConfServerMap:

title: Stream Upstream Servers

description: |

An array of stream upstream servers for dynamic configuration.

type: array

items:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

example:

- id: 0

server: 10.0.0.1:12348

weight: 1

max_conns: 0

max_fails: 1

fail_timeout: 10s

slow_start: 0

backup: false

down: false

- id: 1

server: 10.0.0.1:12349

weight: 1

max_conns: 0

max_fails: 1

fail_timeout: 10s

slow_start: 0

backup: false

down: false

NginxStreamUpstreamConfServer:

title: Stream Upstream Server

description: |

Dynamically configurable parameters of a stream upstream

server:

type: object

properties:

id:

type: integer

description: The ID of the stream upstream server.

The ID is assigned automatically and cannot be changed.

readOnly: true

server:

type: string

description: Same as the

addresses

parameter of the stream upstream server.

When adding a server, it is possible to specify it as a domain name.

In this case, changes of the IP addresses

that correspond to a domain name will be monitored and automatically

applied to the upstream configuration

without the need of restarting njet.

This requires the

resolver

directive in the “<code>stream</code>” block.

See also the

resolve

parameter of the stream upstream server.

service:

type: string

description: Same as the

service

parameter of the stream upstream server.

This parameter cannot be changed.

readOnly: true

weight:

type: integer

description: Same as the

weight

parameter of the stream upstream server.

max_conns:

type: integer

description: Same as the

m

ax_conns

parameter of the stream upstream server.

max_fails:

type: integer

description: Same as the

max

_fails

parameter of the stream upstream server.

fail_timeout:

type: string

description: Same as the

f

ail_timeout

parameter of the stream upstream server.

slow_start:

type: string

description: Same as the

slo

w_start

parameter of the stream upstream server.

backup:

type: boolean

description: When <code>true</code>, adds a

backu

p

server.

This parameter cannot be changed.

readOnly: true

down:

type: boolean

description: Same as the

down</

a>

parameter of the stream upstream server.

parent:

type: string

description: Parent server ID of the resolved server.

The ID is assigned automatically and cannot be changed.

readOnly: true

host:

type: string

description: Hostname of the resolved server.

The hostname is assigned automatically and cannot be changed.

readOnly: true

example:

id: 0

server: 10.0.0.1:12348

weight: 1

max_conns: 0

max_fails: 1

fail_timeout: 10s

slow_start: 0

backup: false

down: false

NginxError:

title: Error

description: |

njet error object.

type: object

properties:

error:

type: object

properties:

status:

type: integer

description: HTTP error code.

text:

type: string

description: Error description.

code:
type: string
description: Internal njet error code.

request_id:
type: string
description: The ID of the request, equals the value of the
<a

href="[\\$request_id](https://njet.org/en/docs/http/nginx_http_core_module.html#var_request_id)

variable.

href:
type: string
description: Link to reference documentation.

6.8 动态location配置API说明

openapi: 3.0.3

info:

title: njet-api
description: njet-api
version: 1.0.0

servers:

- url: '/'

paths:

/dyn_loc:

put:

tags:

- dyn_loc

summary: Delete a dynamic location from server

description: Delete a dynamic location from server.

operationId: delLocation

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/del_location'

post:

tags:

- dyn_loc

summary: Add a dynamic location to server

description: Add a dynamic location to server.

operationId: addLocation

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/add_location'

###DEFINITIONS

components:

schemas:

del_location:

title: dyn_loc

description: delete dynamic location from server

type: object

properties:

type:

type: string

description: fix value "del".

example: del

addr_port:

type: string

description: addr and port of server.

example: 192.168.40.203:8081

server_name:

type: string

description: server name .

example: cluster.tmlake.com

location_rule:

type: string

description: Regular Expression.

example: =

location_name:

type: string

description: the name of location.

example: /clb

add_location:

title: dyn_loc

description: add dynamic location to server

type: object

properties:

type:

type: string

example: add

addr_port:

type: string

description: addr and port of server.

example: 192.168.40.203:8081

server_name:

type: string

description: server name .

example: cluster.tmlake.com

location_rule:

type: string

description: Regular Expression.

example: =

location_name:

type: string

description: the name of location.

example: /clb

location_body:

type: string

description: the body content of location .

example: api write=off;

proxy_pass:

type: string

description: the proxy_pass info .

example: \$dest_upstream;

6.9 动态sendmsg模块kv API说明

openapi: 3.0.3

info:

title: kv api

description: 维护kv

version: 1.0.0

servers:

- url: '/'

paths:

/kv:

post:

tags:

- sendmsg kv

summary: kvstore key set

requestBody:

content:

application/json:

schema:

\$ref: '#/components/schemas/kvdata'

required: true

responses:

'200':

description: Success

'500':

description: Internal Server Error

get:

tags:

- sendmsg kv

summary: get value from kvstore

parameters:

- in: query

name: key

schema:

type: string

example: 'test_key'

responses:

'200':

description: Success

content:

text/plain:

schema:

type: string

example: 'test msg'

'404':

description: Not Found

'500':

description: Internal Server Error

components:

schemas:

kvdata:

title: kv post data

type: object

properties:

key:

type: string

example: 'test_key'

value:

type: string

example: 'test msg'

7. 版权声明

版权所有 © 通明智云(北京)科技有限公司2020 保留一切权利

本文档的程序或内容受版权法的保护，未经通明智云(北京)科技有限公司的书面许可，不得擅自摘抄、复制本文档资料的全部或部分内容，并不得以任何形式传播。

注意

您购买的产品、服务或特性等应受通明智云(北京)科技有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，通明智云(北京)科技有限公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

通明智云(北京)科技有限公司

地址：北京市北京经济技术开发区科谷一街8号院1号楼24层2403

网址：<http://www.tmlake.com>